

Digital Solutions 2025

Creating and Innovating in the Digital Age

First Edition

Nathaniel Brown & Google AI (Gemini)



Digital Solutions 2025

Creating and Innovating in the Digital Age

First Edition



This version of can be viewed and downloaded free of charge for personal use only. It must not be redistributed, sold, or used in derivative works.

© Nathaniel Brown & Google AI (Gemini), 2026.

Table of contents

Part 1 – Unit 1: Creating With Code	10
1. Understanding Digital Problems	12
1.1. Computational Thinking and Thinking Tools	13
1.1.1. The Four Pillars of Computational Thinking	13
1.1.2. Methods of Decomposition	13
1.1.3. Thinking Tools for Analysis	15
1.1.4. Logical Relationships	17
1.2. Identifying Human Needs and Opportunities	17
1.2.1. Defining Needs, Wants, and Opportunities	17
1.2.2. Essential Elements of a Solution	18
1.2.3. Modern Problem Contexts	18
1.3. Analysing the Solution Environment	22
1.3.1. Scope, Constraints, and Limitations	22
1.3.2. Solution Requirements	23
1.3.3. User Perspective and UX Requirements	24
1.3.4. Technical Interface Constraints	26
1.3.5. Measurable Success Criteria	26
1.4. Social and Economic Impacts	26
1.4.1. Impact Analysis Framework	26
1.4.2. Evaluating Existing Solutions	27
1.4.3. Emerging Technology Trends	27
1.5. Professional Communication and Documentation	30
1.5.1. Technology-Specific Language	30
1.5.2. Visualizing Ideas	30
1.5.3. Communication Modes	32
1.5.4. Ethical Scholarship	32
1.6. Revision Questions	33
2. User Experiences and Interfaces	34
2.1. The Foundation of User Experience (UX)	35
2.1.1. Driven by People	35
2.2. Usability Principles	37
2.2.1. 1. Accessibility	37
2.2.2. 2. Effectiveness	38
2.2.3. 3. Safety	38

2.2.4. 4. Utility	40
2.2.5. 5. Learnability	40
2.3. Visual Communication in Design	43
2.3.1. Elements of Visual Communication	43
2.3.2. Principles of Visual Communication	44
2.4. Prototyping and Symbolising Ideas	47
2.4.1. The Iterative Design Cycle	47
2.5. Evaluation and Refinement	49
2.5.1. Methods of Appraisal	49
2.6. Revision Questions	53
3. Algorithms and Programming Techniques	54
3.1. Core Algorithmic Concepts	55
3.1.1. Fundamental Constructs	56
3.2. Algorithmic Representation	58
3.2.1. Pseudocode Conventions	58
3.3. Programming Features and Practices	59
3.3.1. Data Structures: Arrays	59
3.3.2. Quality and Dependability	60
3.4. Data Interaction and Advanced Techniques	62
3.4.1. Database Interaction	62
3.4.2. Data Validation	62
3.4.3. Cybersecurity	62
3.5. Revision Questions	63
4. Programmed Solutions	64
4.1. Implementation of Logic and Operations	65
4.1.1. Mathematical and Logical Operators	65
4.1.2. Variables, Constants, and Scope	65
4.1.3. Control Structures and Modular Programming	67
4.2. Generating the Prototype Solution	70
4.2.1. Component Development and Data Handling	70
4.3. Testing, Evaluation, and Impact Analysis	70
4.3.1. Technical Debugging and Performance	70
4.3.2. User-Centric Review and Broader Impacts	72
4.4. Revision Questions	75
Part 2 – Unit 2: Application and Data Solutions	76
5. Data-driven Problems and Solution Requirements	78

5.1. Analyzing Data Challenges	80
5.1.1. Input Issues and Variations	80
5.1.2. Integrity and Reliability	80
5.1.3. Decomposition and Pattern Recognition	82
5.2. System Modeling and Relationships	84
5.2.1. Relational Concepts and Keys	84
5.3. Design and Impact Considerations	90
5.3.1. Impact Analysis	90
5.3.2. Emerging Technologies	90
5.4. Revision Questions	93
6. Data and Programming Techniques	94
6.1. Core Elements of Data-Driven Solutions	95
6.1.1. Problem Scope and Constraints	95
6.1.2. Technical Requirements	95
6.1.3. Data Acquisition	96
6.1.4. Social and Economic Impacts	96
6.1.5. The Conceptual Hierarchy: DIKW	96
6.2. Principles of Data Management	97
6.2.1. Acquisition, Integrity, and Security	97
6.2.2. Organization and Standards	98
6.2.3. Data Anomalies and Redundancy	98
6.2.4. Advanced Processing Techniques	98
6.2.5. Validation vs. Verification	98
6.3. Relational Database Fundamentals	98
6.3.1. Relational Schema (RS)	98
6.3.2. Tables and Components	98
6.3.3. Referential Integrity	99
6.3.4. Normalisation	99
6.4. Algorithmic Logic and Symbolic Representation	100
6.4.1. Visual Representation and Flow	100
6.4.2. Computational Thinking and Algorithm Design	101
6.4.3. Pseudocode Development	101
6.4.4. Core Programming Constructs	101
6.4.5. Code Clarity and Documentation	101
6.5. Revision Questions	101
7. Prototype Data Solutions	102
7.1. Solution Planning and Requirements	103

7.1.1. Success Criteria	103
7.1.2. Data Types and Constraints	103
7.1.3. Defining Relationships	103
7.2. Database Optimization and SQL Implementation	104
7.2.1. Optimization: Reaching 3NF	104
7.2.2. Masterful Data Retrieval	106
7.2.3. Advanced Functions and Joins	106
7.2.4. Data Modification (CRUD)	106
7.3. Generating the Prototype Solution	107
7.3.1. Access, Manipulation, and Display	107
7.3.2. CRUD Operations	107
7.3.3. User Interface (UI) Generation	107
7.4. Validation and Technical Testing	107
7.4.1. Data Validation	107
7.4.2. Technical Quality	107
7.4.3. Usability Testing	108
7.5. Evaluation and Refinement	109
7.5.1. Algorithmic Appraisal	109
7.5.2. Data Quality Evaluation	109
7.5.3. Measuring Success	110
7.6. Revision Questions	110
Part 3 – Unit 3: Digital Innovation	112
8. Interactions Between Users, Data, and Digital Systems	114
8.1. Innovation and Emerging Technologies	115
8.1.1. The Meaning of Innovation	115
8.1.2. Emerging Technologies	115
8.1.3. Personal, Social, and Economic Impacts	116
8.2. Digital Solution Components and Contexts	117
8.2.1. Web Applications	117
8.2.2. Mobile Applications	118
8.2.3. Interactive Media	118
8.2.4. Intelligent Systems	118
8.3. System Analysis and Modeling	118
8.3.1. Identifying System Components	118
8.3.2. Human-Digital Interactions	118
8.3.3. Technical Specifications	118

8.3.4. Modeling Processes	119
8.4. Usability and Visual Communication	119
8.4.1. Core Usability Principles	119
8.4.2. Interface Appraisal	119
8.4.3. Visual Communication	120
8.5. Data Flow and Advanced Processes	121
8.5.1. Data Flow Diagrams (DFD)	121
8.5.2. Advanced Data Handling	122
8.5.3. Prototyping Synthesis	122
8.6. Revision Questions	122
9. Real-world Problems and Solution Requirements	124
9.1. Problem Analysis and Scoping	125
9.1.1. Breaking Down the Problem	125
9.1.2. Scope, Constraints, and Limitations	125
9.1.3. Requirements and Success Criteria	125
9.1.4. Stakeholders and Policies	125
9.2. Project Management and Development Tools	126
9.2.1. Programming Development Tools	126
9.2.2. Managing the Project	126
9.2.3. Evaluating Ideas	128
9.3. System Interactions and Data Structures	129
9.3.1. Input, Process, Output (IPO)	129
9.3.2. Human-System Interaction	130
9.3.3. Data Structures and Sources	130
9.4. Algorithmic Design and Computational Thinking	131
9.4.1. The Thinking Process	131
9.4.2. Basic Constructs	132
9.4.3. Drafting and Documentation	132
9.4.4. The Technical Proposal	133
9.4.5. Styles of Presentation	133
9.4.6. Tailoring the Message	133
9.4.7. Visual Communication in Proposals	133
9.5. Revision Questions	134
10. Innovative Digital Solutions	136
10.1. Prototyping and Visual Design	137
10.1.1. Idea Refinement	137
10.1.2. Low-Fidelity UI Prototypes	137

10.1.3. Visual Communication Principles	137
10.1.4. Demonstration	137
10.2. Technical System Implementation	138
10.2.1. Programmed Modules	138
10.2.2. File Operations	139
10.2.3. Control and Documentation	139
10.3. Advanced Data Manipulation with SQL	140
10.3.1. Data Definition (DDL)	140
10.3.2. Data Modification (DML)	140
10.3.3. Complex Retrieval	141
10.3.4. Justification	141
10.4. Systems Thinking and Synthesis	141
10.4.1. Conceptual Modeling	141
10.4.2. Component Synthesis	142
10.5. Testing, Evaluation, and Refinement	143
10.5.1. Algorithm Appraisal	143
10.5.2. Technical Debugging	143
10.5.3. Usability Testing	143
10.5.4. Impact and Success Evaluation	143
10.6. Revision Questions	144
Part 4 – Unit 4: Digital Impacts	146
11. Digital Methods for Exchanging Data	148
11.1. Security and Authentication Strategies	149
11.1.1. Identity and Authentication	149
11.1.2. Symmetric Encryption	149
11.1.3. Asymmetric Encryption: The RSA Algorithm	150
11.1.4. Storage and Transfer Techniques	151
11.2. Encryption Algorithms and Analysis	152
11.2.1. Classical Encryption Methods	152
11.2.2. Modern Algorithmic Contexts	152
11.2.3. Algorithmic Lifecycle	152
11.3. Privacy, Ethics, and Data Management	152
11.3.1. Australian Privacy Principles (2014)	152
11.3.2. Data Stewardship	152
11.3.3. Design for Users	153
11.4. Network Principles and Performance	153

11.4.1.	Performance Metrics	153
11.4.2.	Transmission Principles	153
11.4.3.	Protocols and Standards	153
11.4.4.	Quality of Service (QoS)	153
11.4.5.	The CIA Triad	153
11.5.	Modern Data Exchange Methods	155
11.5.1.	REST, JSON, and XML	155
11.5.2.	APIs	155
11.5.3.	Front-End and Back-End Collaboration	156
11.5.4.	Modeling with DFDs	156
11.6.	Revision Questions	156
12.	Complex Digital Data Exchange Problems	158
12.1.	Problem Scoping and Variable Management	159
12.1.1.	Defining the Environment	159
12.1.2.	Variable Scope: Local vs. Global	159
12.1.3.	Decomposition and Data Sources	160
12.2.	Security Risks and Data Integrity	160
12.2.1.	The CIA Triad and Privacy	160
12.2.2.	Emerging Technologies and Integrity	160
12.3.	Logical Modeling and Development Resources	161
12.3.1.	Data Flow Diagrams (DFDs)	161
12.3.2.	Evaluating Resources	162
12.4.	Algorithmic Planning and Technical Documentation	163
12.4.1.	Success Criteria	163
12.4.2.	Developing the Logic	163
12.4.3.	Annotation and Comments	164
12.4.4.	Professional Communication and Presentation	165
12.4.5.	Technical Language and Conventions	165
12.4.6.	Ethical Scholarship and Referencing	165
12.4.7.	Mode-Appropriate Communication	165
12.5.	Revision Questions	166
13.	Prototype Digital Data Exchanges	168
13.1.	Synthesising Secure Exchange Components	169
13.1.1.	Core Programming Features	169
13.1.2.	Algorithmic Building Blocks	169
13.2.	Implementing Encryption and Data Formats	170
13.2.1.	Encryption Logic	170

13.2.2. Data Interchange Formats	171
13.2.3. External Data Handling	171
13.3. Database Operations and Programmed Methods	172
13.3.1. SQL for Data Exchange	172
13.3.2. Programmed Logic	174
13.4. Quality Assurance and Technical Testing	175
13.4.1. Coding Standards	175
13.4.2. Debugging Techniques	175
13.4.3. Refinement	175
13.5. Evaluating Impacts and Security	176
13.5.1. Data Ownership and AI	176
13.5.2. Security Recommendations	176
13.5.3. Success Criteria Appraisal	176
13.6. Revision Questions	177
14. Glossary	178
15. Index	180

Part 1

Unit 1: Creating With Code

Chapter 1

Understanding Digital Problems

Table of contents

1.1. Computational Thinking and Thinking Tools	13
1.2. Identifying Human Needs and Opportunities	17
1.3. Analysing the Solution Environment	22
1.4. Social and Economic Impacts	26
1.5. Professional Communication and Documentation	30
1.6. Revision Questions	33

Aims and Objectives:

- Master the four pillars of computational thinking: decomposition, abstraction, pattern recognition, and algorithm design.
- Critically identify and distinguish between human needs, wants, and creative opportunities in diverse modern contexts.
- Analyse solution environments by defining scope, identifying multi-dimensional constraints, and setting measurable success criteria.
- Evaluate the social, personal, and economic impacts of digital solutions, including emerging technology trends.
- Demonstrate professional communication through technical documentation, visual schematic models, and ethical scholarship.



Digital problems are defined as situations where a human need, want, or opportunity can be addressed through a new or re-imagined digital solution. The first and most critical step in the problem-solving process is not coding, but deeply understanding the problem's constituents and the environment in which it exists.

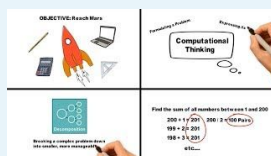
1.1. Computational Thinking and Thinking Tools

Computational thinking is a problem-solving methodology that allows us to take a complex problem, understand what the problem is, and develop possible solutions. These solutions can then be presented in a way that either a computer, a human, or both, can understand.

1.1.1. The Four Pillars of Computational Thinking

To solve digital problems effectively, developers apply four key techniques:

1. **Decomposition:** The process of breaking a complex problem down into smaller, more manageable sub-problems. For example, building a mobile game can be decomposed into physics engines, user interface design, asset management, and networking.
2. **Abstraction:** Filtering out unnecessary details to focus on the essential characteristics of a problem. In a mapping app, abstraction involves showing roads and landmarks while hiding individual trees or blade of grass.
3. **Pattern Recognition:** Looking for similarities or trends within problems. If we notice that many users struggle with a specific menu, we recognize a pattern that suggests a UI redesign is needed.
4. **Algorithm Design:** Developing a step-by-step strategy or set of rules to solve the problem. This is the blueprint for the code that will eventually be written.



Video: Computational Thinking: What Is It? How Is It Used?
– A clear breakdown of the four pillars.

<https://www.youtube.com/watch?v=qbnTZCj0ugI>

1.1.2. Methods of Decomposition

Decomposition is not just about splitting tasks; it is about strategic analysis. Strategies include:

- **Top-Down Design:** Starting with the overall purpose of the system and repeatedly breaking it into smaller functional modules.
- **Data-Oriented Decomposition:** Focusing on how data is transformed as it moves through the system, breaking the problem down based on data inputs, storage requirements, and outputs.

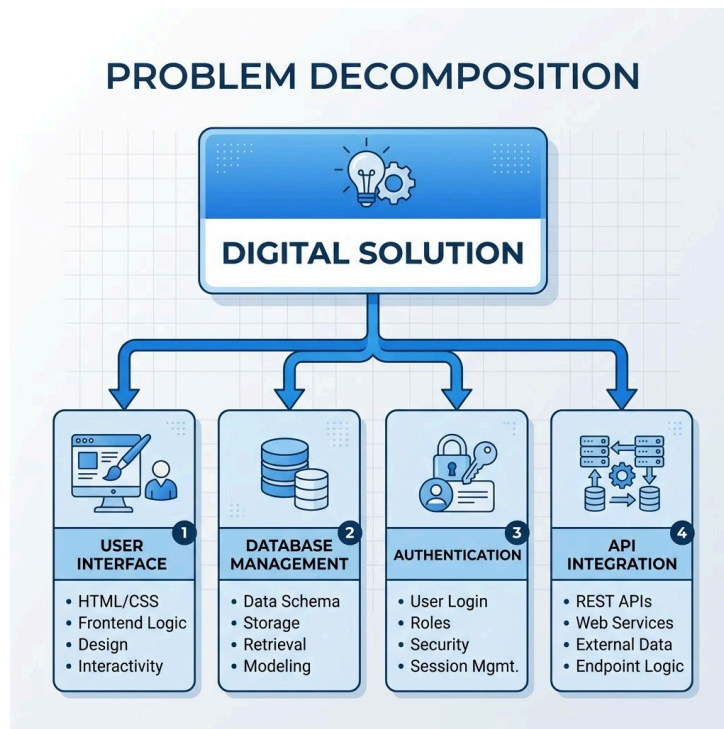


Figure 1.1 – A decomposition diagram breaking a complex digital solution into functional modules.¹

1.1.3. Thinking Tools for Analysis

Visualisation tools are essential for exploring problem structures and communicating ideas between stakeholders.

- **Mind Maps:** A non-linear way of brainstorming that helps identify the various dimensions of a problem and the potential interconnections between different requirements.
- **Flowcharts:** A formal schematic model used to represent the logical flow of an algorithm or process, using standardized symbols (rectangles for processes, diamonds for decisions).
- **Concept Maps:** Focused on identifying the “logical relationships” between entities, such as how a database table for ‘Users’ relates to a table for ‘Orders’.

¹Source: Gemini

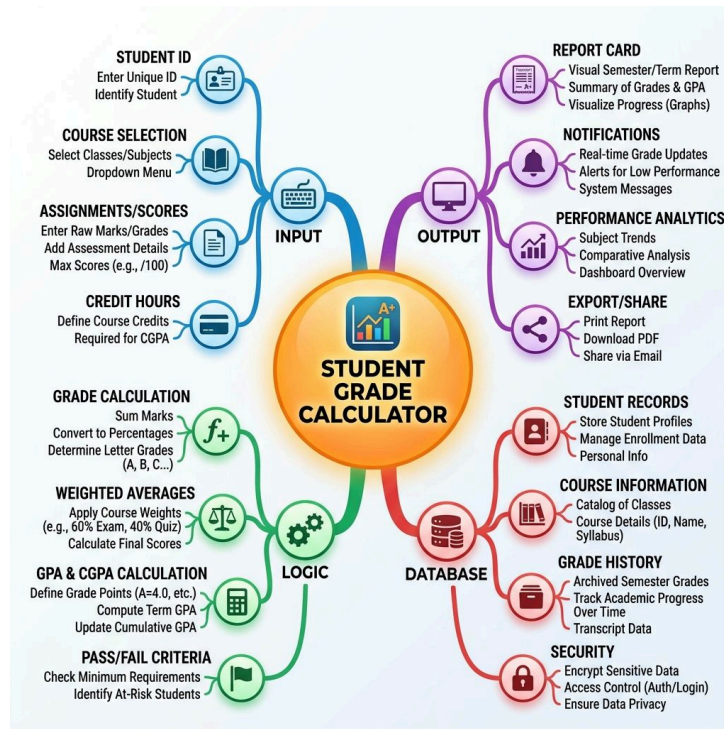


Figure 1.2 – An example mind map used for problem analysis in a grade calculator project.²

²Source: Gemini

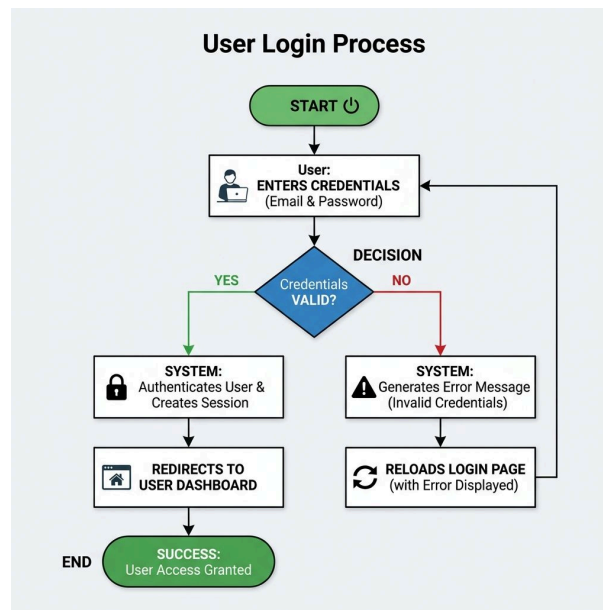


Figure 1.3 – A logic flowchart representing a user authentication process.³

1.1.4. Logical Relationships

In a digital system, logical relationships define how individual components interact. This involves understanding the **Input-Process-Output (IPO)** model:

- **Input:** What data enters the system (e.g., sensor data, user keystrokes).
- **Process:** How that data is transformed (e.g., sorting, calculating, validating).
- **Output:** What the system produces (e.g., a screen display, a physical movement of a robotic arm).

“The computer was born to solve problems that did not exist before.”

1.2. Identifying Human Needs and Opportunities

A digital solution is only successful if it provides value to its users. Identifying this value requires distinguishing between different types of requirements.

1.2.1. Defining Needs, Wants, and Opportunities

- **Needs:** These are essential requirements. If a banking app cannot securely store money, it has failed its primary need. Needs are “non-negotiable” features.
- **Wants:** These are desirable features that enhance the user experience but are not critical for functionality. A “dark mode” or custom profile icons are typically wants.
- **Opportunities:** This involve identifying a gap in the market or a way to “re-imagine” an existing process using technology. For example, ride-sharing apps took the “need” for transportation and turned it into an “opportunity” by using GPS and mobile payments to solve the “wants” of convenience and transparency.

³Source: Gemini

1.2.2. Essential Elements of a Solution

Every solution must address the core “pain points” of the user. This involves identifying the “minimal viable product” (MVP)—the smallest set of features required to prove that the solution addresses the identified need or opportunity.

1.2.3. Modern Problem Contexts

Technology solves problems in increasingly complex environments:

- **Search Engines:** Use massive-scale data processing to solve the problem of information discovery.
- **Robotics:** Combine sensors and actuators to automate physical tasks, from vacuuming floors to performing delicate surgeries.
- **Mobile Applications:** Leverage pervasive computing to provide tools that are always available, using sensors like accelerometers and GPS to add context to digital interactions.
- **Internet of Things (IoT):** Involves connecting everyday objects to the internet, allowing for remote monitoring and automation of environments (e.g., smart agriculture systems that water crops based on soil moisture data).



Video: What is the Internet of Things? – Exploring the interconnected world of IoT.

<https://www.youtube.com/watch?v=6mBO2vqLv38>

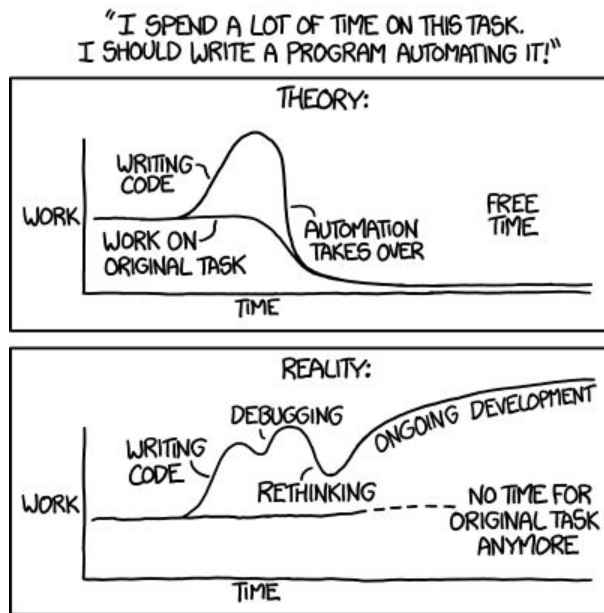


Figure 1.4 – The reality of automation: spending more time on the tool than the task.⁴

⁴Source: xkcd.com/1319



Figure 1.5 – An IoT ecosystem in a smart home environment.⁵

⁵Source: Gemini

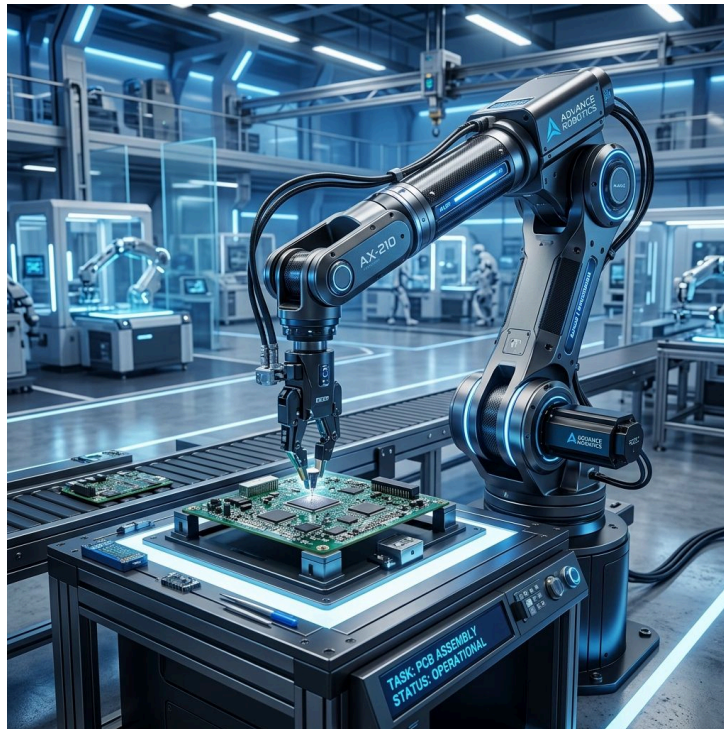


Figure 1.6 – An industrial robotic arm performing precision factory tasks.⁶

1.3. Analysing the Solution Environment

The environment includes all the external factors that influence how a solution is developed and how it will perform.

1.3.1. Scope, Constraints, and Limitations

- **Scope:** A clearly defined boundary of what the solution **will** and **will not** do. Defining what is “out of scope” is just as important as defining what is “in scope” to prevent “feature creep.”
- **Constraints:** External factors that limit the design or development. These include **Technical Constraints** (hardware limits), **Economic Constraints** (budget), **Legal Constraints** (Privacy Act), and **Social Constraints** (cultural expectations).
- **Limitations:** These are the inherent boundaries of the chosen technology or environment, such as the maximum range of a Wi-Fi signal or the battery life of a wearable device.

⁶Source: Gemini



Figure 1.7 – A humorous look at the difference between “easy” and “impossible” tasks in software development.⁷

1.3.2. Solution Requirements

Requirements are detailed descriptions of the system’s goals:

- **Functional Requirements:** Describe what the system **shall** do (e.g., “The system shall allow users to reset their password via email”).
- **Information Requirements:** Define the data that must be captured, stored, and processed (e.g., “The system must store user emails in a hashed format”).
- **Non-Functional Requirements:** Describe how the system **should** perform (e.g., “The system must load in under 2 seconds”).

⁷Source: xkcd.com/1425

1.3.3. User Perspective and UX Requirements

User Experience (UX) is not just about “looking good”—it’s about how the user **feels** and **interacts** with the system. Key considerations include:

- **Technical Literacy:** Is the app designed for a software engineer or a primary school student?
- **User Experience (UX):** How the user **feels** and **interacts** with the system. Key considerations include the environment and emotional response.



Video: Wireframes: The Ultimate UX Design Tool Tutorial – Learning how to structure user experiences.

<https://www.youtube.com/watch?v=Mr8gl3oflNg>

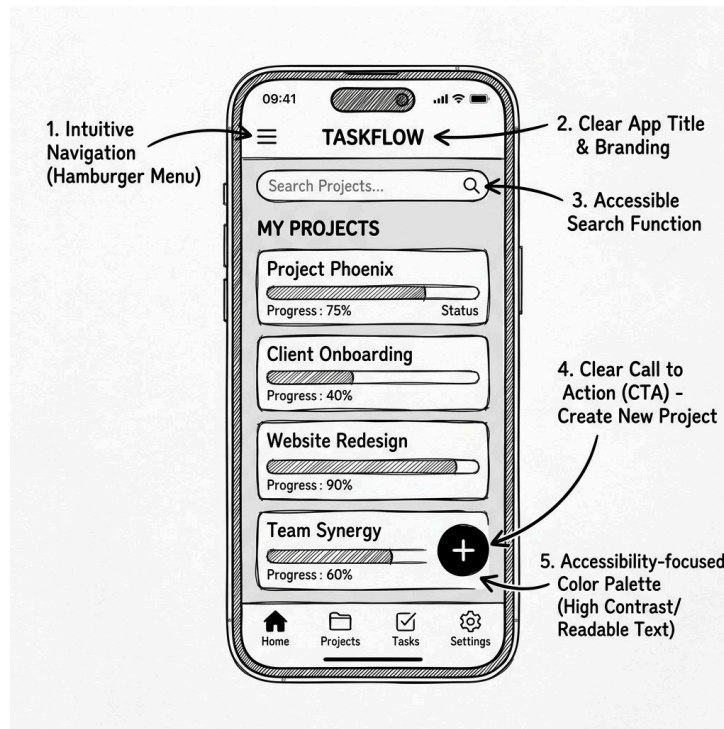


Figure 1.8 – An annotated wireframe highlighting key UX design considerations.⁸

1.3.4. Technical Interface Constraints

Hardware and software limitations directly impact UI design. Screen size, resolution, and input methods (e.g., voice control vs. physical buttons) dictate how information can be presented and how users can interact with the system.

1.3.5. Measurable Success Criteria

Success criteria are the “testable” standards used to evaluate the final product. They must be **SMART** (Specific, Measurable, Achievable, Relevant, and Time-bound). For example, “Make the app fast” is a poor criterion; “The app shall process 100 transactions per second with 99.9% uptime” is a measurable success criterion.

1.4. Social and Economic Impacts

Every digital solution has consequences that extend beyond the user and the developer.

1.4.1. Impact Analysis Framework

To evaluate these consequences, we use a structured framework:

- **Personal Impact:** How does the solution affect the individual? (e.g., privacy, health, cognitive load).
- **Social Impact:** How does it change how people communicate or how communities interact? (e.g., social media’s impact on public discourse).
- **Economic Impact:** How does it affect jobs, businesses, and global markets? (e.g., automation in manufacturing reducing manual labor but creating demand for robotics technicians).

⁸Source: Gemini

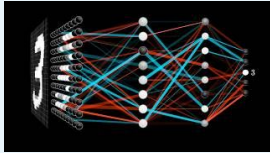
1.4.2. Evaluating Existing Solutions

Rarely is a problem entirely new. Evaluation involves looking at current solutions to identify their strengths and weaknesses. By analyzing “competitor” apps, developers can find opportunities to innovate by addressing the “fail points” of existing technology.

1.4.3. Emerging Technology Trends

Modern digital problems are often solved using cutting-edge technologies:

- **Machine Learning (ML):** A subset of AI that allows systems to learn from data patterns rather than being explicitly programmed for every scenario.
- **Neural Networks:** Algorithms modeled after the human brain that excel at pattern recognition in images, speech, and large datasets.
- **Natural Language Processing (NLP):** The ability of a computer to understand and respond to human language, enabling more natural “human-computer interaction.”



Video: But what is a neural network? – A visual introduction to the math behind AI.

<https://www.youtube.com/watch?v=aircAruvnKk>



Figure 1.9 – The “black box” nature of machine learning algorithms.⁹

⁹Source: xkcd.com/1838

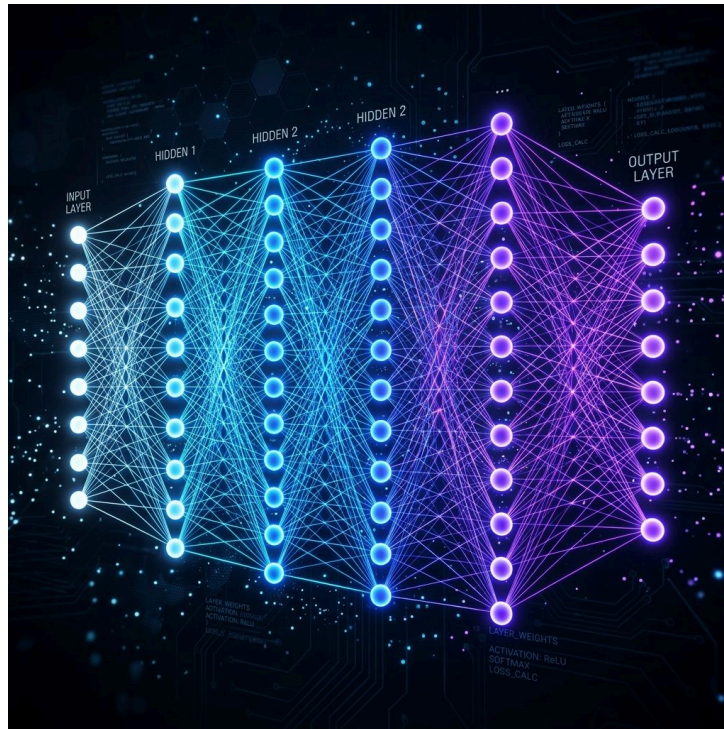


Figure 1.10 – A visualization of an artificial neural network used in deep learning models.¹⁰



Grace Hopper was a pioneer of computer programming who invented one of the first linkers (the A-0 system) and popularised machine-independent programming languages, leading to COBOL. She reminds us that innovation often comes from reimagining how we communicate with machines. Her work laid the foundation for the “Abstraction” we use today in high-level programming languages.¹¹

1.5. Professional Communication and Documentation

Professionalism in Digital Solutions requires the ability to communicate technical ideas to both technical and non-technical audiences.

1.5.1. Technology-Specific Language

Using precise vocabulary is essential for clarity. Terms like “latency,” “scalability,” “redundancy,” and “encapsulation” have specific meanings that allow developers to communicate complex ideas without ambiguity.

1.5.2. Visualizing Ideas

Before any code is written, ideas must be visualized through:

- **Annotated Sketches:** Quick, hand-drawn representations of a UI or system architecture.
- **Schematic Models:** Formal diagrams (like DFDs or flowcharts) that show the logic and data flow.
- **Wireframes:** Low-fidelity visual representations of a user interface.

¹⁰Source: Gemini

¹¹Source: Beyer, K. W. (2009). **Grace Hopper and the Invention of the Information Age**. MIT Press.



Video: UX Tutorial: How to WIREFRAME like a pro – Practical techniques for professional documentation.
<https://www.youtube.com/watch?v=WDCtcRpAqi4>



Figure 1.11 – An annotated interface sketch for a mobile dashboard.¹²

1.5.3. Communication Modes

Developers must adapt their communication mode to their audience:

- **Technical Manuals:** Focus on implementation, API endpoints, and data structures for other developers.
- **User Guides:** Focus on “how-to” steps and troubleshooting for the end-user.
- **Project Reports:** Focus on project scope, budget, and success criteria for stakeholders or clients.

1.5.4. Ethical Scholarship

Ethical scholarship is the practice of accurately attributing ideas and code to their original sources. In the digital world, this includes:

- **Acknowledging Open Source:** Giving credit to libraries or frameworks used in a project.
- **Referencing:** Using standard conventions (like APA or Harvard) to cite research, images, or historical anecdotes.
- **Integrity:** Ensuring that success criteria are reported honestly and that data privacy is respected.



The First “Bug”: In 1947, Grace Hopper’s team found a physical moth stuck in a relay of the Harvard Mark II computer. They taped the moth into their logbook and labeled it the “first actual case of bug being found.” This historical anecdote serves as a reminder of the meticulous nature required in digital problem analysis and the importance of accurate documentation.¹³

¹²Source: Gemini

¹³Source: National Museum of American History, Smithsonian Institution (Logbook of the Harvard Mark II).

1.6. Revision Questions

1. Explain how **Abstraction** and **Decomposition** work together to simplify a complex digital problem.
2. Differentiate between a **Constraint** and a **Limitation** using a mobile application as an example.
3. Why must **Success Criteria** be measurable? Provide an example of a non-measurable criterion and rewrite it to be measurable.
4. Use the **Impact Analysis Framework** to evaluate the potential personal, social, and economic impacts of an AI-driven medical diagnostic tool.
5. Describe two different **Thinking Tools** and explain when you would use one over the other.
6. What is the difference between a **Functional Requirement** and a **Non-Functional Requirement**? Give an example of each for a school database system.

Chapter 2

User Experiences and Interfaces

Table of contents

2.1. The Foundation of User Experience (UX)	35
2.2. Usability Principles	37
2.3. Visual Communication in Design	43
2.4. Prototyping and Symbolising Ideas	47
2.5. Evaluation and Refinement	49
2.6. Revision Questions	53

Aims and Objectives:

- Define User Experience (UX) and its significance in digital solution design.
- Critically apply the five usability principles to evaluate interface quality.
- Master the elements and principles of visual communication for interface design.
- Develop skills in iterative prototyping, from low-fidelity sketches to high-fidelity mock-ups.
- Understand the methods of evaluation and refinement used to improve user engagement.



Developing a digital solution isn't just about writing code; it's about how a human interacts with that code. A beautiful interface that is impossible to use is a failure of design, not just aesthetics.

2.1. The Foundation of User Experience (UX)

User Experience (UX) encompasses the meaning and importance of how individuals engage with a digital product. It is a holistic view of the user's journey, focusing on providing value, ease of use, and emotional satisfaction.

2.1.1. Driven by People

A successful interface is driven by people and their specific needs. Designers must move beyond “one size fits all” approaches and consider the diverse characteristics of their audience:

- **Age:** Younger users may prefer dynamic, gesture-based interfaces, while older users may require larger touch targets, higher contrast, and less cognitive load.
- **Digital Literacy:** Experienced users may want shortcuts (like Ctrl+C) and advanced features, whereas beginners need guided onboarding, clear cues, and prominent “Help” buttons.
- **Physical Abilities:** Considerations for accessibility ensure that the solution is fit for purpose for users with visual, auditory, or motor impairments. This includes screen reader compatibility and keyboard-only navigation.



User Personas: To help designers empathise with their audience, they often create “Personas”—fictional characters that represent the different user types within a targeted demographic. A persona might include a name, age, technical skill level, and specific goals or frustrations.

<u>DESIGNER</u>	<u>WHAT THEY ARE RESPONSIBLE FOR</u>
UI	ELEMENTS OF THE INTERFACE THAT THE USER ENCOUNTERS
UX	THE USER'S EXPERIENCE OF USING THE INTERFACE TO ACHIEVE GOALS
UZ	THE PSYCHOLOGICAL ROOTS OF THE USER'S MOTIVATION FOR SEEKING OUT THE INTERACTION
U α	THE USER'S SELF-ACTUALIZATION
U Ω	THE ARC OF THE USER'S LIFE
U ∞	LIFE'S EXPERIENCE OF TIME
U●	THE ARC OF THE MORAL UNIVERSE

Figure 2.1 – A commentary on the industry's distinction between UI and UX.¹

¹Source: xkcd.com/2141



Video: Usability 101: Introduction to User Experience – Foundations of UX and why it matters for every project.
<https://www.youtube.com/watch?v=st9AEPOjGpU>

2.2. Usability Principles

To ensure a solution is high-quality, it must be assessed against five core **usability principles**. These serve as the “success criteria” for an effective user interface.

2.2.1. 1. Accessibility

Ensuring the system can be used by people with the widest range of capabilities.

- **Example:** A banking app that includes high-contrast modes for visually impaired users and voice-over support for blind users.
- **Key Tool: WCAG (Web Content Accessibility Guidelines)** provide an international standard for making web content more accessible.



Video: What is Web Accessibility? – An introduction to inclusive design for all users.

<https://www.youtube.com/watch?v=20SHvU2PKsM>

2.2.2. 2. Effectiveness

How well the system performs the tasks it was designed for. Can the user achieve their goal accurately and completely?

- **Example:** A GPS application is effective if it provides the most efficient route and the user reaches their destination without getting lost.

2.2.3. 3. Safety

Protecting the user from dangerous conditions and undesirable situations. This includes protecting data from accidental loss and ensuring the system is resilient against malicious input.

- **Example:** “Confirm Deletion” dialogs in a photo app prevent accidental loss of data. “Undo” functionality (Ctrl+Z) allows users to recover from mistakes quickly.

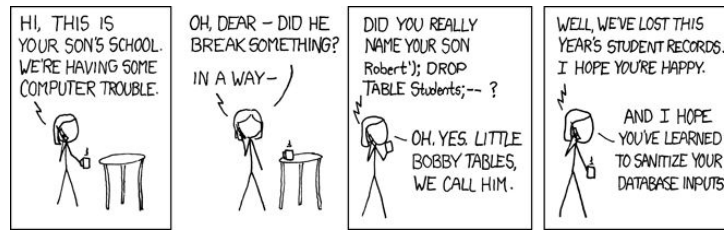


Figure 2.2 – The classic ‘Bobby Tables’ comic illustrating the importance of input validation for safety.²

²Source: xkcd.com/327

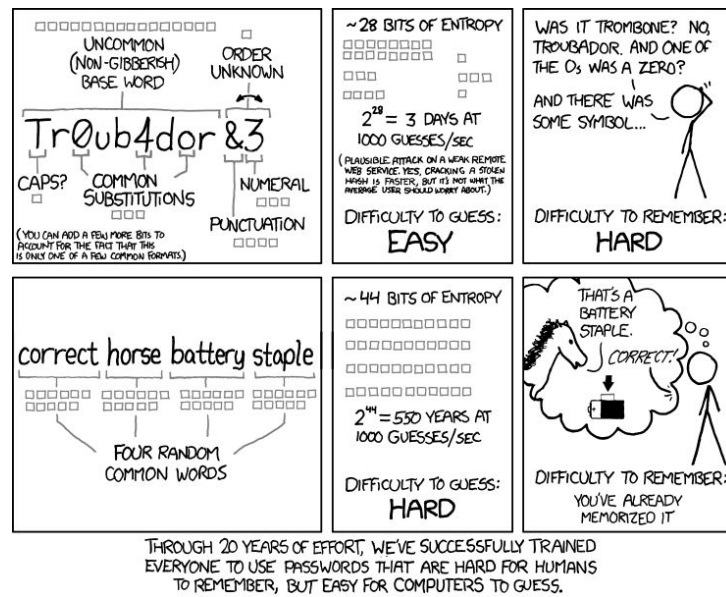


Figure 2.3 – A critique of security measures that make systems unusable for humans but easy for computers to crack.³

2.2.4. 4. Utility

Providing the right kind of functionality. Does the app actually do what the user needs it to do?

- **Example:** A calculator app that only adds numbers has low utility if the user needs to calculate square roots.

2.2.5. 5. Learnability

How easy it is for users to accomplish basic tasks the first time they encounter the design.

- **Example:** Using a “trash can” icon for deleting files is highly learnable because it maps to a real-world “mental model.”

³Source: xkcd.com/936

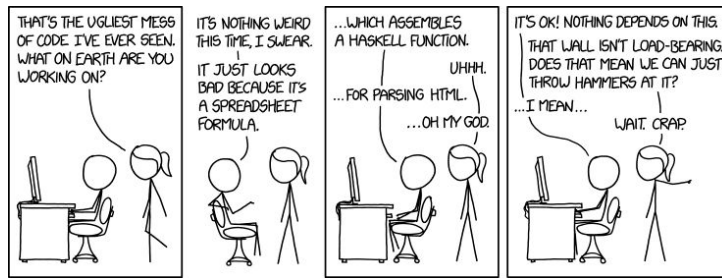


Figure 2.4 – The relationship between code quality and user experience: bad code eventually leads to a broken interface.⁴

⁴Source: xkcd.com/1926

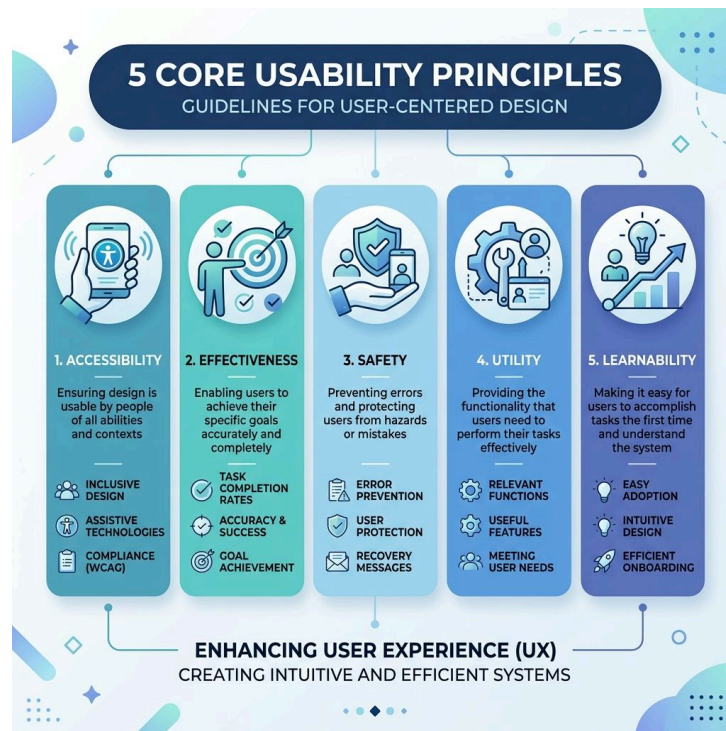


Figure 2.5 – The five pillars of usability: the foundation of high-quality digital solutions.⁵

⁵Source: Gemini



Figure 2.6 – Examples of accessibility features in a modern user interface.⁶

2.3. Visual Communication in Design

Effective interfaces rely on the strategic application of visual communication to guide the user’s attention and facilitate intuitive interaction.

2.3.1. Elements of Visual Communication

These are the “ingredients” used to build an interface.

- **Space:** “White space” prevents an interface from feeling cluttered and helps focus attention.
- **Line:** Used to divide sections or connect related items.
- **Colour:** Conveys emotion or urgency (e.g., Green for “Success”, Red for “Error”).
- **Shape:** Icons use simple shapes (like a magnifying glass for ‘search’) to represent complex ideas.
- **Texture and Tone:** Depth and focus are created through shadows (texture) and lightness/darkness (tone).
- **Form, Proportion, and Scale:** Elements that are more important should be larger (scale) to draw the eye first.

⁶Source: Gemini

⁷Source: Kare, S. (2024). **Susan Kare: Icons**. Susan Kare Design.



Susan Kare is an artist and graphic designer who created many of the interface elements for the original Apple Macintosh in the 1980s. She designed iconic fonts (like Chicago and Geneva) and many of the original icons (like the trash can, the paint bucket, and the “Happy Mac”) that established the visual language of modern computing. Her work proves that even with limited pixels, good design can create a friendly and intuitive interface.⁷

2.3.2. Principles of Visual Communication

These are the “rules” for combining the elements effectively.

- **Balance and Contrast:** Contrast (e.g., black text on white background) is essential for readability. Balance can be symmetrical (formal) or asymmetrical (dynamic).
- **Proximity and Alignment:** Grouping related items together (proximity) helps users understand relationships between data. Alignment creates a sense of order and professionalism.
- **Repetition and Harmony:** Using the same button style across an entire app creates a sense of unity and reduces the user’s “cognitive load.”
- **Hierarchy:** Directing the user’s attention to the most important information first. This is often achieved through font weight, size, and colour.

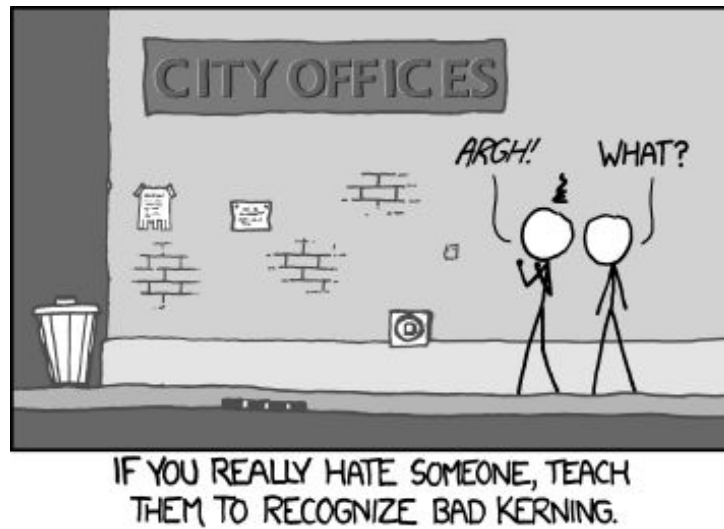


Figure 2.7 – The impact of subtle design flaws: Once you notice poor kerning (typography spacing), it's impossible to un-see.⁸

⁸Source: xkcd.com/1015

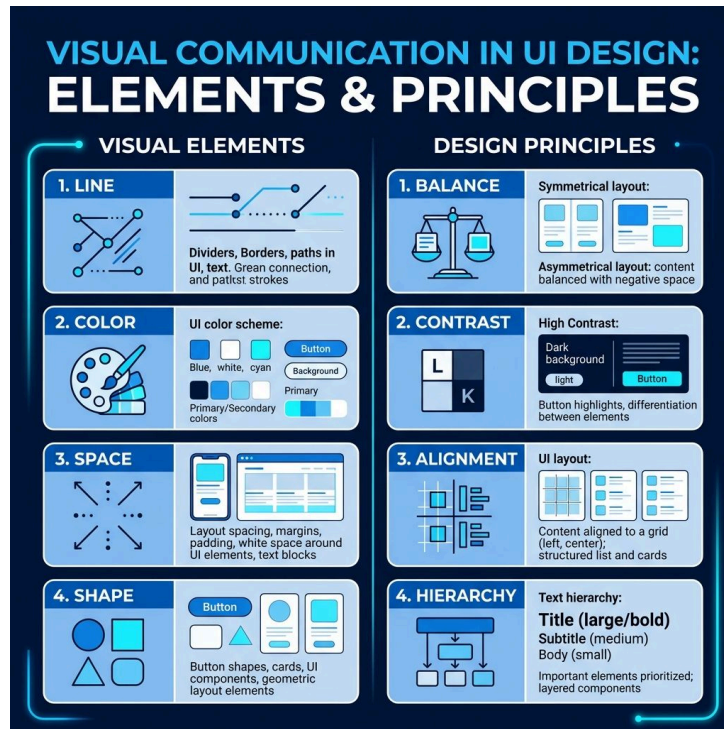


Figure 2.8 – Applying elements and principles of visual communication to UI design.⁹

⁹Source: Gemini



Video: Visual Design Principles for UI – Mastering hierarchy, contrast, and alignment in digital interfaces.

<https://www.youtube.com/watch?v=uwNCINmekGU>

2.4. Prototyping and Symbolising Ideas

Before generating a final solution, developers must **symbolise** their ideas to communicate intent and test concepts without the overhead of full coding.

2.4.1. The Iterative Design Cycle

Prototyping is a journey of increasing detail and “fidelity”:

- **Low-fidelity Prototypes:** Quick and inexpensive representations like **Sketches**, **Mood Boards** (for visual style), **Storyboards** (for user flow), and **Sitemaps**. These are used to fail fast and pivot ideas cheaply.
- **Technical Representations:** More formal models like **Schematic Diagrams**, **Wireframes**, and **Mock-ups**. Wireframes are the “blueprints” of a UI, defining where elements sit without focusing on final colours or fonts.
- **Annotations:** Critical notes that explain the purpose of specific features. For example, an annotation might describe what happens when a user clicks a specific button or how the system should respond to a swipe gesture.

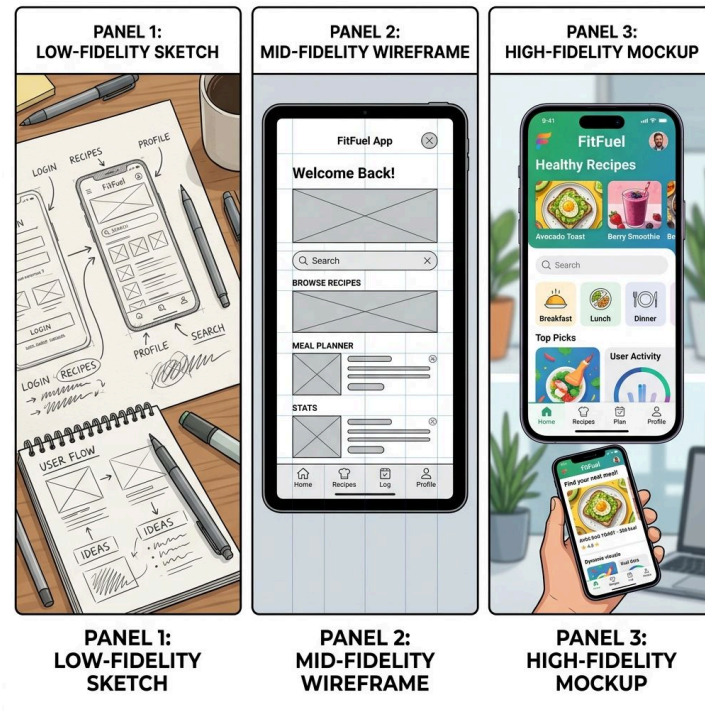


Figure 2.9 – The evolution of a prototype from a rough sketch to a high-fidelity mock-up.¹⁰

¹⁰Source: Gemini



Video: Low-Fi vs Hi-Fi Prototyping – When to use each mode in the design cycle.

<https://www.youtube.com/watch?v=sVKtLP85KJU>

2.5. Evaluation and Refinement

Evaluation is an iterative process that occurs throughout the design cycle. It ensures that the final solution meets both the user's needs and the defined usability standards.

2.5.1. Methods of Appraisal

1. **Heuristic Reviews:** Testing the interface against established usability principles. A common framework is **Nielsen's 10 Usability Heuristics**, which include rules like “Visibility of system status” (always let the user know what's happening).
2. **User Experience Testing:** Observing and recording actual user interactions. Designers look for “friction points”—areas where users pause, get confused, or make errors.
3. **Refinement:** Making iterative changes based on testing data. This is not a “one and done” step but a loop: Test -> Evaluate -> Refine -> Repeat.
4. **Justified Recommendations:** At the end of a project, developers must explain **why** they made certain changes, using evidence from their testing and evaluation phases.



Video: 10 Usability Heuristics Explained – A deep dive into Jakob Nielsen’s classic evaluation framework.

<https://www.youtube.com/watch?v=TQ5jY1kKUNk>

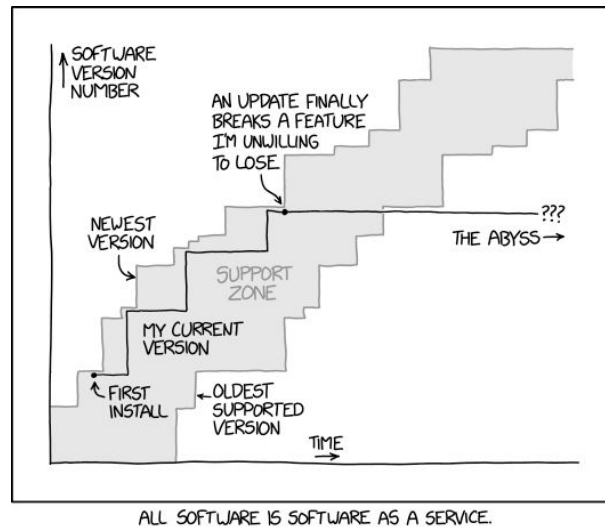


Figure 2.10 – The psychological impact of UI updates: users often perceive changes as a disruption to their established mental models.¹¹

¹¹Source: xkcd.com/2224



Figure 2.11 – The risk of “refinement”: every change to a UI breaks someone’s established workflow.¹²

¹²Source: xkcd.com/1172



Video: How to Run a User Test – Practical techniques for gathering feedback from real users.

<https://www.youtube.com/watch?v=6Bw0n6JvwXk>



Don Norman, often called the “father of UX,” coined the term “User Experience” while working at Apple. His book, **The Design of Everyday Things**, highlights how even simple objects like doors can have “usability” problems, emphasizing that designers are responsible for making technology intuitive. He famously said, “Design is really an act of communication, which means having a deep understanding of the person with whom the designer is communicating.”¹³



Brenda Laurel is a pioneer in human-computer interaction (HCI) and VR. She advocated for “design as performance,” suggesting that interfaces should be thought of as a stage where users act out their goals. Her work in the 1980s and 90s paved the way for modern interactive media and immersive user experiences. She was also a founder of Purple Moon, a company dedicated to creating digital games specifically designed for girls.¹⁴

2.6. Revision Questions

1. Differentiate between Accessibility and Effectiveness in the context of a public transport app.
2. How does the principle of **Hierarchy** guide a user through a complex news website?
3. Describe the purpose of **Annotations** in a high-fidelity mock-up.
4. Explain why evaluation must be an **iterative** process rather than a final step.
5. Identify a user characteristic and explain how it would influence the choice of **Colour** and **Scale** in an interface design.
6. Provide an example of a “Safety” feature in a professional coding environment (IDE).
7. What is a “User Persona,” and how does it help a designer during the initial phase of a project?

¹³Source: Norman, D. (2013). **The Design of Everyday Things**. Basic Books.

¹⁴Source: Laurel, B. (1991). **Computers as Theatre**. Addison-Wesley.

Chapter 3

Algorithms and Programming Techniques

Table of contents

3.1. Core Algorithmic Concepts	55
3.2. Algorithmic Representation	58
3.3. Programming Features and Practices	59
3.4. Data Interaction and Advanced Techniques	62
3.5. Revision Questions	63

Aims and Objectives:

- Master core algorithmic constructs: Assignment, Sequence, Selection, Iteration, and Modularisation.
- Develop language-independent logic using formal Pseudocode conventions.
- Implement robust code through industry-standard practices like modularisation and proper documentation.
- Analyse and apply data validation, SQL interaction, and cybersecurity techniques.
- Evaluate code quality through desk checks and debugging processes.



In the **Digital Solutions** syllabus, students explore the logical foundations of software development, moving from conceptual planning to technical implementation. Learning centers on creating efficient, maintainable code through computational thinking and industry-standard practices.

3.1. Core Algorithmic Concepts

Algorithms are the essential building blocks of any digital solution. At their most basic level, all algorithms consist of three stages: **Input**, **Process**, and **Output** (the IPO model).

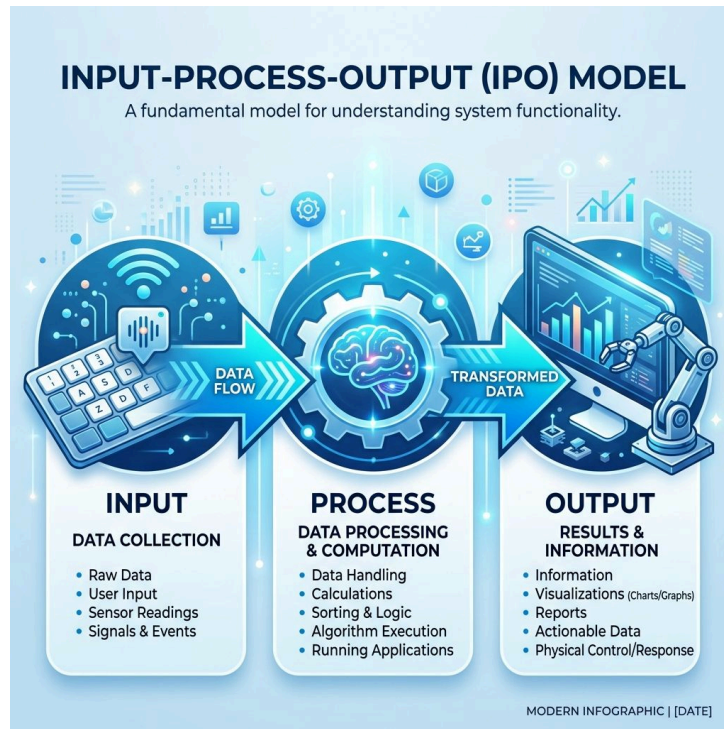


Figure 3.1 – The Input-Process-Output (IPO) model of computation.¹

3.1.1. Fundamental Constructs

To build complex logic, developers use five basic constructs:

- **Assignment:** Storing the value of an expression into a variable.
 - **Example:** `SET yearsToRetire TO 65 - currentAge`
- **Sequence:** Processing a series of instructions one after the other in a specific order.
- **Selection:** Determining the next instruction based on a **condition**—a logical expression evaluating to true or false.
 - **Example:** `IF age < 18 THEN OUTPUT "Access Denied" ELSE OUTPUT "Access Granted" ENDIF`
- **Iteration:** Repeating a set of instructions. This includes **counted loops** (e.g., `FOR`) and **while loops** (e.g., `WHILE score < 100 DO...`).
- **Modularisation:** Reducing system complexity by deconstructing it into independent units, such as functions and procedures. This allows for code reuse and easier debugging.

¹Source: Gemini

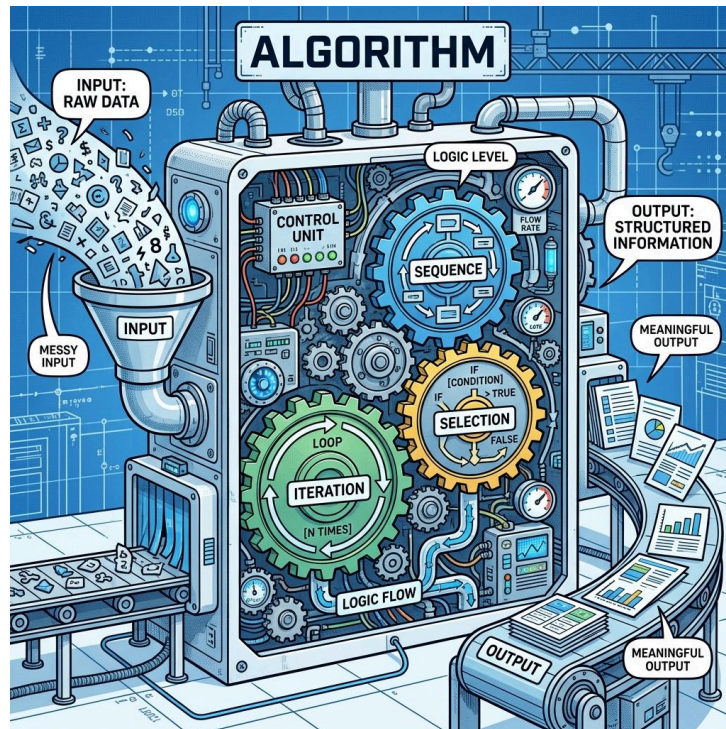
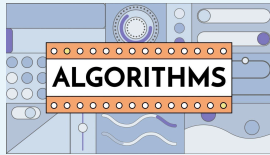


Figure 3.2 – An algorithm as a ‘logical machine’: sequence, selection, and iteration working in harmony.²

²Source: Gemini



Video: What is an Algorithm? – A foundational overview of logic and computation.

https://www.youtube.com/watch?v=kM9ASKAni_s

3.2. Algorithmic Representation

Developers use **Pseudocode** as a formal, language-independent method to describe and communicate algorithmic steps before writing actual code.

3.2.1. Pseudocode Conventions

Effective pseudocode follows a set of strict conventions to ensure clarity:

- **Keywords:** Use capitalised terms for structural constructs (e.g., **DECLARE**, **IF**, **THEN**, **ELSE**, **WHILE**, **ENDWHILE**, **FUNCTION**).
- **Formatting:** Use consistent **indentation** to show the hierarchy and logical grouping of code blocks.
- **Clarity:** Pseudocode should be descriptive enough for a human to understand but structured enough to be translated into any programming language.



```
FUNCTION CalculateTax(income)
  DECLARE taxRate = 0.2
  IF income < 18200 THEN
    RETURN 0
  ELSE
    RETURN (income - 18200) * taxRate
  ENDIF
ENDFUNCTION
```



Video: What is Pseudocode Explained – Mastering the bridge between human logic and computer code.

<https://www.youtube.com/watch?v=qfckDdsEIq8>

3.3. Programming Features and Practices

To generate functional components, students must master five basic programming features: **variables** (local and global), **control structures**, **data structures** (like Arrays), **syntax**, and **libraries/classes**.

3.3.1. Data Structures: Arrays

An **Array** is a data structure that can hold a fixed number of values of a single type. Arrays are used to store multiple items under a single variable name, accessible via an index.

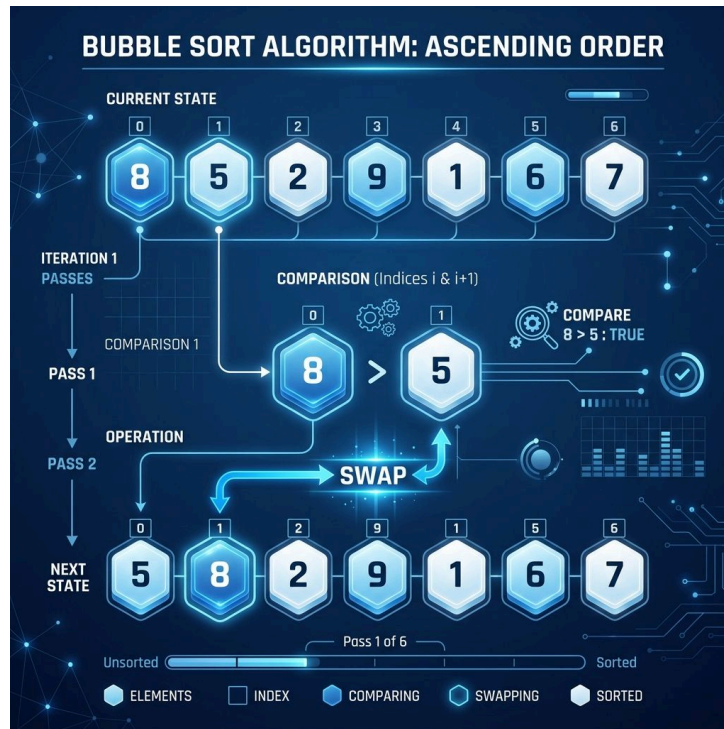


Figure 3.3 – Visualising a sorting algorithm operating on an array of data.³

3.3.2. Quality and Dependability

Industry-standard practices ensure that solutions are robust and sustainable:

- **Efficiency:** Writing simple, high-performing code that uses minimal resources.
- **Portability:** Ensuring code can be adapted to different environments or platforms with minimal changes.
- **Testing and Debugging:** Using processes like **desk checks** (manually tracing code logic) to predict outputs and identify errors.
- **Documentation:** Using consistent naming conventions (like camelCase) and **code comments** to clarify the purpose of statements for other developers.

³Source: Gemini

HOW LONG CAN YOU WORK ON MAKING A ROUTINE TASK MORE EFFICIENT BEFORE YOU'RE SPENDING MORE TIME THAN YOU SAVE?
(ACROSS FIVE YEARS)

		HOW OFTEN YOU DO THE TASK					
		50/DAY	5/DAY	DAILY	WEEKLY	MONTHLY	YEARLY
HOW MUCH TIME YOU SHAVE OFF	1 SECOND	1 DAY	2 HOURS	30 MINUTES	4 MINUTES	1 MINUTE	5 SECONDS
	5 SECONDS	5 DAYS	12 HOURS	2 HOURS	21 MINUTES	5 MINUTES	25 SECONDS
	30 SECONDS	4 WEEKS	3 DAYS	12 HOURS	2 HOURS	30 MINUTES	2 MINUTES
	1 MINUTE	8 WEEKS	6 DAYS	1 DAY	4 HOURS	1 HOUR	5 MINUTES
	5 MINUTES	9 MONTHS	4 WEEKS	6 DAYS	21 HOURS	5 HOURS	25 MINUTES
	30 MINUTES		6 MONTHS	5 WEEKS	5 DAYS	1 DAY	2 HOURS
	1 HOUR		10 MONTHS	2 MONTHS	10 DAYS	2 DAYS	5 HOURS
	6 HOURS				2 MONTHS	2 WEEKS	1 DAY
	1 DAY					8 WEEKS	5 DAYS

Figure 3.4 – The trade-off between writing efficient code and the time spent writing it.⁴⁴Source: xkcd.com/1205

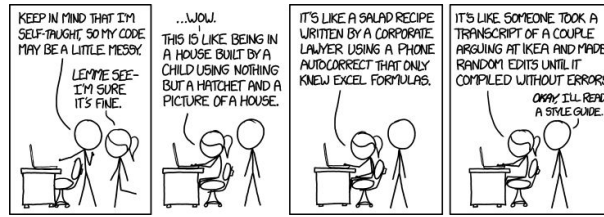


Figure 3.5 – A humorous look at the subjective nature of ‘Code Quality’.⁵

3.4. Data Interaction and Advanced Techniques

As digital solutions become more complex, they must interact with external data sources and ensure the security of that data.

3.4.1. Database Interaction

Relational databases are managed using **SQL statements**. The four primary operations (CRUD) are:

- **SELECT:** Retrieving data from a table.
- **INSERT:** Adding new records.
- **UPDATE:** Modifying existing records.
- **DELETE:** Removing records.

3.4.2. Data Validation

Data integrity is maintained by checking information before it is stored. Techniques include:

- **Range Check:** Ensuring a value is within a specified range (e.g., age between 0 and 120).
- **Type Check:** Ensuring the input is the correct data type (e.g., numeric vs string).
- **Existence Check:** Ensuring a required field is not left blank.

3.4.3. Cybersecurity

Protecting data during exchange involves:

- **Encryption:** Transforming plaintext into ciphertext using algorithms. **Symmetric** encryption uses one key, while **Asymmetric** encryption uses a public and private key pair.
- **Hashing:** Creating a fixed-length unique “fingerprint” of data. Adding a **Salt** (random data) before hashing increases security against pre-calculated attacks.

⁵Source: xkcd.com/1513



Video: How Encryption Works – A visual guide to symmetric and asymmetric encryption.

<https://www.youtube.com/watch?v=NuyzuNBFWxQ>



Ada Lovelace is often considered the first computer programmer. In the 1840s, she wrote an algorithm intended to be executed by Charles Babbage’s early mechanical computer, the Analytical Engine. She realized that computers could do more than just calculate numbers; they could manipulate symbols according to rules.⁶



Margaret Hamilton led the team that developed the on-board flight software for NASA’s Apollo program. She coined the term “software engineering” to give her team’s work the same respect as other engineering disciplines. Her focus on “Error Detection and Recovery” was critical to the success of the moon landing.⁷

3.5. Revision Questions

1. Describe the three stages of the **IPO model** using a login screen as an example.
2. What is the primary difference between **Selection** and **Iteration**?
3. Why is it important to use **Modularisation** in a large software project?
4. Write a short snippet of **Pseudocode** that uses a **WHILE** loop to print numbers from 1 to 10.
5. Explain the difference between **Symmetric** and **Asymmetric** encryption.
6. What is a **Desk Check**, and why is it used during the development phase?
7. Describe two common data validation techniques.

⁶Source: Essinger, J. (2014). **Ada’s Algorithm**. Melville House.

⁷Source: NASA History Office. **Margaret Hamilton: The Girl Who Taught Computers to Talk**.

Chapter 4

Programmed Solutions

Table of contents

4.1. Implementation of Logic and Operations	65
4.2. Generating the Prototype Solution	70
4.3. Testing, Evaluation, and Impact Analysis	70
4.4. Revision Questions	75

Aims and Objectives:

- Apply mathematical and logical operators to manage complex program flow.
- Develop efficient, modular code using local and global variables and constants.
- Generate and refine prototype digital solutions using text-based programming languages.
- Critically evaluate solutions against usability principles and broader social/economic impacts.
- Master professional testing and debugging techniques, including desk checks.



In this topic, students focus on the technical execution of their digital ideas, using a **text-based language** to create functional components and complete prototype solutions. The goal is to move from abstract logic to a tangible, working digital product.

4.1. Implementation of Logic and Operations

Proficiency in programming requires mastering the core syntax and logical structures that control how data is processed.



Kathleen Booth (1922–2022) was a British computer scientist and mathematician who wrote the first assembly language and designed the assembler for the first computer systems at Birkbeck College, London. Her work provided the crucial link between low-level machine code and higher-level programming logic, a concept that is foundational to every text-based programming language we use today.¹

4.1.1. Mathematical and Logical Operators

Operators are the fundamental tools for performing calculations and making decisions:

- **Mathematical Operators:** Beyond basic arithmetic, students use operators like **Modulus** (%) for finding remainders and **Exponentiation** (**) or `pow()`.
- **Comparison Operators:** Used to compare values (e.g., `==`, `!=`, `<`, `>`).
- **Logical Operators:** Combining conditions using **AND**, **OR**, and **NOT**.



Truth Tables: To understand complex logic, developers often use truth tables. For example, the **AND** operator only returns true if **both** inputs are true.

- `true AND true = true`
- `true AND false = false`
- `false AND false = false`

4.1.2. Variables, Constants, and Scope

- **Constants:** Unlike variables, the value of a constant cannot be changed once it is set. This is used for values like `PI = 3.14159` or `MAX_USERS = 100`.
- **Local Variables:** Defined within a function; they are only accessible within that specific area.

¹Source: Booth, K. H. V. (1958). **Programming for an Automatic Digital Calculator**. Butterworths.

- **Global Variables:** Defined outside any specific function; they can be accessed from anywhere.



Video: Logic Operators and Truth Tables – Understanding how AND, OR, and NOT control program flow.
<https://www.youtube.com/watch?v=zOjov-2OZ0E>

4.1.3. Control Structures and Modular Programming

- **Nested Logic:** Placing an IF statement inside another allows for more granular decision-making.
- **Nested Loops:** A loop inside another loop is often used to process 2D data, like a grid or a table.



```
for (let i = 0; i < 3; i++) {  
  for (let j = 0; j < 3; j++) {  
    console.log(`Row ${i}, Column ${j}`);  
  }  
}
```

- **Modular Programming:** Creating functions that utilize **Return Values** to pass data back to the main program. This makes code more reusable and testable.

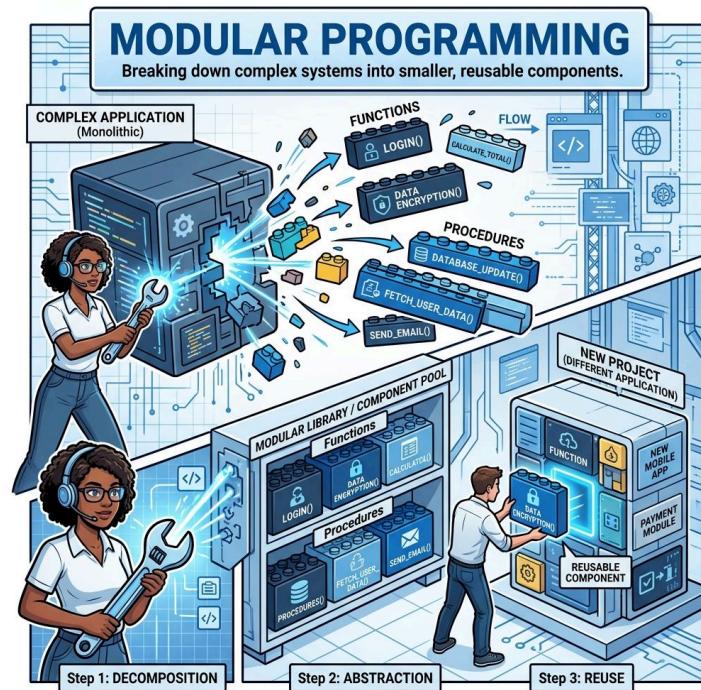
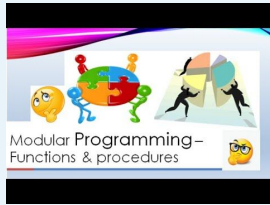


Figure 4.1 – Modular programming: breaking complex systems into reusable, portable components.²

²Source: Gemini



Video: Modular Programming and Functions – How to break complex problems into reusable components.

<https://www.youtube.com/watch?v=rPWT4VOezNc>

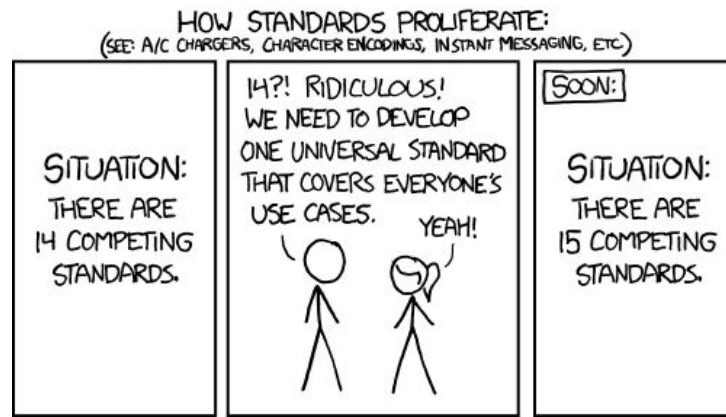


Figure 4.2 – The irony of modular programming and standards: trying to unify everything often leads to even more complexity.³

4.2. Generating the Prototype Solution

The technical work culminates in the assembly of individual components into a cohesive, functional digital product.

4.2.1. Component Development and Data Handling

- **Original Code:** Students generate specific solution components by writing original statements or effectively utilizing existing libraries and classes.
- **I/O (Input/Output):** Programs must handle data accurately. This might involve parsing structured data like **JSON** (JavaScript Object Notation) to display information to the user dynamically.

4.3. Testing, Evaluation, and Impact Analysis

4.3.1. Technical Debugging and Performance

- **Debugging:** Using **Desk Checks** to trace algorithmic steps. This is a manual process where you act as the computer, updating variable values in a table to ensure the logic works as intended.

³Source: xkcd.com/927



Figure 4.3 – The ‘rabbit hole’ of debugging: one small fix often leads to an unexpected journey of discovery.⁴

⁴Source: xkcd.com/1722



Bolen
Presents

Video: Debugging and Desk Checking – Essential techniques for finding and fixing errors in your logic.
<https://www.youtube.com/watch?v=BHBqtVazgkk>

4.3.2. User-Centric Review and Broader Impacts

- **Usability:** The final interface is appraised against the five principles: Accessibility, Effectiveness, Safety, Utility, and Learnability.
- **Impact Analysis:** Consider the **Social and Economic Impacts**. For example, a bug in an automated medical dispenser (Social Impact) or a vulnerability in a banking app (Economic Impact) can have devastating real-world consequences.



Figure 4.4 – A historical perspective on the social impact of new technology: every generation fears the ‘revolutionary’ medium of their time.⁵

⁵Source: xkcd.com/1227



Video: Social and Economic Impacts of ICT – Analysing how digital solutions shape our modern world.

https://www.youtube.com/watch?v=_Log1AKKQxY

4.4. Revision Questions

1. Provide an example of a **nested loop** and describe a situation where it would be used.
2. Differentiate between a **Variable** and a **Constant**.
3. What is the benefit of a function having a **Return Value**?
4. Describe a potential **Economic Impact** of a security breach in an e-commerce platform.
5. How does a **Desk Check** help find logic errors that a compiler might miss?

Part 2

Unit 2: Application and Data Solutions

Chapter 5

Data-driven Problems and Solution Requirements

Table of contents

5.1. Analyzing Data Challenges	80
5.2. System Modeling and Relationships	84
5.3. Design and Impact Considerations	90
5.4. Revision Questions	93

Aims and Objectives:

- Understand the nature of data-driven problems and how to decompose them.
- Master relational database concepts: primary and foreign keys, data types, and constraints.
- Symbolise data movement using Data Flow Diagrams (DFD) with Gane-Sarson notation.
- Evaluate the social, economic, and personal impacts of data storage and management.
- Explore emerging technologies like Machine Learning and Neural Networks in the context of data.



In Unit 2, students explore the nature of data-driven problems, focusing on how data is structured, validated, and managed within a digital system. This requires a shift from simple logic to managing the complexity of large, interconnected datasets.



Figure 5.1 – The DIKW Hierarchy (Data-Information-Knowledge-Wisdom).¹

5.1. Analyzing Data Challenges

Developing a data-driven solution begins with understanding the complexities of handling information to ensure it remains reliable and useful.

5.1.1. Input Issues and Variations

Data insertion is rarely straightforward. Developers must analyze problems associated with:

- **Data Formats:** Ensuring that inputs from various sources (e.g., date formats, currency, or unit measurements) are normalized before storage.
- **Insertion Requirements:** Managing how different user roles (e.g., admin vs. student) interact with the same dataset.

5.1.2. Integrity and Reliability

To ensure data is accurate for use and storage, developers apply **Validation Rules**.

- **Constraint Examples:** Using `NOT NULL` to ensure required fields are filled, or `CHECK` constraints to ensure a value (like age) falls within a valid range.

¹Source: Gemini



Elizabeth “Jake” Feinler was a pioneer in managing the massive amounts of data that would eventually become the internet. As director of the Network Information Center (NIC), she led the team that developed the first online directories and resource registries for the ARPANET. Her work on naming conventions and data organization laid the foundation for the Domain Name System (DNS) we use every day to navigate the web.²

²Source: Internet Hall of Fame. **Elizabeth Feinler: Official Biography.**

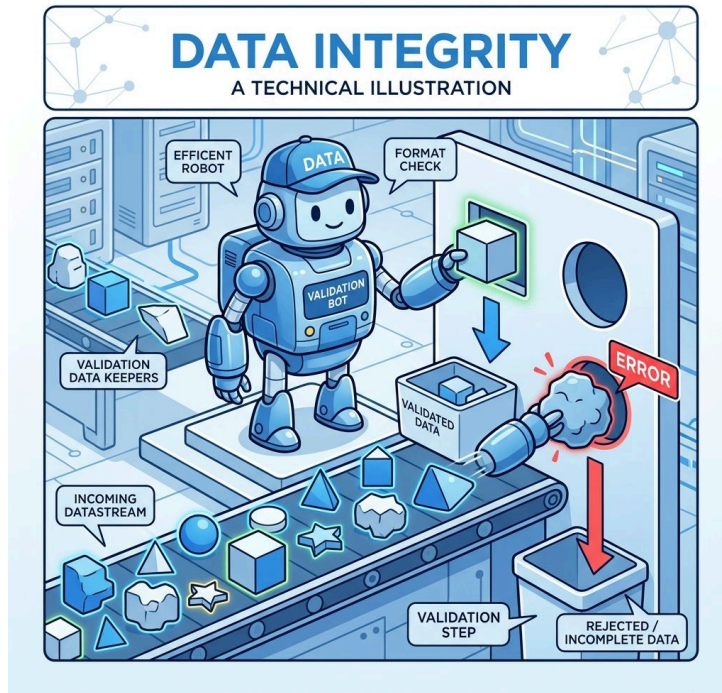


Figure 5.2 – Data validation: the gatekeeper of integrity in a digital system.³

5.1.3. Decomposition and Pattern Recognition

Using computational thinking, students break down complex user requirements into manageable storage needs. **Decomposition** involves identifying the core entities (like ‘Users’, ‘Orders’, or ‘Products’) and the specific attributes needed for each.

³Source: Gemini

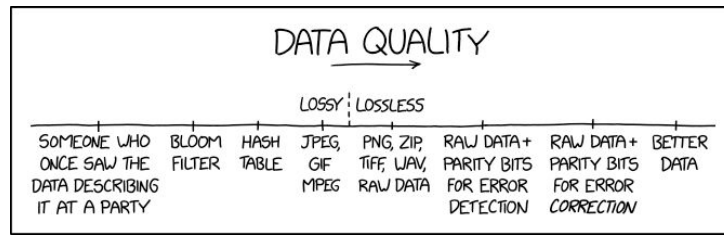


Figure 5.3 – The challenge of maintaining data quality over time.⁴

⁴Source: xkcd.com/2083



Video: Data Validation and Integrity – Practical techniques for ensuring your database remains accurate and reliable.
<https://www.youtube.com/watch?v=HXV3zeQKqGY>

5.2. System Modeling and Relationships

To communicate and plan a data-driven system, developers use specific symbolic tools and relational concepts.

5.2.1. Relational Concepts and Keys

Relational Databases organize data into tables. Key concepts include:

- **Primary Key (PK):** A unique identifier for every record in a table.
- **Foreign Key (FK):** A field in one table that links to the primary key of another table, creating a relationship between them.

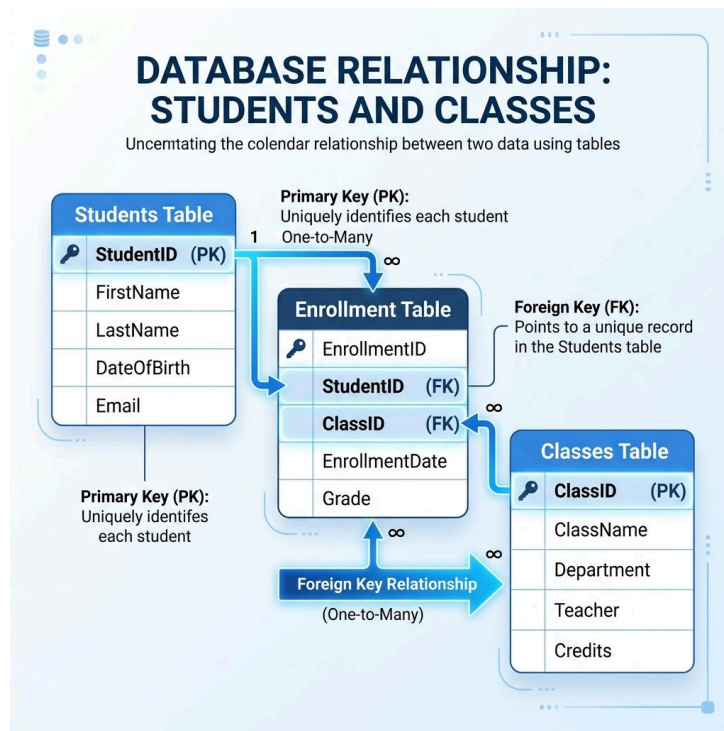


Figure 5.4 – Visualising the relationship between Primary and Foreign Keys.⁵

⁵Source: Gemini

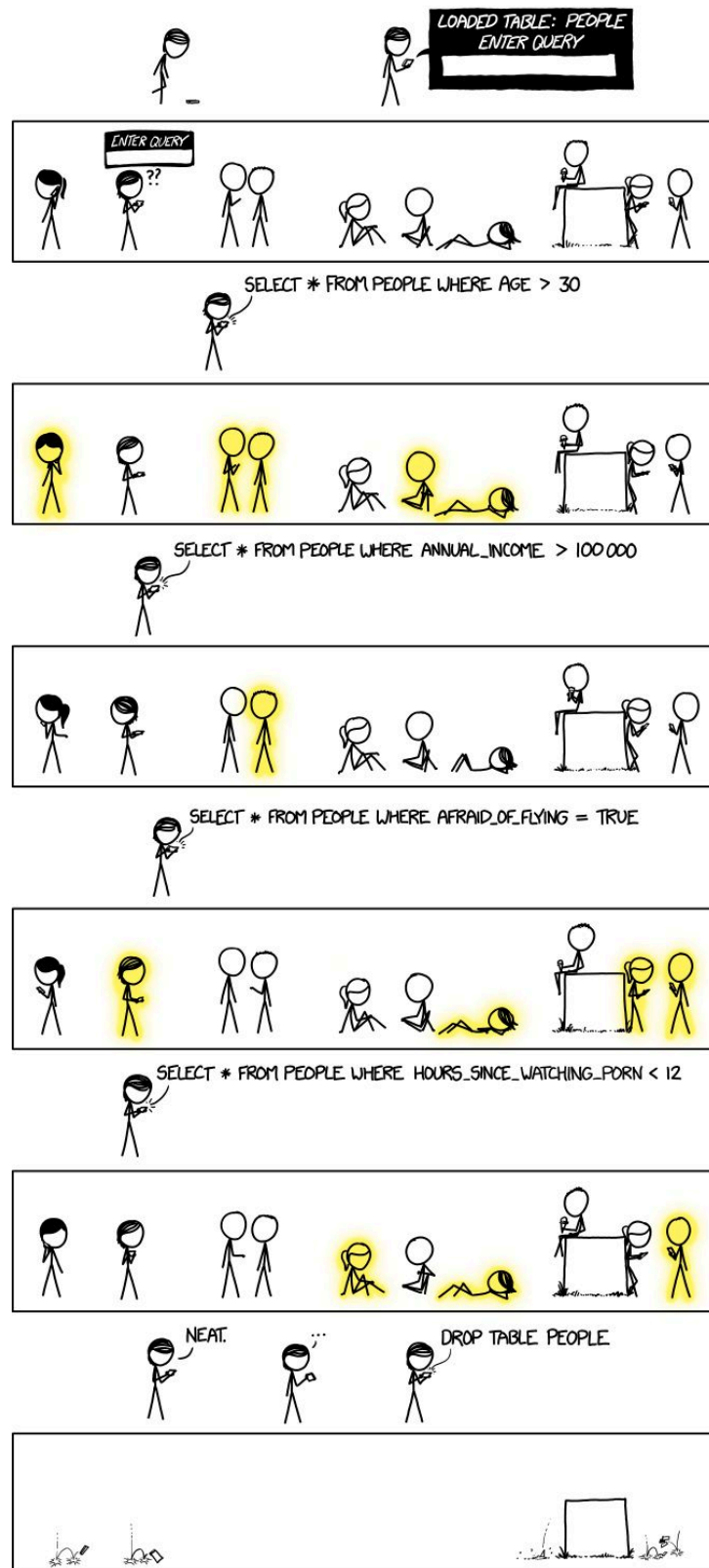
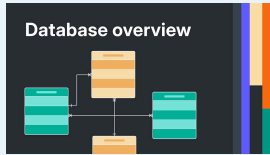


Figure 5.5 – The importance of precise queries when interacting with relational data.⁶

⁶Source: xkcd.com/1409



Video: Database Tutorial for Beginners – Understanding how tables, keys, and relationships build a robust data system.
<https://www.youtube.com/watch?v=wR0jg0eQsZA>

Students use **Gane-Sarson notation** to symbolise the movement of data through a system using **Data Flow Diagrams (DFD)**.

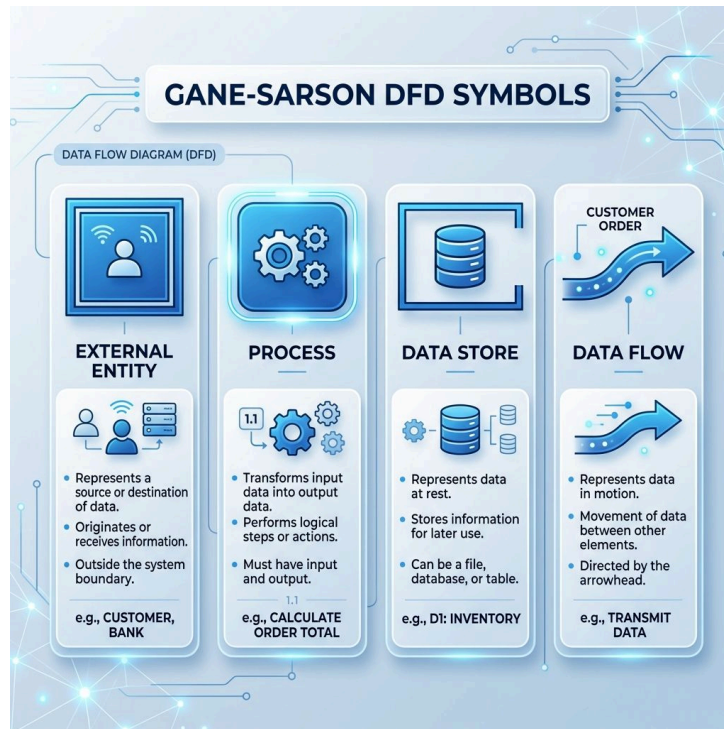


Figure 5.6 – Standard symbols used in Gane-Sarson DFD notation.⁷

⁷Source: Gemini

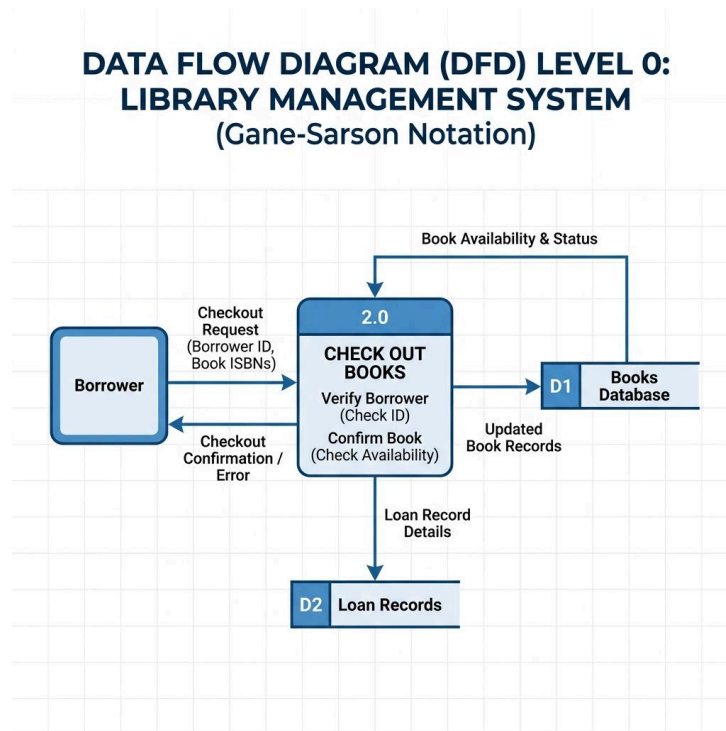
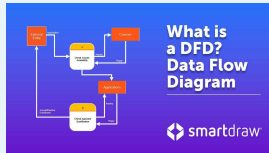


Figure 5.7 – Case Study: A DFD for a Library Management System, showing data movement from a Borrower to the Books Database.⁸

⁸Source: Gemini



Video: Data Flow Diagrams - What is DFD? – Using Gane-Sarson notation to map system logic.

<https://www.youtube.com/watch?v=6VGTvgaJlIM>

5.3. Design and Impact Considerations

5.3.1. Impact Analysis

The storage and management of data have significant consequences:

- **Personal:** Issues of **Data Privacy** and laws like the **GDPR** or the **Australian Privacy Act**, which protect individuals' rights to their personal information.
- **Social:** The risk of **Bias in Algorithms**, where data used to train AI models may inadvertently lead to unfair or discriminatory outcomes.
- **Economic:** The importance of **Data Sovereignty**—ensuring that data is subject to the laws and governance of the country in which it is collected.

5.3.2. Emerging Technologies

Advancements like **Machine Learning** and **Neural Networks** are changing how we store and interpret data.

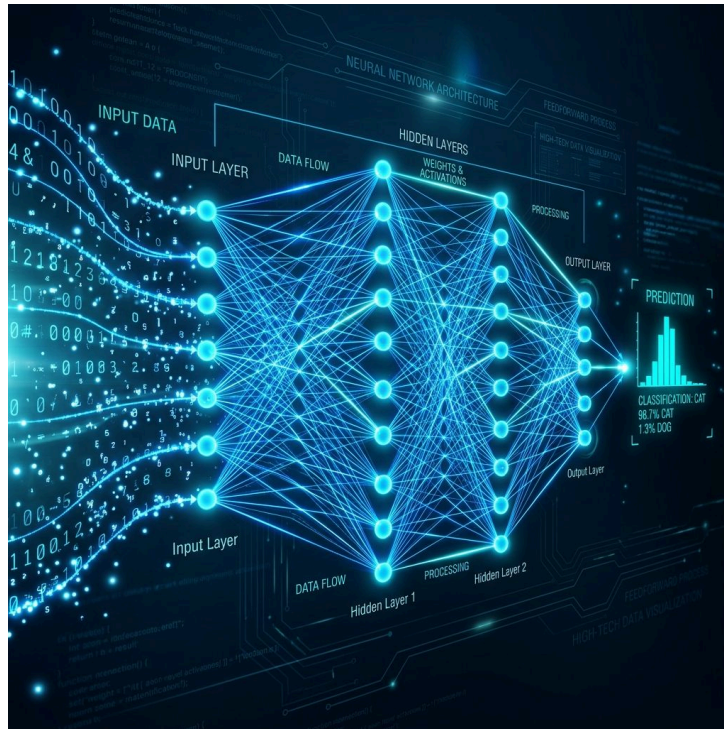


Figure 5.8 – A simplified model of a Neural Network processing data through hidden layers.⁹

⁹Source: Gemini

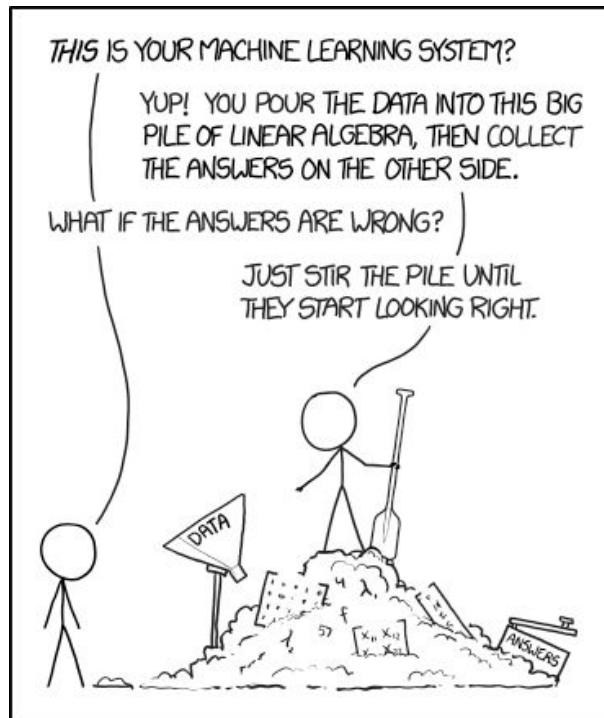
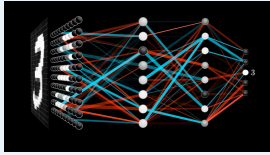


Figure 5.9 – The reality of current Machine Learning: pouring data into a pile of linear algebra and hoping for the best.¹⁰

¹⁰Source: xkcd.com/1838



Video: Neural Networks Explained – How data flows through connected nodes to ‘learn’ patterns.

<https://www.youtube.com/watch?v=aircAruvnKk>



Edgar F. Codd (1923–2003) was a British computer scientist who, while working for IBM, invented the relational model for database management. His work, specifically his “12 Rules” for relational databases, revolutionized how we think about data storage, moving us away from complex hierarchical structures to the intuitive tables we use today.¹¹

5.4. Revision Questions

1. Differentiate between a **Primary Key** and a **Foreign Key**.
2. Identify the four main symbols in a **Gane-Sarson DFD** and describe their purpose.
3. How do **Validation Rules** (like constraints) improve the reliability of a database?
4. What is **Data Sovereignty**, and why is it important for governments?
5. Describe a scenario where **Algorithm Bias** could impact a community.
6. How does a **Neural Network** differ from a traditional relational database in how it handles data?

¹¹Source: Codd, E. F. (1970). **A Relational Model of Data for Large Shared Data Banks**.

Chapter 6

Data and Programming Techniques

Table of contents

6.1. Core Elements of Data-Driven Solutions	95
6.2. Principles of Data Management	97
6.3. Relational Database Fundamentals	98
6.4. Algorithmic Logic and Symbolic Representation	100
6.5. Revision Questions	101

Aims and Objectives:

- Recognise essential components of data-driven solutions, including problem scope and constraints.
- Master SQL syntax and understand technical requirements for data storage.
- Understand principles of data acquisition, integrity, and security.
- Differentiate between data, information, and wisdom (DIKW hierarchy).
- Interpret database structures through relational schemas and normalisation (3NF).
- Design and represent algorithms using Data Flow Diagrams (DFDs) and pseudocode.

6.1. Core Elements of Data-Driven Solutions

A data-driven solution is more than just a piece of software; it is a systematic approach to solving problems by leveraging data effectively.

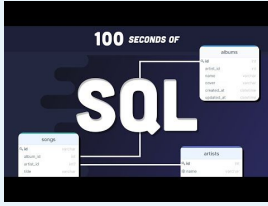
6.1.1. Problem Scope and Constraints

Before developing a solution, students must recognise the essential components, including the **scope** of the problem and any inherent **constraints** or limitations. Constraints might include technical limitations (e.g., hardware compatibility), economic factors (e.g., budget), or social considerations (e.g., user accessibility) [cite: 2751, 2752, 2753].

6.1.2. Technical Requirements

Technical mastery involves understanding the infrastructure that supports data solutions. This includes:

- **SQL Syntax:** Mastering Structured Query Language to interact with data stores.
- **Hardware and Platforms:** Identifying the necessary physical and virtual environments [cite: 2754, 2755, 2761].
- **Data Stores:** Choosing between local databases, cloud storage, or distributed systems.



Video: SQL in 100 Seconds – A high-speed overview of database querying.

<https://www.youtube.com/watch?v=zsjuFFKOm3c>

6.1.3. Data Acquisition

Data is acquired from various real-world sources. These include:

- **Files:** CSV, JSON, or XML files.
- **Peripheral Devices:** Sensors, scanners, or IoT devices.
- **Online Repositories:** APIs and public datasets [cite: 2756, 2767].

6.1.4. Social and Economic Impacts

The storage and management of databases have personal, social, and economic impacts that must be evaluated from multiple perspectives:

- **Individual:** Privacy and personal data control.
- **Organizational:** Efficiency, security, and data-driven decision-making.
- **Governmental:** Regulation (like GDPR or the Privacy Act) and societal wellbeing [cite: 2757].

6.1.5. The Conceptual Hierarchy: DIKW

A fundamental conceptual hierarchy distinguishes between **Data**, **Information**, and **Wisdom**. While data represents raw facts, information provides context, and wisdom allows for the ethical and strategic application of that knowledge [cite: 2759].



Example: Healthcare Monitoring

- **Data:** A heart rate sensor records “110 bpm”.
- **Information:** In the context of a resting patient, “110 bpm” is categorized as tachycardia (high heart rate).
- **Knowledge:** The medical system cross-references this with the patient’s history of heart disease.
- **Wisdom:** A doctor decides to administer specific medication and orders an immediate ECG, balancing technical data with professional judgment.

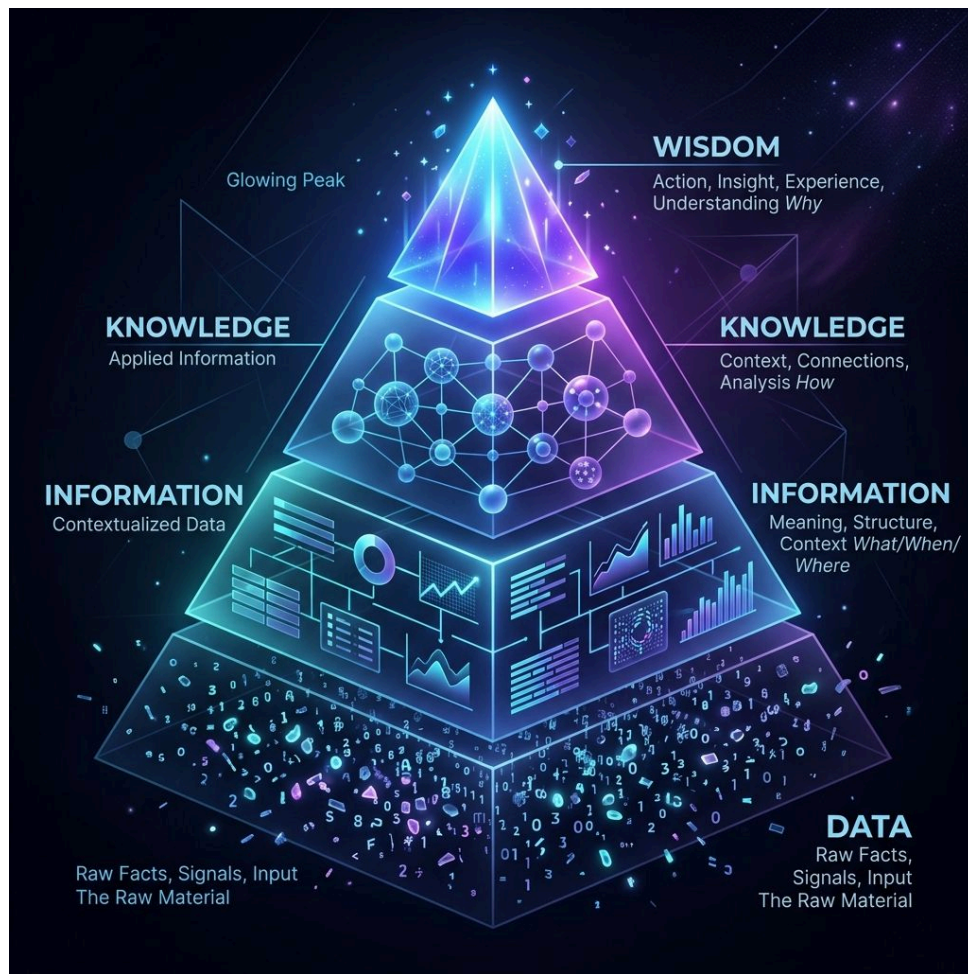


Figure 6.1 – The DIKW Hierarchy: Transforming raw data into strategic wisdom.

6.2. Principles of Data Management

Effective data management ensures that information remains usable, accurate, and secure throughout its lifecycle.

6.2.1. Acquisition, Integrity, and Security

The three pillars of data management are **acquisition** (gathering from reliable sources), **Data Integrity** (ensuring accuracy and consistency), and **Data Security** (protection from unauthorized access) [cite: 2766, 2767, 2770, 2773].

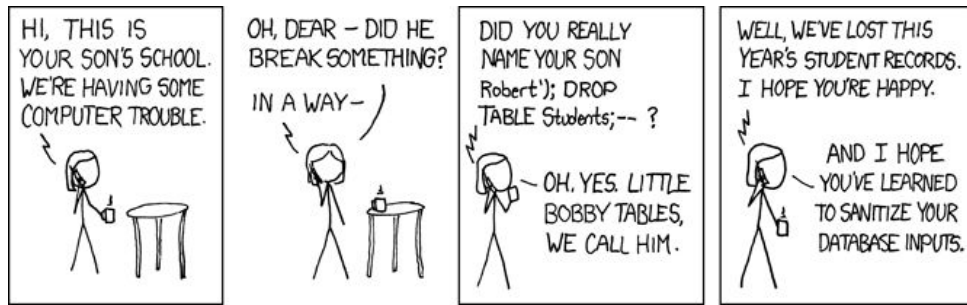


Figure 6.2 – Sanitizing inputs is crucial for maintaining database integrity.¹

6.2.2. Organization and Standards

Organization involves using consistent **naming conventions**, standardized **data formats** (like ISO dates), and logical structures to ensure data is usable across different systems [cite: 2768].

6.2.3. Data Anomalies and Redundancy

To maintain high-quality data sets, students must identify and address:

- **Redundancy:** Unnecessary duplication of data.
- **Anomalies:** Inconsistencies that occur during insertion, deletion, or modification of data [cite: 2771, 2772, 2786].

6.2.4. Advanced Processing Techniques

Protecting sensitive information requires advanced techniques such as:

- **Encryption:** Scrambling data to prevent unauthorized reading.
- **De-identification:** Removing personally identifiable information (PII) to protect privacy [cite: 2787].

6.2.5. Validation vs. Verification

A critical distinction is made between:

- **Data Validation:** Ensuring data is “clean” and follows predefined rules (e.g., range checks).
- **Data Verification:** Ensuring data matches the original source (e.g., double-entry) [cite: 2776].

6.3. Relational Database Fundamentals

6.3.1. Relational Schema (RS)

Database structures are interpreted through a **Relational Schema**, which visually maps how different data points and tables relate to one another [cite: 2765].

6.3.2. Tables and Components

Students must work with relational tables consisting of:

- **Rows:** Individual records (tuples).

¹Source: xkcd.com/327

- **Columns:** Specific attributes or fields.
- **Nulls:** Placeholders for missing or unknown data.
- **Views:** Virtual tables representing a subset of data [cite: 2779].

6.3.3. Referential Integrity

Maintaining referential integrity requires a clear understanding of:

- **Primary Keys:** Unique identifiers for each row in a table.
- **Foreign Keys:** Fields that link to a primary key in another table [cite: 2777, 2778].

6.3.4. Normalisation

Database performance is optimized by applying **Normalisation** techniques. The goal is to reach **Third Normal Form (3NF)**, which ensures that all attributes are functionally dependent only on the primary key [cite: 2763, 2764, 2777].



Normalization Case Study: Sales Invoices

- **Unnormalized (0NF):** A single table containing InvoiceID, Date, Customer-Name, Item1, Price1, Item2, Price2. (Repeating groups).
- **First Normal Form (1NF):** Eliminate repeating groups. Each item is its own row: InvoiceID, ItemID, Price.
- **Second Normal Form (2NF):** Ensure all non-key columns depend on the entire primary key. Remove “Price” to a separate **Items** table since it depends on ItemID, not InvoiceID.
- **Third Normal Form (3NF):** Remove transitive dependencies. If CustomerAddress depends on CustomerName, and CustomerName depends on InvoiceID, move address to a separate **Customers** table.

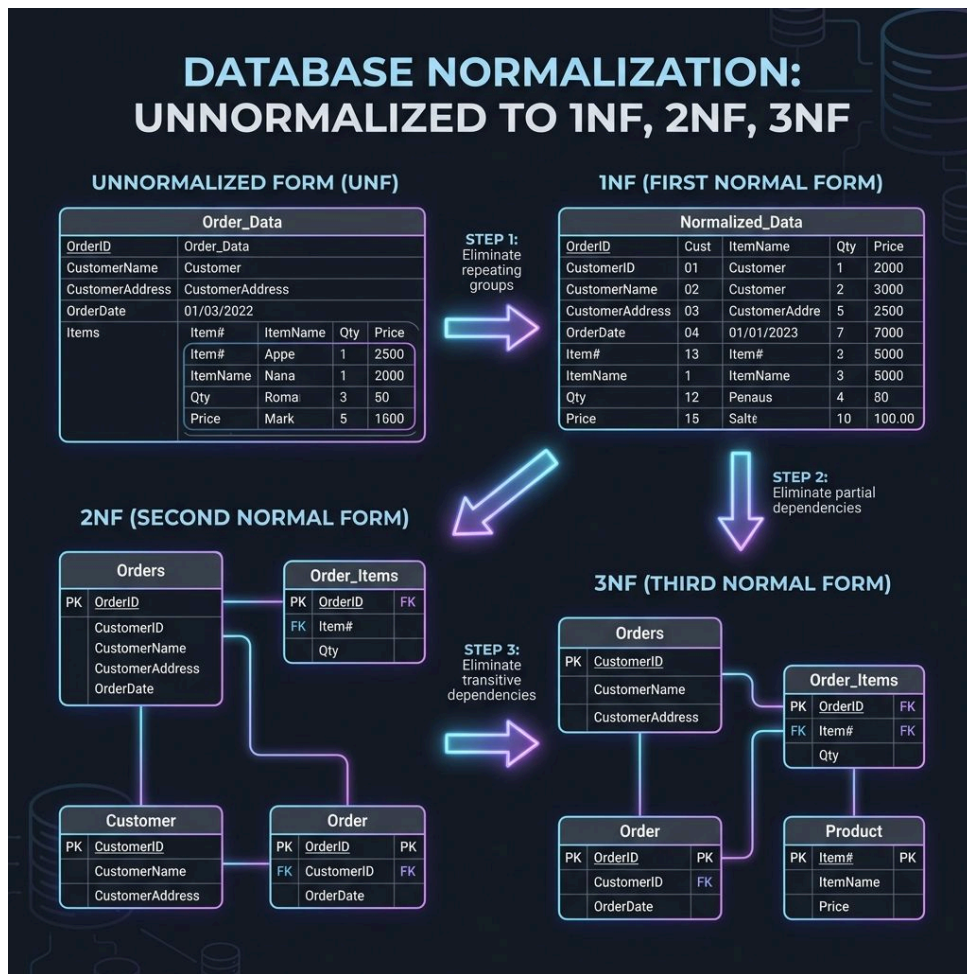


Figure 6.3 – The hierarchical steps of database normalization to ensure data integrity.



Dr. Karen Spärck Jones was a pioneering British computer scientist whose work in the 1970s introduced the concept of Inverse Document Frequency (IDF). Her work is foundational to the search engines we use every day to query massive databases!²

6.4. Algorithmic Logic and Symbolic Representation

The logic of a data solution is represented through symbols and structured language before being implemented in code.

6.4.1. Visual Representation and Flow

System interrelationships and data flow are represented visually using:

- **Sketches and Mock-ups:** To visualize the user interface and basic layout.
- **Annotated Data Flow Diagrams (DFD):** To map the movement of data between processes, data stores, and external entities [cite: 2784, 2785].

²Source: Spärck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. Journal of Documentation, 28(1), 11–21.

6.4.2. Computational Thinking and Algorithm Design

Students use computational thinking to design algorithms that follow a clear **Input, Process, Output (IPO)** structure [cite: 2762].

6.4.3. Pseudocode Development

Well-ordered algorithms are developed using **pseudocode**. This allows developers to plan:

- **Data Insertion:** Adding new records to a data store.
- **Manipulation:** Sorting, filtering, or calculating data.
- **Validation:** Checking data integrity before storage.
- **Presentation:** Formatting data for the user [cite: 2789, 2790, 2791].



Example: Data Validation Logic

```
BEGIN VALIDATE_AGE
  INPUT age
  IF age < 0 OR age > 120 THEN
    OUTPUT "Error: Invalid age entered."
    SET valid_flag = False
  ELSE
    OUTPUT "Age validated successfully."
    SET valid_flag = True
  ENDIF
END
```

6.4.4. Core Programming Constructs

Implementation utilizes basic constructs to control the flow of the program:

- **Assignment:** Storing values in variables.
- **Sequence:** Executing instructions in a specific order.
- **Selection:** Making decisions using conditional statements (if/else).
- **Iteration:** Repeating tasks using loops (for/while).
- **Modularisation:** Breaking code into reusable functions or modules [cite: 2788].

6.4.5. Code Clarity and Documentation

Clear documentation is essential. **Code comments** are mandatory to clarify the intent and functionality of specific programming statements, making the code maintainable for others [cite: 2792].

6.5. Revision Questions

1. Compare and contrast data validation and data verification with real-world examples.
2. Define the components of the DIKW hierarchy and explain how data becomes wisdom.
3. Describe the conditions required for a database table to be in Third Normal Form (3NF).
4. Explain the purpose of a Data Flow Diagram (DFD) in the design phase of a project.
5. Why is de-identification critical when managing large public datasets?
6. Identify the five core programming constructs used in algorithm implementation.

Chapter 7

Prototype Data Solutions

Table of contents

7.1. Solution Planning and Requirements	103
7.2. Database Optimization and SQL Implementation	104
7.3. Generating the Prototype Solution	107
7.4. Validation and Technical Testing	107
7.5. Evaluation and Refinement	109
7.6. Revision Questions	110

Aims and Objectives:

- Define success criteria for user interfaces and programmed components.
- Master complex SQL retrieval and modification techniques.
- Generate functional prototype solutions that perform CRUD operations.
- Implement data validation and technical testing for reliability.
- Evaluate digital solutions against initial requirements and data quality standards.

7.1. Solution Planning and Requirements

Planning is the foundation of any successful prototype. Before writing a single line of SQL, developers must establish the roadmap for their solution.

7.1.1. Success Criteria

Students begin by identifying the specific **Success Criteria** required to plan both the user interface and the programmed components of the proposed solution. Success criteria act as a checklist to ensure the final product meets the needs of the users and the project brief [cite: 581].

**Example Success Criteria: School Library System**

- **Functional:** The system must allow users to search for books by ISBN in under 2 seconds.
- **Data Integrity:** No book record can be deleted if it is currently “checked out”.
- **UI/UX:** The “Return Book” button must be clearly visible and accessible within two clicks from the dashboard.



Alan Cooper is often referred to as the “Father of Visual Basic” and is a pioneer in the field of interaction design. He introduced the concept of **Personas**—goal-directed design tools that use fictional characters to represent user types. His work shifted the focus of software development from what the computer could do to what the **user** needed to achieve, a philosophy that is central to modern prototyping and UX.¹

7.1.2. Data Types and Constraints

It is essential to determine the appropriate **Data Types** (e.g., INTEGER, VARCHAR, DATE) and **Constraints** (e.g., NOT NULL, UNIQUE, CHECK) for each field within the system. These rules prevent invalid data from entering the database at the source [cite: 582].

7.1.3. Defining Relationships

The selection and application of **primary keys** and **foreign keys** must be finalized during the planning stage. This ensures that the relational links between tables are robust and maintain referential integrity [cite: 582].

¹Source: Cooper, A. (2004). **The Inmates Are Running the Asylum**. Sams Publishing.

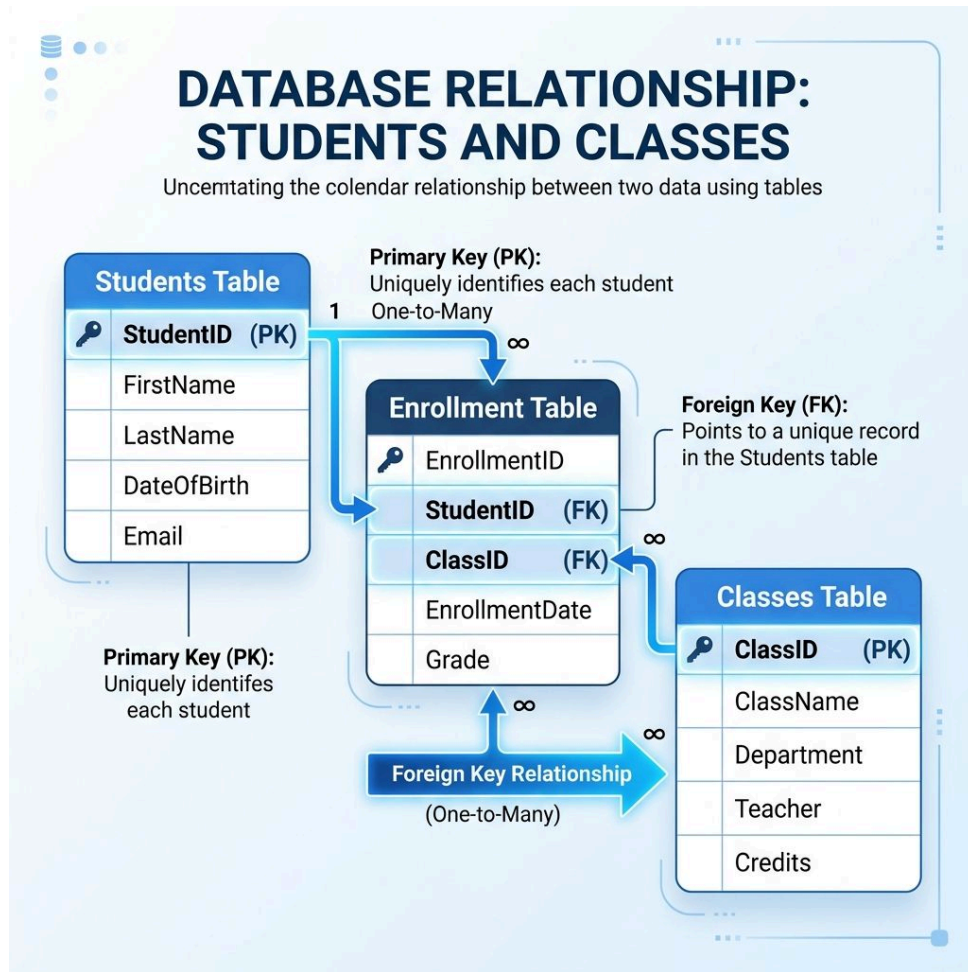


Figure 7.1 – Visualizing the link between Primary and Foreign Keys.

7.2. Database Optimization and SQL Implementation

Once the plan is in place, the focus shifts to optimizing the database structure and implementing the logic required to interact with the data.

7.2.1. Optimization: Reaching 3NF

Existing database structures are evaluated and modified to reach **Third Normal Form (3NF)**. This process ensures the database is efficient, minimizes redundancy, and prevents data anomalies [cite: 583].

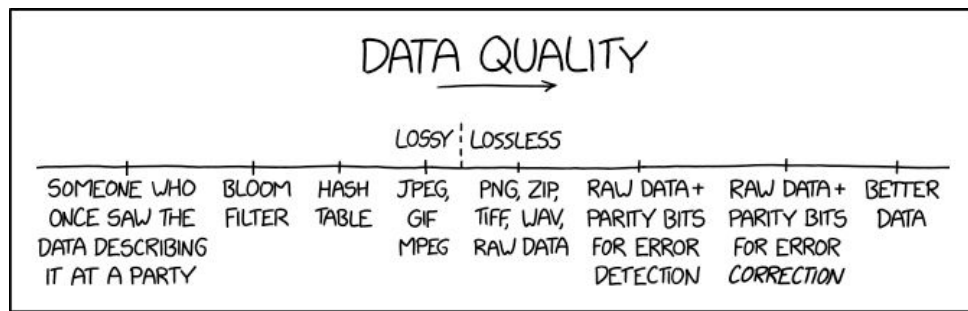
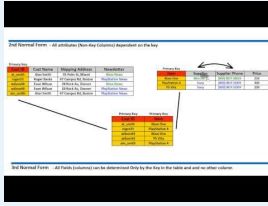


Figure 7.2 – Database quality is only as good as the input logic.²

²Source: xkcd.com/2739



Video: Database Normalization (1NF, 2NF, 3NF) Walk-through – A practical guide to database optimization.
<https://www.youtube.com/watch?v=UrYLYV7WSHM>

7.2.2. Masterful Data Retrieval

Data Retrieval is achieved through complex SQL statements. Students must master the following clauses:

- **SELECT:** Choose specific columns.
- **WHERE:** Filter rows based on conditions.
- **GROUP BY:** Aggregate data into groups.
- **HAVING:** Filter groups.
- **ORDER BY:** Sort the final results [cite: 585].



Deep Dive: Grouping and Filtering

```
SELECT Department, COUNT(StudentID) as TotalStudents
FROM Students
WHERE EnrollmentYear > 2023
GROUP BY Department
HAVING COUNT(StudentID) > 5
ORDER BY TotalStudents DESC;
```

Analysis: This query first filters for students enrolled after 2023. It then groups them by department. The **HAVING** clause acts as a filter **after** the aggregation, only showing departments with more than 5 new students.

7.2.3. Advanced Functions and Joins

To extract deep insights, advanced functions and join operations are utilized:

- **Aggregate Functions:** COUNT, MIN, MAX, AVG.
- **Filtering:** Using IN for set membership.
- **Inner-Joins:** Combining data from multiple tables where keys match.
- **Sub-queries:** Nesting one query inside another for complex logic [cite: 585].

7.2.4. Data Modification (CRUD)

Data modification is handled via SQL commands that manage the lifecycle of data and the tables themselves:

- **CREATE:** Define new tables or views.
- **INSERT:** Add new records.
- **UPDATE:** Modify existing records.
- **DELETE:** Remove records [cite: 586].



SQL Best Practice: The Atomic Update When updating records, always use a specific **WHERE** clause (usually the Primary Key) to avoid accidentally modifying the entire table:

```
-- Safe update
UPDATE Students SET Status = 'Graduated' WHERE StudentID = 1042;

-- Dangerous update (Changes everyone!)
UPDATE Students SET Status = 'Graduated';
```

7.3. Generating the Prototype Solution

The culmination of planning and SQL implementation is the creation of a functional prototype.

7.3.1. Access, Manipulation, and Display

The core objective is to generate a prototype digital solution capable of **accessing, manipulating, and displaying** data effectively. This means the solution should not only query the database but also present that data in a meaningful way to the user [cite: 587].

7.3.2. CRUD Operations

A complete prototype solution must enable the user to perform all standard operations: **insert, update, retrieve, and delete (CRUD)** data. Importantly, these operations should work seamlessly across both single and multiple linked tables [cite: 588].

7.3.3. User Interface (UI) Generation

A functional **User Interface (UI)** is the bridge between the user and the database. The UI must facilitate SQL database interactions—such as forms for data entry and tables for data display—without requiring the end-user to write SQL themselves [cite: 590].

7.4. Validation and Technical Testing

A prototype is only as good as its reliability. Testing ensures the solution is robust and user-friendly.

7.4.1. Data Validation

The system must include **data validation** processes to ensure that all entered data is reliable and valid for storage. This includes checking for correct data types, range checks, and mandatory fields to prevent database errors [cite: 589].

7.4.2. Technical Quality

SQL and programmed components are tested specifically for:

- **Reliability:** Does the system perform consistently?
- **Maintainability:** Is the code easy to update and debug?
- **Efficiency:** Does the SQL run quickly without wasting resources? [cite: 591]

7.4.3. Usability Testing

User interfaces undergo rigorous testing against established **usability principles** (such as visibility of system status and error prevention) to ensure a high-quality user experience [cite: 592].

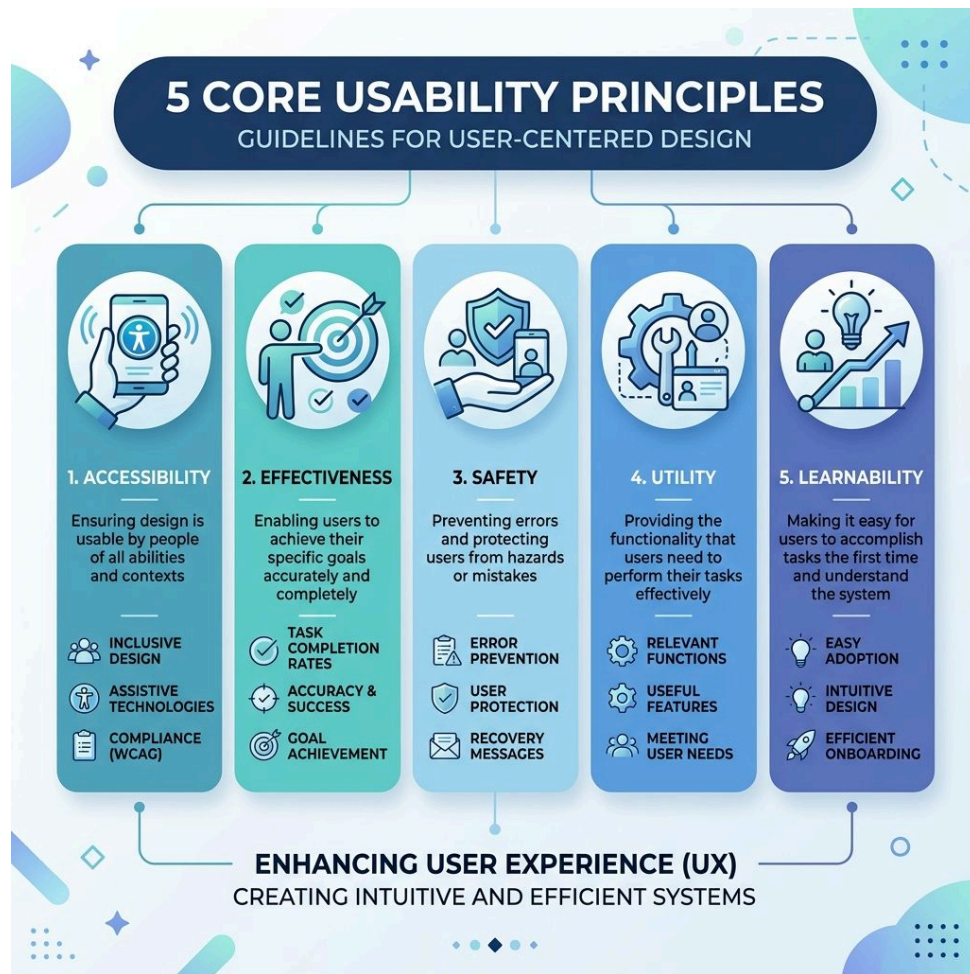


Figure 7.3 – Applying core usability principles to the prototype interface.

7.5. Evaluation and Refinement

The final stage of the prototyping process involves appraising the solution to identify areas for improvement.

7.5.1. Algorithmic Appraisal

Algorithmic steps within the solution are appraised using debugging and testing processes, such as **desk checks**. By manually stepping through the logic, students can identify logical errors before they manifest as software bugs [cite: 594].

7.5.2. Data Quality Evaluation

Overall data quality is evaluated based on two primary success criteria:

- **Accuracy:** Is the data correct and reflective of reality?
- **Completeness:** Are all necessary data points present? [cite: 595]

7.5.3. Measuring Success

The final prototype digital solution is measured against the initial **success criteria** established in the planning phase. This determines the overall effectiveness and suitability of the solution for its intended purpose [cite: 596].

7.6. Revision Questions

1. Why is reaching **Third Normal Form (3NF)** essential for a prototype data solution?
2. Write a complex SQL query that uses **GROUP BY**, **HAVING**, and an **INNER JOIN**.
3. Explain the difference between **data validation** and **usability testing**.
4. Describe how a **desk check** helps in appraising the logic of an algorithm.
5. How do **success criteria** influence the evaluation phase of a digital project?
6. Provide an example of a SQL **SUB-QUERY** and explain its logic.

Part 3

Unit 3: Digital Innovation

Chapter 8

Interactions Between Users, Data, and Digital Systems

Table of contents

8.1. Innovation and Emerging Technologies	115
8.2. Digital Solution Components and Contexts	117
8.3. System Analysis and Modeling	118
8.4. Usability and Visual Communication	119
8.5. Data Flow and Advanced Processes	121
8.6. Revision Questions	122

Aims and Objectives:

- Explore the meaning of innovation and opportunities in emerging technologies.
- Describe components for web, mobile, interactive media, and intelligent systems.
- Analyze system components and observable human-digital interactions.
- Apply the five core usability principles and visual communication elements.
- Model data flow using DFDs and manage advanced data processes.

8.1. Innovation and Emerging Technologies

Innovation is the engine of the digital age, transforming theoretical possibilities into practical solutions.

8.1.1. The Meaning of Innovation

Innovation is more than just technological advancement; it is the process of creating value through new ideas, products, or methods. Students explore how innovation presents diverse opportunities across personal, business, and social domains [cite: 620].



Anita Borg (1949–2003) was an American computer scientist who founded the Institute for Women and Technology (now the Anita Borg Institute) and the Grace Hopper Celebration of Women in Computing. She was a passionate advocate for diversity in the tech industry and believed that technology should be designed by everyone, for everyone. Her work on distributed operating systems was revolutionary, but her legacy is most strongly felt in her efforts to bridge the gender gap in digital innovation.¹

8.1.2. Emerging Technologies

The landscape of digital innovation is currently being redefined by:

- **Deep Learning:** Systems that learn and improve from experience without being explicitly programmed.
- **Neural Networks:** Computational models inspired by the human brain's structure.
- **Natural Language Processing (NLP):** Enabling systems to understand and generate human language [cite: 621].



Case Study: AI in Medical Diagnosis In modern healthcare, **Deep Learning** algorithms are trained on millions of MRI scans to identify early-stage tumors. This **innovation** doesn't replace doctors but provides a powerful **utility** (usability principle) by flagging anomalies that the human eye might miss, significantly improving patient outcomes (social impact).

¹Source: Anita Borg Institute. **Anita Borg: Computer Scientist and Advocate.**



Figure 8.1 – The logic behind modern machine learning systems.²

8.1.3. Personal, Social, and Economic Impacts

The adoption of innovative solutions carries significant weight. Students must determine the potential impacts, such as increased efficiency (economic), improved connectivity (social), and new privacy challenges (personal) [cite: 645].

²Source: xkcd.com/1838

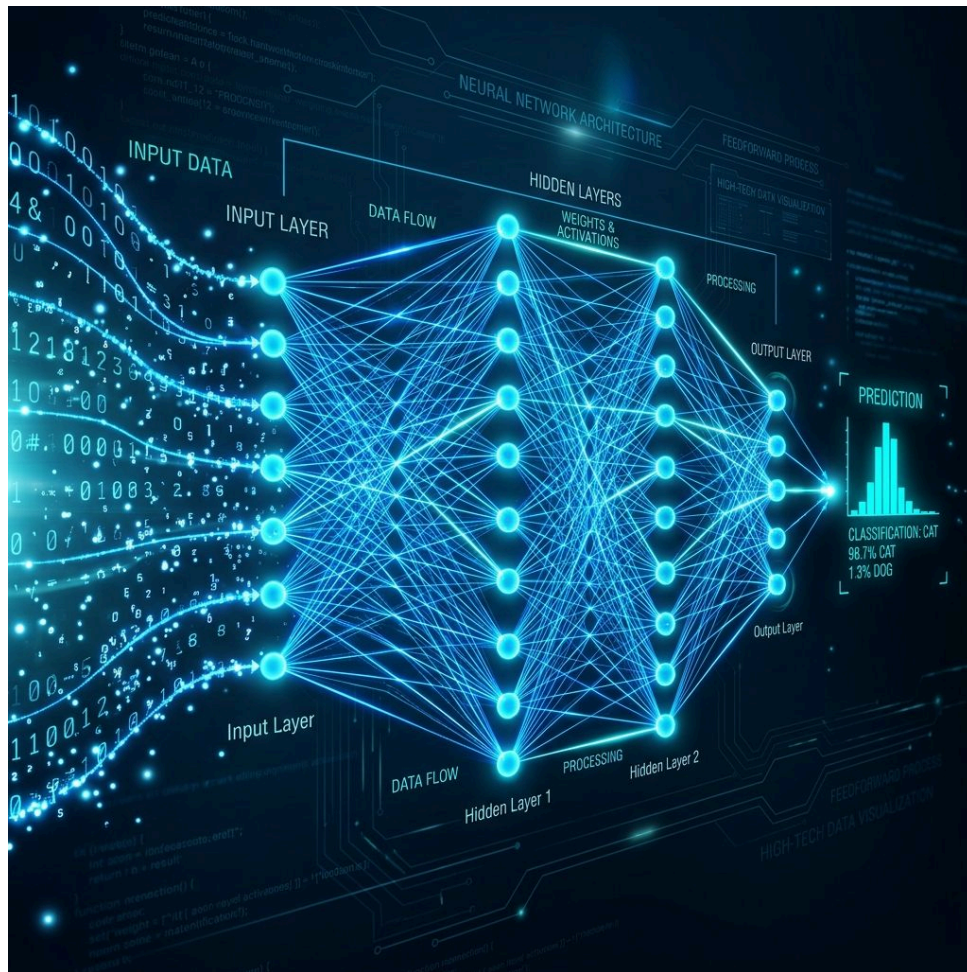


Figure 8.2 – Neural networks are a cornerstone of modern intelligent systems.

8.2. Digital Solution Components and Contexts

The architecture of a digital solution depends heavily on its intended context. Components are recognized and described based on the specific technology context selected for study [cite: 622].

8.2.1. Web Applications

Web applications rely on a client-server architecture. Key components include:

- **Server-Side:** Web and database servers, and pre-processing logic (e.g., PHP, Node.js).
- **Client-Side:** Browsers, devices, and front-end code (HTML/JS).
- **Internal Data:** Structured databases and session management [cite: 623].



Example: E-commerce Interaction

1. **Client-Side:** User clicks “Add to Cart” (Browser event).
2. **Server-Side:** A PHP script receives the request and checks the **Internal Data** (Database) for stock availability.
3. **Client-Side:** The browser receives a “Success” message and updates the cart count using JavaScript.

8.2.2. Mobile Applications

Mobile solutions are characterized by their interaction with physical hardware and sensors:

- **Hardware:** Touchscreens, accelerometers, and GPS.
- **Input/Output:** Voice recognition, haptic feedback, and camera.
- **Logic:** Event handlers for user interactions and specific application data structures [cite: 623].

8.2.3. Interactive Media

Interactive media systems prioritize the user's engagement with multimedia:

- **Peripheral Devices:** Joysticks, VR headsets, and specialized controllers.
- **Assets:** Multimedia files (video, 3D models) and object libraries.
- **Storage:** External data stores used to manage large asset libraries [cite: 623].

8.2.4. Intelligent Systems

Intelligent systems bridge the gap between the digital and physical worlds:

- **Sensors & Actuators:** Devices that detect environment changes and those that perform actions (e.g., motors).
- **Data Streams:** Handling both analogue and digital data inputs.
- **Communication:** Network protocols (e.g., MQTT, Zigbee) and specific code libraries for AI or hardware control [cite: 623].

8.3. System Analysis and Modeling

Analyzing a system requires breaking it down into its constituent parts to understand how they interact to achieve a goal.

8.3.1. Identifying System Components

Problems are analyzed to identify the essential building blocks of a solution:

- **Hardware:** The physical devices involved.
- **Software:** The code and applications.
- **Data:** The information being processed.
- **Networks:** The communication channels.
- **Users:** The people interacting with the system [cite: 624, 625].

8.3.2. Human-Digital Interactions

Students identify observable interactions within the system. This includes human-human communication facilitated through digital platforms like social media, email, or video conferencing [cite: 627].

8.3.3. Technical Specifications

Defining the technical elements is crucial for system interoperability:

- **Protocols & Standards:** The rules governing data exchange (e.g., HTTP, TCP/IP).
- **Inputs & Outputs:** The data entering and leaving the system.
- **Control Mechanisms:** The logic that governs system behavior [cite: 626, 628, 629].

8.3.4. Modeling Processes

Interactions and processes are represented using logical diagrams and consistent symbols. This allows developers to visualize complex system behaviors before implementation [cite: 630].

8.4. Usability and Visual Communication

A system is only effective if its users can interact with it efficiently and intuitively.

8.4.1. Core Usability Principles

Students must symbolize and explain the five core usability principles [cite: 631, 632]:

Principle	Description
Accessibility	Ensuring the system is usable by people with diverse abilities and disabilities.
Effectiveness	How accurately and completely users can achieve their goals.
Safety	Protecting users from dangerous conditions and unintended actions.
Utility	The range of functions the system provides to meet user needs.
Learnability	How easy it is for new users to accomplish basic tasks.

8.4.2. Interface Appraisal

A variety of interfaces are explored to understand user interaction [cite: 633]. To support diverse user profiles, flexible development methods are investigated [cite: 641]. Interfaces are appraised using **Heuristic Reviews** to ensure they meet established usability standards [cite: 646].



Video: 10 Usability Heuristics for User Interface Design – A guide to evaluating digital interfaces.

https://www.youtube.com/watch?v=6B7L_O0_8Ww



Heuristic Spotlight: Error Prevention A classic example of the **Safety** principle is the “Confirm Delete” dialog box. By requiring an extra step for a destructive action, the system prevents accidental data loss, thereby enhancing the system’s **reliability**.

8.4.3. Visual Communication

Visual communication is the art of presenting information clearly. It is guided by specific elements and principles [cite: 643, 644]:

- **Elements:** Space, line, colour, shape, texture, tone, form, proportion, scale.
- **Principles:** Balance, contrast, proximity, harmony, alignment, repetition, hierarchy.

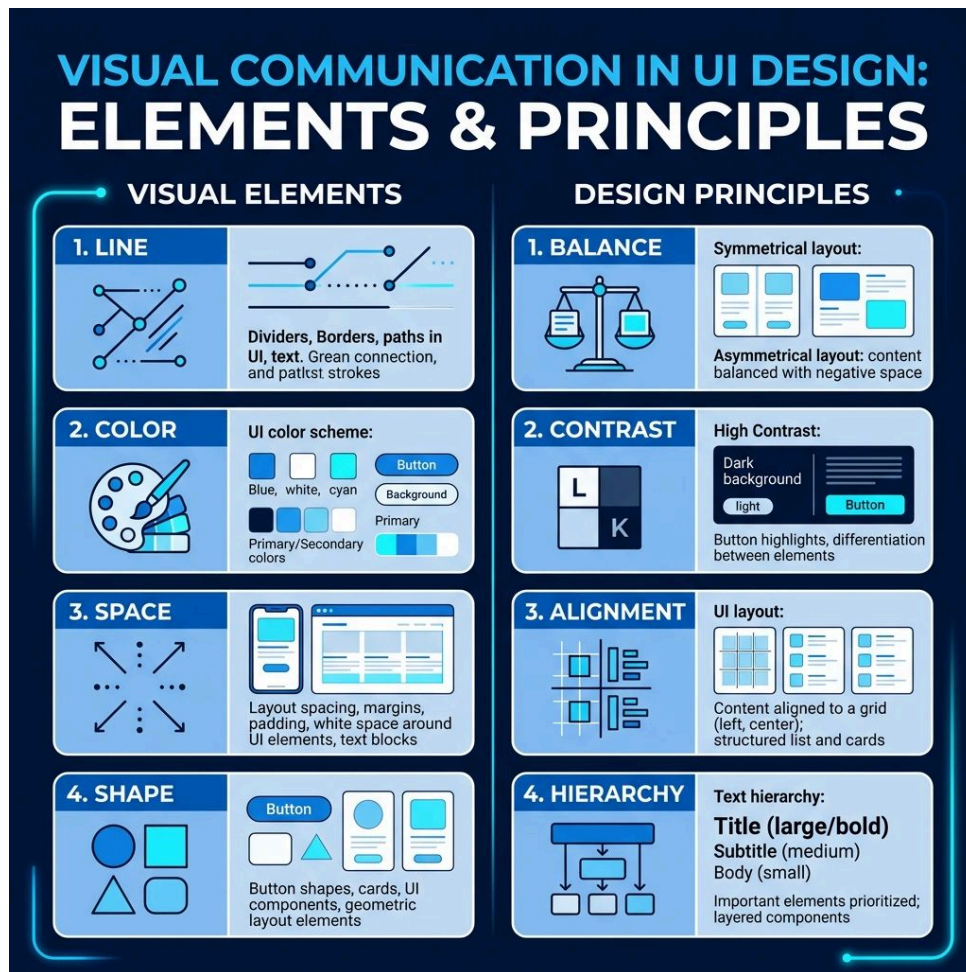


Figure 8.3 – Applying visual communication principles to UI design.

8.5. Data Flow and Advanced Processes

The movement of data through a system must be carefully mapped to ensure integrity and efficiency.

8.5.1. Data Flow Diagrams (DFD)

Data flow through a system is represented visually using **Data Flow Diagrams (DFD)**. DFDs use standardized symbols to represent processes, data stores, external entities, and the flow of data between them [cite: 634].

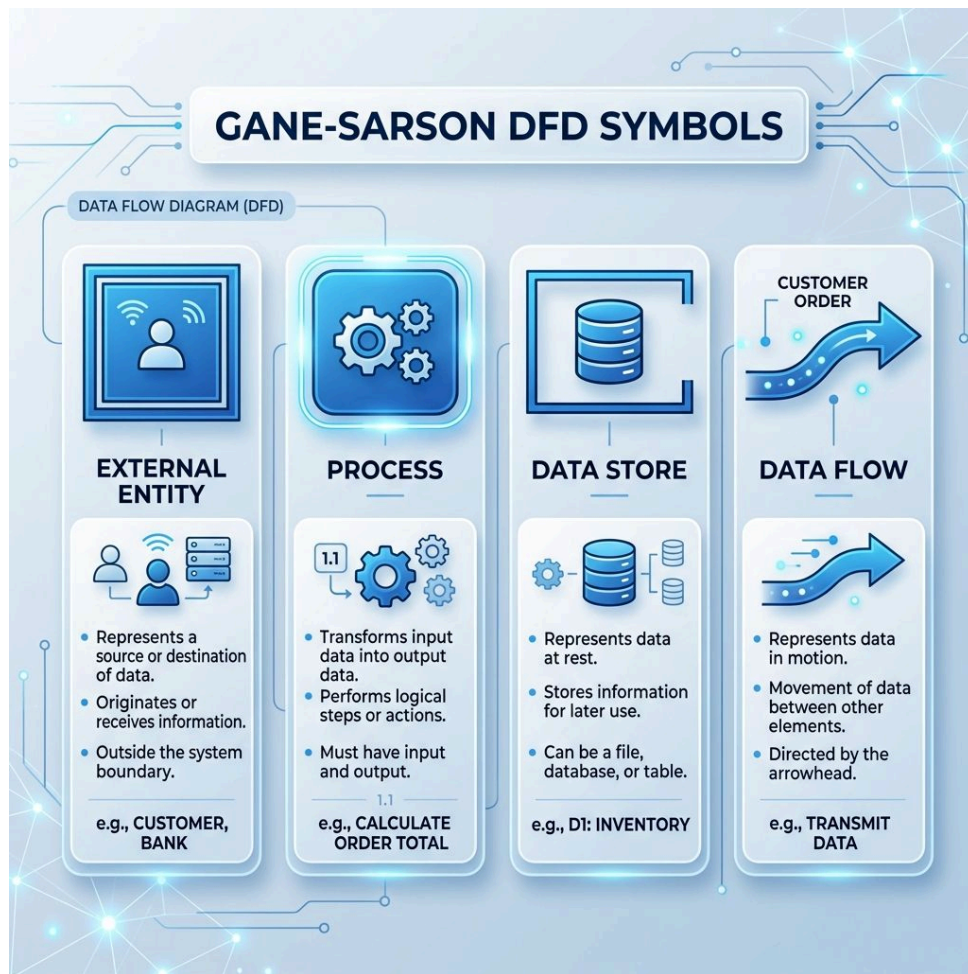


Figure 8.4 – Standard symbols used in Data Flow Diagrams.

8.5.2. Advanced Data Handling

Students symbolize and use advanced data processes to manage complex datasets:

- **Table Joins:** Combining data from multiple tables based on a related column.
- **Referential Integrity:** Ensuring that relationships between tables remain consistent [cite: 635].
- **Redundancy & Anomalies:** Applying strategies to reduce data duplication and prevent update anomalies [cite: 635].

8.5.3. Prototyping Synthesis

Methods for synthesizing user interfaces, processing logic, and data components are explored to generate comprehensive prototype solutions [cite: 642].

8.6. Revision Questions

1. Compare **Machine Learning** and **Neural Networks** in the context of digital innovation.
2. Describe the hardware and software components unique to an **Intelligent System**.
3. Identify three technical elements required for system interoperability.
4. Explain the five core **Usability Principles** and why they are critical for UX design.

5. Contrast **Visual Elements** (like colour and line) with **Visual Principles** (like balance and hierarchy).
6. Draw a simple **Data Flow Diagram (DFD)** for a library book return system.

—

Chapter 9

Real-world Problems and Solution Requirements

Table of contents

9.1. Problem Analysis and Scoping	125
9.2. Project Management and Development Tools	126
9.3. System Interactions and Data Structures	129
9.4. Algorithmic Design and Computational Thinking	131
9.5. Revision Questions	134

Aims and Objectives:

- Break down complex problems to identify development areas.
- Define functional and non-functional requirements and success criteria.
- Utilize project management and development tools effectively.
- Analyze system interactions and data structure appropriateness.
- Apply computational thinking to algorithmic design.
- Develop and present a comprehensive technical proposal.

9.1. Problem Analysis and Scoping

Every great digital solution starts with a clear understanding of the problem it aims to solve.

9.1.1. Breaking Down the Problem

Complex problems are often overwhelming. Students must break them down to identify manageable aspects and specific areas for development. This process of decomposition ensures that no critical component is overlooked [cite: 651, 652].

9.1.2. Scope, Constraints, and Limitations

Defining the **scope** (what the solution will and won't do) is essential. Alongside this, students must identify:

- **Constraints:** Fixed requirements (e.g., must run on Android).
- **Limitations:** Factors that restrict the solution (e.g., bandwidth or storage capacity) [cite: 653, 654].

9.1.3. Requirements and Success Criteria

A project's success is measured against its requirements:

- **Functional Requirements:** What the system **must do** (e.g., process a payment).
- **Non-functional Requirements:** How the system **must perform** (e.g., load in under 2 seconds) [cite: 655, 1277].
- **Success Criteria:** Specific metrics used to measure the quality, appropriateness, and effectiveness of the final solution [cite: 656, 1282].

**Example: Smart Canteen App**

- **Functional:** The app must allow students to order food and pay using their school ID card.
- **Non-functional:** The system must support up to 500 concurrent users during the lunch rush without crashing.
- **Success Criteria:** 95% of users report that they can complete an order in under 60 seconds.

9.1.4. Stakeholders and Policies

Identifying relevant **stakeholders** (users, owners, regulators) is critical. Projects must also consider existing standards or policies (e.g., data privacy laws) that influence development [cite: 1278, 1279].

9.2. Project Management and Development Tools

Managing the development lifecycle requires the right tools and a structured approach to collaboration.

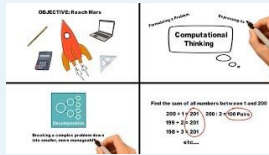
9.2.1. Programming Development Tools

To create effective solutions, developers must utilize programming development tools (like IDEs and debuggers) proficiently. These tools ensure that code is written, tested, and refined efficiently [cite: 648].

9.2.2. Managing the Project

Project management tools (like Gantt charts or Kanban boards) are used to track:

- **Tasks & Milestones:** What needs to be done and by when.
- **Dependencies:** Which tasks must be completed before others can begin.
- **Resources:** The people and technology required for each stage [cite: 649].



Video: What is Computational Thinking? – The core of digital problem solving.

<https://www.youtube.com/watch?v=q6S-77oI4H0>

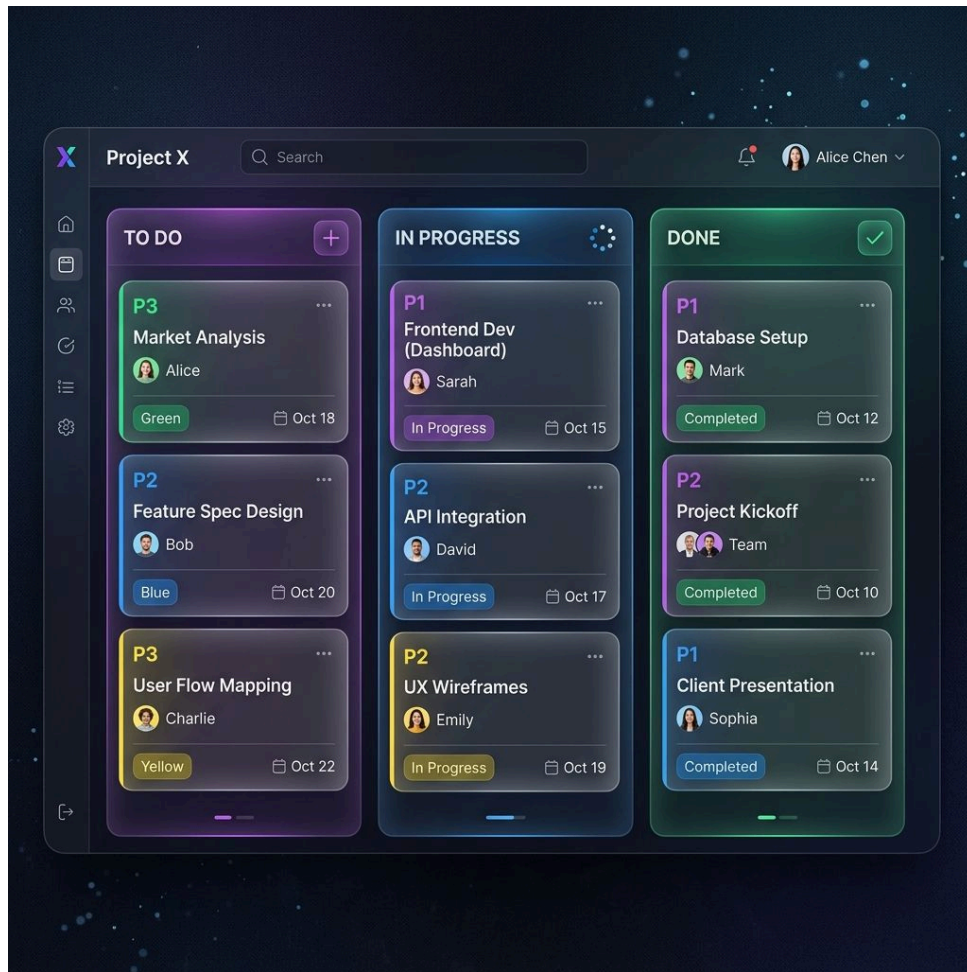


Figure 9.1 – Using Kanban boards to visualize and manage project tasks effectively.

9.2.3. Evaluating Ideas

Effective project management involves constant evaluation. Through innovation and collaboration, teams can assess potential ideas to find the most effective approach for the given problem [cite: 660].



Figure 9.2 – A well-defined workflow is the backbone of successful development!¹

9.3. System Interactions and Data Structures

Understanding how data moves through a system and how it is stored is fundamental to technical planning.

9.3.1. Input, Process, Output (IPO)

System interactions are described in terms of the **IPO model**:

- **Inputs:** The raw data or user commands entering the system.

¹Source: xkcd.com/1172

- **Processes:** The algorithmic steps that transform or validate the data.
- **Outputs:** The final results or displays presented to the user [cite: 657].



Example: Smart Thermostat IPO

- **Input:** Current ambient temperature sensor (22°C) and user setpoint (24°C).
- **Process:** Compare current temp with setpoint. If current < setpoint, activate the heating relay.
- **Output:** Digital display shows “Heating Active” and sends a signal to the furnace.

9.3.2. Human-System Interaction

Interaction styles vary based on technology:

- **Machine-Human:** Automated responses like chatbots or diagnostic notifications.
- **Human-Machine:** User-driven inputs such as voice commands or physical gestures [cite: 658].

9.3.3. Data Structures and Sources

Students must choose the most appropriate way to store data. This involves comparing:

- **Flat-file Structures:** Simple data storage (e.g., a single CSV file).
- **Relational Databases:** Complex, linked tables (e.g., SQL databases) [cite: 659].

Furthermore, different **file formats** (like JSON vs. XML) and **data structures** (like arrays vs. dictionaries) are evaluated to determine their suitability for the project context [cite: 661, 662].

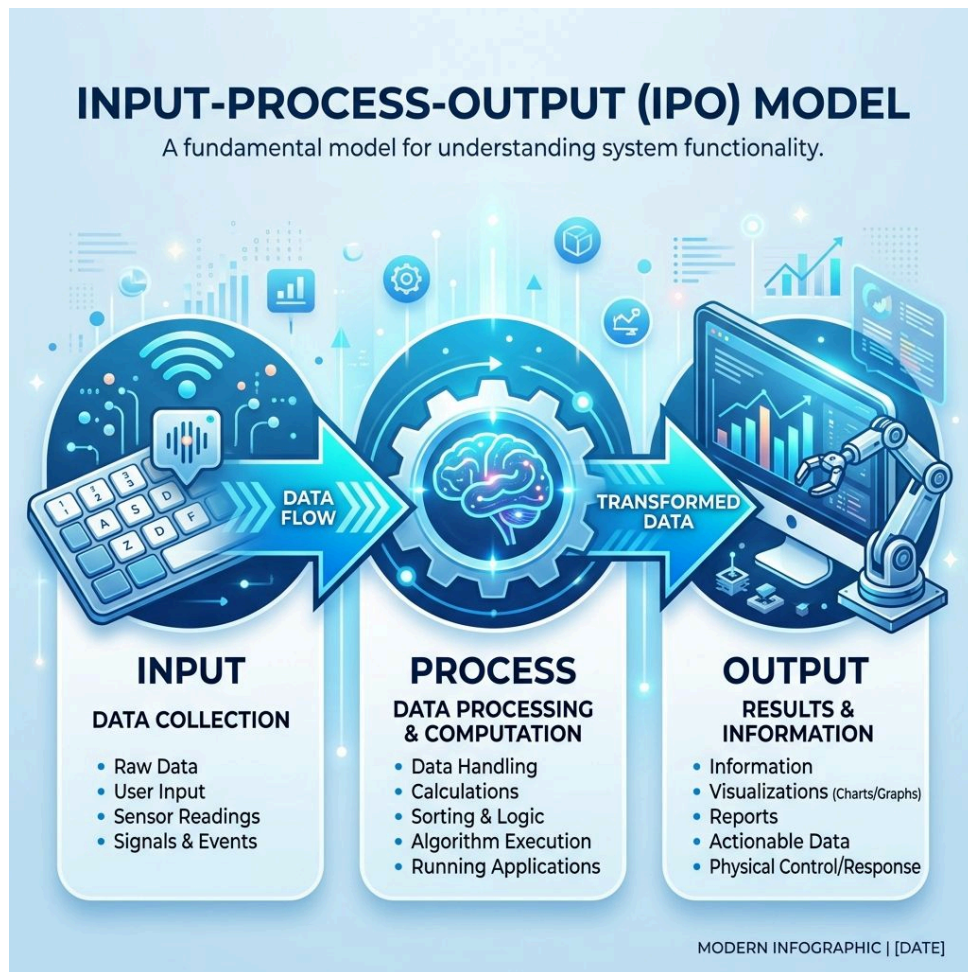


Figure 9.3 – The IPO model provides a simple yet powerful framework for system analysis.

9.4. Algorithmic Design and Computational Thinking

Computational thinking is the cognitive process used to formulate problems and their solutions.

9.4.1. The Thinking Process

Students apply core computational thinking processes to identify possible algorithmic approaches:

- **Creating:** Developing new logic from scratch.
- **Debugging:** Systematically identifying and fixing errors.
- **Persevering:** Working through complex logical challenges.
- **Collaborating:** Sharing ideas and code to improve the final solution [cite: 677].



Computational Thinking in Practice: The Infinite Loop A student discovers their “Average Grade” calculator never stops running.

- **Debugging:** They add `print` statements to see the value of the loop counter.
- **Persevering:** They realize the condition `while i > 0` is always true because `i` is never decremented.
- **Creating:** They add `i = i - 1` to fix the logic.

9.4.2. Basic Constructs

Algorithm implementation utilizes fundamental building blocks:

- **Assignment:** Storing values.
- **Sequence:** Linear execution of steps.
- **Selection & Condition:** decision-making logic.
- **Iteration:** Repeating tasks (loops).
- **Modularisation:** Dividing the program into small, manageable modules [cite: 665].

9.4.3. Drafting and Documentation

Well-ordered and unambiguous algorithms are drafted using **pseudocode**. This ensures that data validation and presentation logic are planned before coding begins [cite: 672, 674]. High modularity and readability are maintained by using language-correct **comment syntax** to explain the purpose of each module [cite: 663, 675].



Video: Pseudocode Explained – Using pseudocode to map out logic before implementation.

<https://www.youtube.com/watch?v=qfckDdsEIq8>



Radia Perlman is an American software designer and network engineer. She is most famous for her invention of the Spanning Tree Protocol (STP), which is fundamental to the operation of network bridges and keeps the internet running smoothly!²

9.4.4. The Technical Proposal

The final stage of the planning phase is communicating the proposed solution to stakeholders.

9.4.5. Styles of Presentation

Students observe and compare different styles of presenting technical information. A proposal for a technical audience will differ significantly from one intended for a business manager or end-user [cite: 680].

9.4.6. Tailoring the Message

The final technical proposal is communicated through a presentation tailored to a specific audience. This requires:

- **Appropriate Language:** Using digital technology-specific terms correctly.
- **Referencing:** Following conventions to cite sources and research.
- **Mode-appropriate Features:** Using the right medium (e.g., slide deck, formal report) [cite: 681, 686, 682, 683].

9.4.7. Visual Communication in Proposals

Visuals are used to clarify complex information about the problem and the programmed components. This includes:

- **Sketches & Wireframes:** Visualizing the user interface.
- **Diagrams:** Representing the data flow or system architecture [cite: 684, 685].

²Source: Perlman, R. (1985). **An algorithm for distributed computation of a spanning tree in an extended LAN**. ACM SIGCOMM Computer Communication Review, 15(4), 44–53.

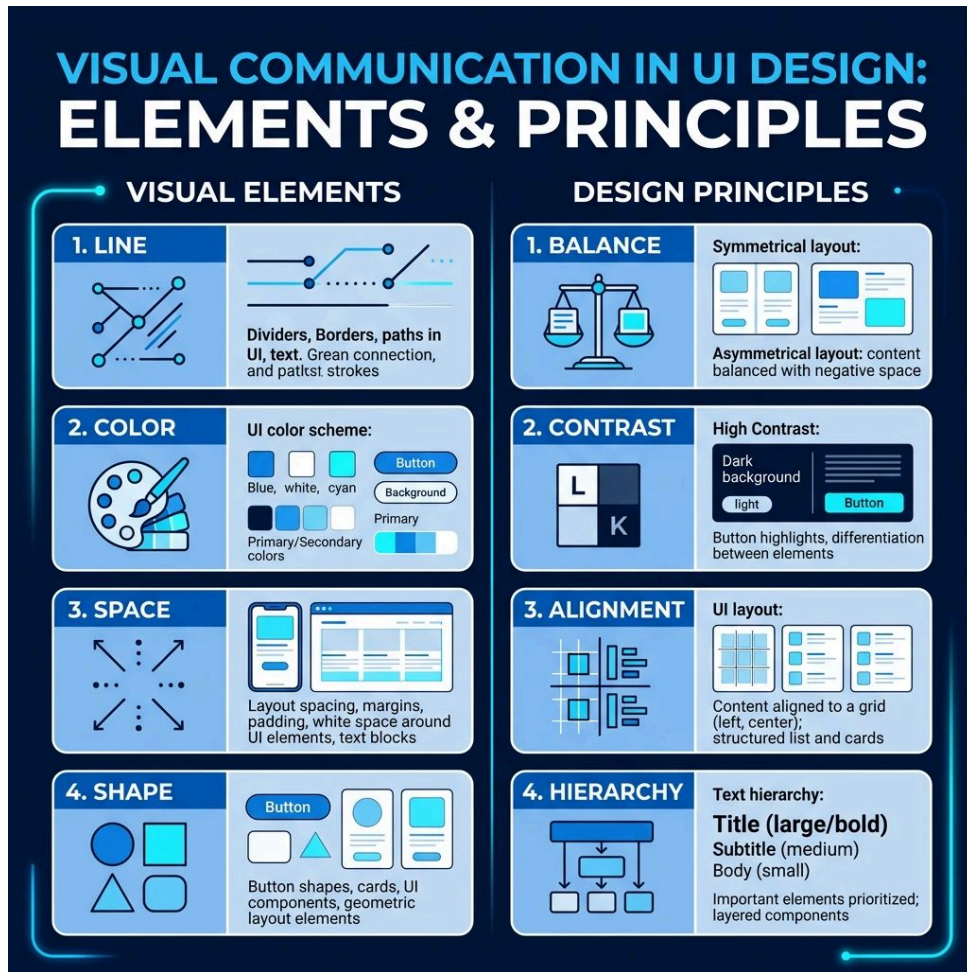


Figure 9.4 – Using visual communication to clarify technical proposals.

9.5. Revision Questions

1. Differentiate between **Functional** and **Non-functional** requirements.
2. How do **success criteria** help in the final evaluation of a digital solution?
3. Contrast **relational databases** with **flat-file structures**. Which is better for a complex system with linked data?
4. Identify the four core processes of **Computational Thinking**.
5. Why is it important to tailor a **technical proposal** to a specific audience?
6. Explain the role of **modularisation** in making an algorithm readable and maintainable.

Chapter 10

Innovative Digital Solutions

Table of contents

10.1. Prototyping and Visual Design	137
10.2. Technical System Implementation	138
10.3. Advanced Data Manipulation with SQL	140
10.4. Systems Thinking and Synthesis	141
10.5. Testing, Evaluation, and Refinement	143
10.6. Revision Questions	144

Aims and Objectives:

- Refine concepts and generate low-fidelity UI prototypes.
- Implement technical systems with functional modules and file operations.
- Master advanced SQL for data definition, modification, and retrieval.
- Apply systems thinking to model and synthesize digital solutions.
- Evaluate and refine prototypes through technical debugging and usability testing.

10.1. Prototyping and Visual Design

Prototyping is an iterative process that allows developers to test ideas and gather feedback before full-scale implementation.

10.1.1. Idea Refinement

Students must refine initial concepts for individual components to ensure **technical feasibility**. This involves assessing whether a proposed feature can be realistically built with available tools and resources [cite: 688].

10.1.2. Low-Fidelity UI Prototypes

Before writing code, low-fidelity prototypes are generated to map out the user journey. Tools include:

- **Sketches & Storyboards:** For quick visualization of flow.
- **Wireframes & Mock-ups:** For layout and structural planning.
- **Sitemaps & Mood Boards:** For organizational and aesthetic direction [cite: 704].

10.1.3. Visual Communication Principles

Applying visual communication elements (like space and colour) and principles (like balance and hierarchy) ensures the user interface is both intuitive and aesthetically pleasing [cite: 643, 644].

**Design Example: High-Priority Alert**

- **Contrast:** Using a bright red background with white text for a “Critical System Failure” message.
- **Hierarchy:** Making the error code (“ERROR 404”) larger than the troubleshooting steps to immediately catch the user’s attention.
- **Space:** Surrounding the “Retry” button with empty space (negative space) to prevent accidental clicks on “Cancel”.

10.1.4. Demonstration

The process concludes with a **demonstration**, where a working prototype is presented to stakeholders to validate the direction of the solution [cite: 689].

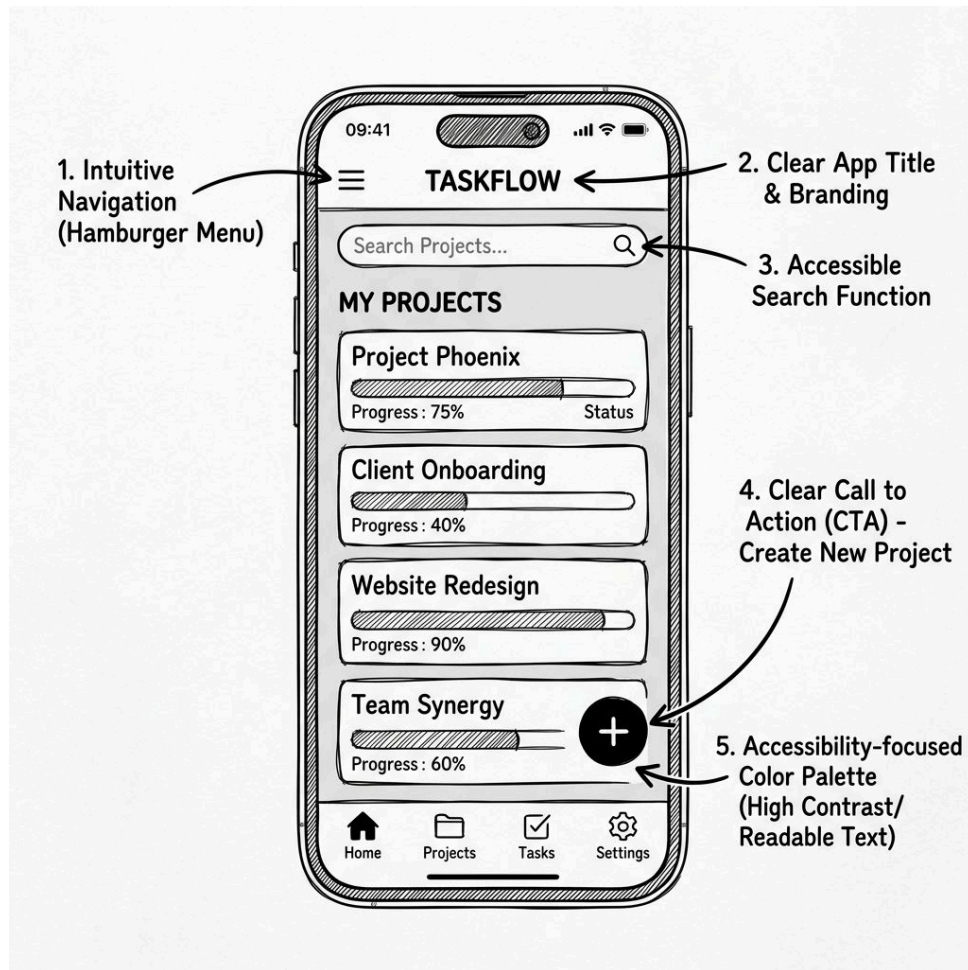


Figure 10.1 – Wireframing is a critical step in low-fidelity prototyping.

10.2. Technical System Implementation

Moving from design to code requires the generation of functional modules and robust data handling.

10.2.1. Programmed Modules

Students generate functional modules that interact with users, handle 2D data sources, and validate data inputs. These modules form the logical core of the digital solution [cite: 707, 708, 709, 710].



Evelyn Berezin (1925–2018) was an American computer scientist who invented the first computer-driven word processor. Her company, Redactron, produced the “Data Secretary” in 1971, which revolutionized office work by allowing users to edit and store documents digitally. Her work on large-scale automated systems, including the first computerized airline reservation system for United Airlines, demonstrated the power of digital innovation to solve complex organizational problems.¹

¹Source: The New York Times. **Evelyn Berezin, 93, Dies; Pioneer of the Word Processor** (2018).

10.2.2. File Operations

Robust solutions often need to persist data outside of a database. Implementation includes code that can:

- **Create & Open:** Initializing file access.
- **Read & Write:** Moving data between memory and storage.
- **Close:** Ensuring data integrity and freeing system resources [cite: 706].

10.2.3. Control and Documentation

Programming development tools are used to generate code that controls all interactions within the solution [cite: 705, 711]. Throughout this process, consistent **code comments** are used to communicate and clarify the purpose of each statement, ensuring the project remains maintainable [cite: 712].

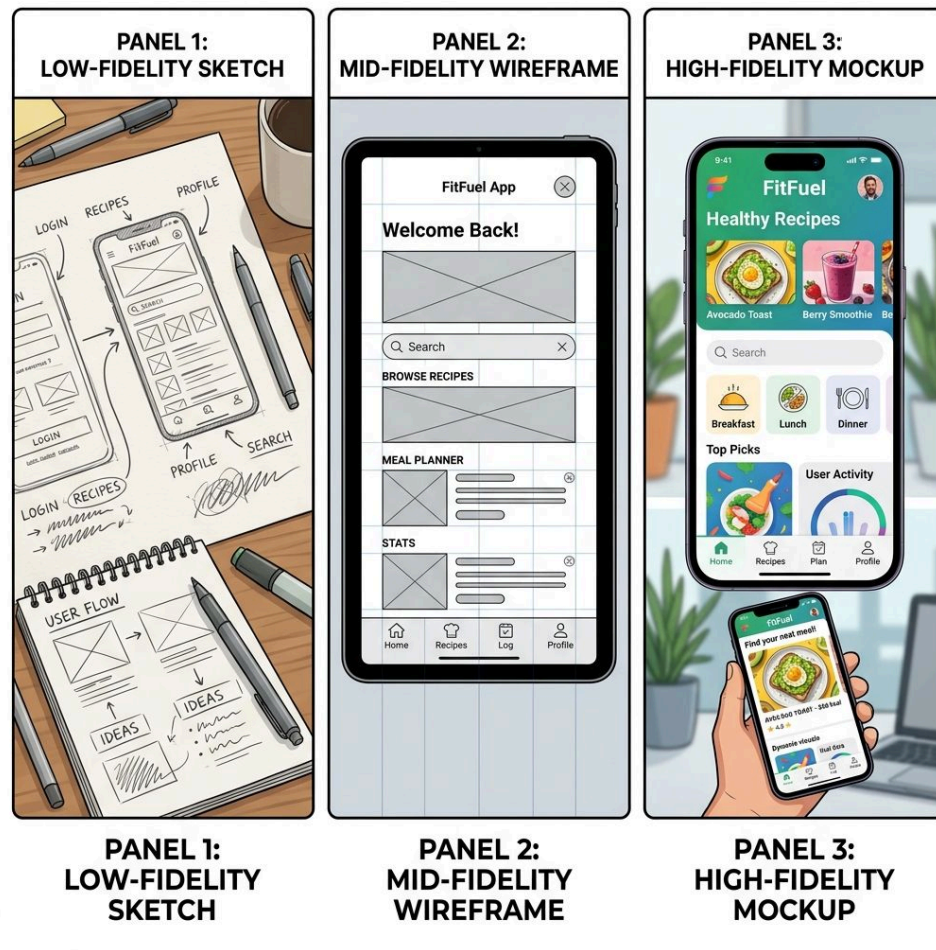


Figure 10.2 – The evolution of a prototype from conceptual logic to functional system.

10.3. Advanced Data Manipulation with SQL

A robust prototype requires advanced control over its data structures and stored information.

10.3.1. Data Definition (DDL)

Students use **Data Definition Language (DDL)** to manage the structure of their database. This includes:

- **CREATE:** Defining new tables and views.
- **DROP:** Removing entire structures from the system.
- **ALTER:** Modifying existing table definitions (e.g., adding a column) [cite: 692].

10.3.2. Data Modification (DML)

Data Manipulation Language (DML) is used to manage the information stored within those structures. This involves implementing **INSERT**, **UPDATE**, and **DELETE** operations [cite: 691].



SQL Walkthrough: Building a User Base

```
-- DDL: Create the structure
CREATE TABLE Users (
  UserID INT PRIMARY KEY,
  Username VARCHAR(50) NOT NULL,
  JoinDate DATE
);

-- DML: Insert the data
INSERT INTO Users (UserID, Username, JoinDate)
VALUES (1, 'AliceTech', '2024-04-18');
```

10.3.3. Complex Retrieval

Advanced SELECT clauses are applied to extract relevant data for the solution. This includes using:

- **WHERE & GROUP BY:** For filtering and aggregation.
- **HAVING & ORDER BY:** For group filtering and sorting.
- **Sub-selection & Inner-Joins:** To combine and nest queries for complex logic [cite: 693].

10.3.4. Justification

Crucially, developers must provide a rationale for selecting specific data from existing sources. This justification ensures the data chosen directly solves the identified problem [cite: 714].

10.4. Systems Thinking and Synthesis

Systems thinking allows developers to view the solution as a collection of interconnected parts working together.

10.4.1. Conceptual Modeling

Students apply **systems thinking** to develop a model that identifies:

- **System Boundaries:** What is inside and outside the solution.
- **Properties:** The characteristics of the system.
- **Inputs & Outputs:** How data flows into and out of the system.
- **UI & Controls:** How the user interacts with and governs system behavior [cite: 694, 695, 696, 697, 698, 699].



Systems Thinking: The Schema Ripple Effect If a developer decides to change the UserID from an integer to a UUID (a long string), they must consider the ripple effect:

- **Database:** DDL ALTER command needed.
- **Logic:** All modules reading UserID must now handle strings.
- **UI:** Forms and URL parameters might need more space for the longer ID.

10.4.2. **Component Synthesis**

Synthesis is the process of integrating the user interface, processing logic, and data components to form a coherent, functional prototype digital solution [cite: 713].



Video: The Prototyping Process – From conceptual logic to a functional user interface.

<https://www.youtube.com/watch?v=JMjozqJSv2M>

10.5. Testing, Evaluation, and Refinement

The final phase of prototyping is a rigorous evaluation to ensure the solution is robust, effective, and user-friendly.

10.5.1. Algorithm Appraisal

Algorithms are evaluated for their **reliability**, **maintainability**, **efficiency**, **effectiveness**, and **usability**. Based on this appraisal, developers make justified recommendations for further refinement [cite: 716].

10.5.2. Technical Debugging

Computational thinking processes, such as **desk checks** and automated debugging techniques, are used to identify and fix logical and syntax errors in program modules [cite: 717, 718].

10.5.3. Usability Testing

User interfaces are refined through:

- **Heuristic Reviews:** Comparing the UI against established usability principles.
- **UX Testing:** Observing and recording real user interactions to identify friction points [cite: 718, 719, 721].

10.5.4. Impact and Success Evaluation

The final prototype is appraised against the initial **success criteria**. This evaluation determines whether the solution successfully addresses the identified problem and meets the needs of its stakeholders [cite: 722, 723, 724, 720].

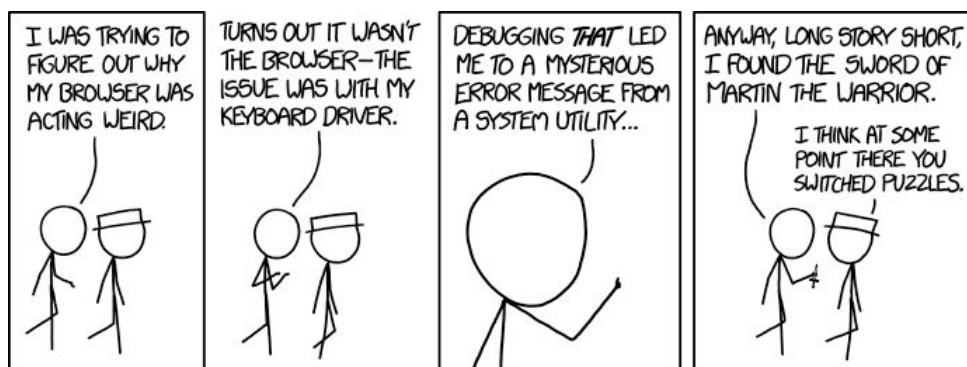


Figure 10.3 – Debugging is as much about persistence as it is about logic!²

²Source: xkcd.com/1722

10.6. Revision Questions

1. Compare **low-fidelity** and **high-fidelity** prototyping. When is each most appropriate?
 2. Explain the difference between **Data Definition Language (DDL)** and **Data Manipulation Language (DML)**.
 3. List the five metrics used to appraise an algorithm.
 4. How does **heuristic review** differ from **UX testing**?
 5. Why is it necessary to justify the selection of specific data sources in a technical solution?
 6. Describe how **systems thinking** helps in identifying system boundaries.
-

Part 4

Unit 4: Digital Impacts

Chapter 11

Digital Methods for Exchanging Data

Table of contents

11.1. Security and Authentication Strategies	149
11.2. Encryption Algorithms and Analysis	152
11.3. Privacy, Ethics, and Data Management	152
11.4. Network Principles and Performance	153
11.5. Modern Data Exchange Methods	155
11.6. Revision Questions	156

Aims and Objectives:

- Recognise encryption and authentication strategies for secure transmission.
- Analyse classical and modern encryption algorithms.
- Apply privacy principles and ethical considerations to data management.
- Understand network performance metrics and core transmission principles.
- Utilize modern data exchange methods including REST, JSON, and APIs.

11.1. Security and Authentication Strategies

Securing data as it travels across networks is a multi-layered challenge involving identity verification and robust mathematical protection.

11.1.1. Identity and Authentication

Authentication strategies ensure that only authorized users can access sensitive data. Students distinguish between:

- **Two-Factor / Multi-Factor Authentication (2FA/MFA):** Using verification codes sent to trusted devices.
- **Biometrics:** Using physical characteristics like fingerprints or facial recognition for identity verification [cite: 748, 749].



Dorothy Denning is an American computer scientist known for her pioneering work in cryptography and network security. She developed several models for secure information flow and was one of the first to propose intrusion detection systems (IDS). Her research into the “lattice model” of secure information flow and her work on digital signatures and data encryption have made her one of the most influential figures in modern cybersecurity.¹

**Example: 2FA in Practice**

1. **Something you know:** User enters their password on a banking website.
2. **Something you have:** The website sends a one-time password (OTP) to the user’s smartphone.
3. **Result:** Even if an attacker steals the password, they cannot access the account without physical possession of the smartphone.

11.1.1.2. Symmetric Encryption

Symmetric encryption uses a single, shared key for both encryption and decryption. Key algorithms include:

- **DES & Triple DES:** Early standards, now largely replaced by more secure methods.
- **AES (Advanced Encryption Standard):** The global standard for high-security data encryption.
- **Blowfish & Twofish:** Fast and flexible alternatives to traditional standards [cite: 750].

¹Source: National Security Agency. **Dorothy Denning: Hall of Honor.**

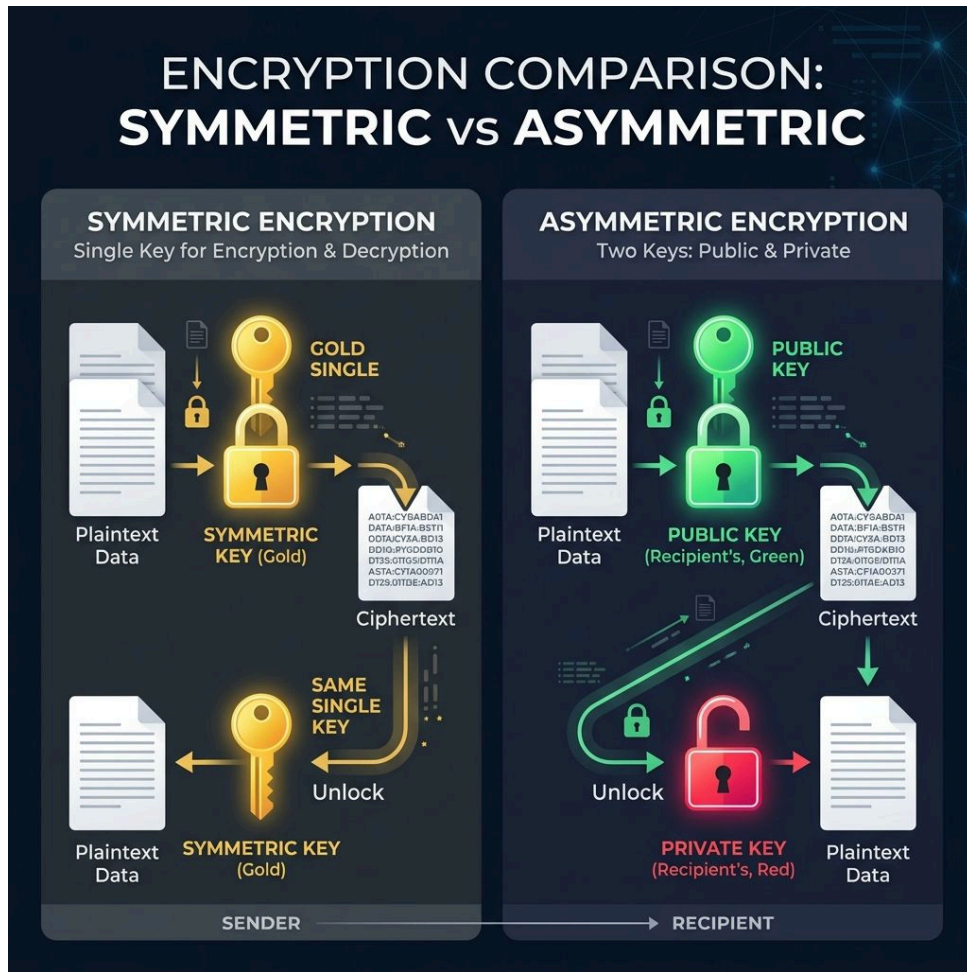


Figure 11.1 – Comparing the mechanics of Symmetric vs. Asymmetric encryption.

11.1.3. Asymmetric Encryption: The RSA Algorithm

Asymmetric encryption uses a pair of keys: a **public key** for encryption and a **private key** for decryption. The **RSA algorithm** is the primary example, relying on the mathematical complexity of factoring large prime numbers [cite: 750].

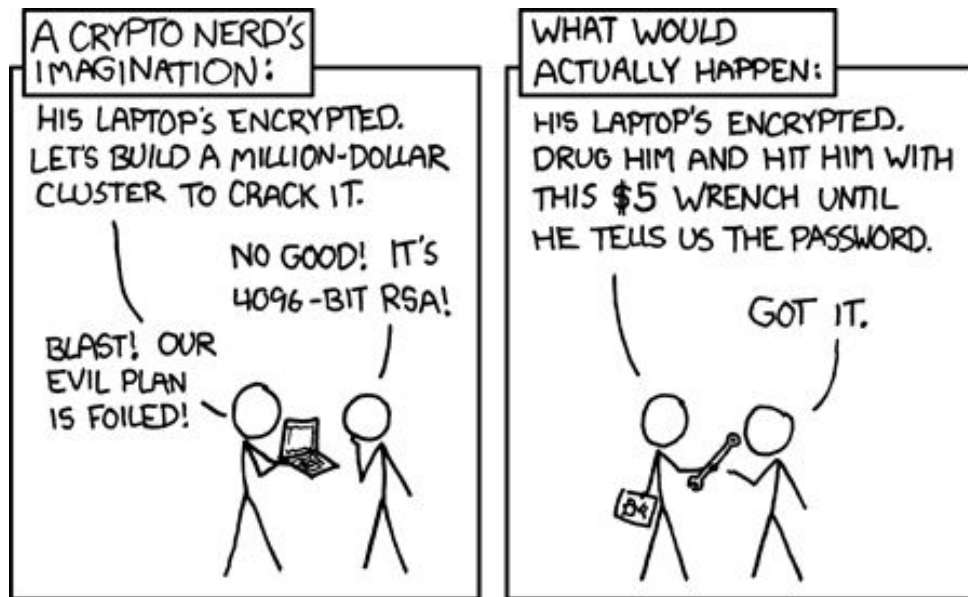


Figure 11.2 – The physical security layer is often the weakest link.²

11.1.4. Storage and Transfer Techniques

Encryption is often paired with other data management techniques:

- **Compression:** Reducing file size for efficient transfer.
- **Hashing:** Creating a unique fingerprint for data to ensure it hasn't been tampered with.
- **Checksums:** Simple error-detection methods to verify data integrity during transfer [cite: 751, 765].

²Source: xkcd.com/538



Video: How Encryption Works – A visual guide to symmetric and asymmetric encryption.

<https://www.youtube.com/watch?v=NuyzuNBFwxQ>

11.2. Encryption Algorithms and Analysis

Understanding the evolution of encryption provides insight into the complexity of modern security.

11.2.1. Classical Encryption Methods

Students analyze and evaluate the foundations of secret writing:

- **Caesar Cipher:** A simple substitution shift.
- **Polyalphabetic Ciphers:** Using multiple substitution alphabets (e.g., Vigenere and Gronsfeld).
- **One-time Pad:** Theoretically unbreakable when keys are truly random and never reused [cite: 762].

11.2.2. Modern Algorithmic Contexts

Encryption and data processing algorithms vary significantly based on their application:

- **Machine Learning & Deep Learning:** processing data patterns for prediction.
- **Reinforcement Learning:** Optimizing decisions through feedback loops [cite: 753].

11.2.3. Algorithmic Lifecycle

Students develop algorithmic solutions (using pseudocode) to manage the secure exchange of data. This involves predicting how inputs are processed and output at various stages, ensuring constructs like sequence and modularisation are applied effectively [cite: 769, 751].

11.3. Privacy, Ethics, and Data Management

Data exchange is not just a technical process; it is governed by legal and ethical frameworks that protect individuals.

11.3.1. Australian Privacy Principles (2014)

Students must understand the **Australian Privacy Principles (2014)**, which outline the standards for handling personally identifiable information (PII). This includes the right for individuals to know why their data is being collected and how it will be used [cite: 757].

11.3.2. Data Stewardship

Responsible data management involves:

- **Naming Conventions:** Using consistent and descriptive labels for data fields.
- **Logical Structures:** Organizing data in formats that ensure long-term usability and integrity [cite: 763].

11.3.3. Design for Users

The development of data exchange solutions is guided by:

- **Usability Principles:** Ensuring accessibility, effectiveness, safety, utility, and learnability [cite: 754].
- **Visual Communication:** Applying core elements and principles (like contrast and hierarchy) to create clear and helpful user interfaces [cite: 755].

11.4. Network Principles and Performance

The efficiency and reliability of data exchange depend on the underlying network architecture and protocols.

11.4.1. Performance Metrics

Students analyze network performance using three critical metrics:

- **Latency:** The time delay for a packet to travel across the network.
- **Jitter:** The variation in latency, which can cause jittery video or audio.
- **Guarantee of Delivery:** Ensuring that data reaches its destination without loss [cite: 758].



Example: Gaming vs. Streaming

- **Gaming:** High **Latency** (lag) is the biggest issue. Even a 200ms delay can make a game unplayable.
- **Streaming:** High **Jitter** is the biggest issue. If video packets arrive at irregular intervals, the video will stutter or buffer, even if the average speed is high.

11.4.2. Transmission Principles

Core principles govern how data moves through a network:

- **Packet Switching:** Breaking data into small packets for efficient routing.
- **Routing & Forwarding:** Determining the best path for data to travel.
- **Flow Control:** Managing the rate of data transfer to prevent network congestion [cite: 759].

11.4.3. Protocols and Standards

Standardized protocols ensure that disparate systems can communicate:

- **TCP/IP:** The fundamental suite of communication protocols for the internet.
- **HTTP/HTTPS:** For web traffic.
- **FTP:** For file transfers.
- **VPN:** For creating secure, encrypted tunnels over public networks [cite: 759].

11.4.4. Quality of Service (QoS)

Mechanisms for error detection and correction, alongside **Quality of Service (QoS)**, ensure that high-priority traffic (like video calls) is prioritized over less critical data [cite: 759].

11.4.5. The CIA Triad

Data security is evaluated through the **CIA Triad**:

- **Confidentiality:** Ensuring only authorized users can read the data.
- **Integrity:** Ensuring the data hasn't been changed.
- **Availability:** Ensuring the system is accessible when needed [cite: 760].



Video: Data Validation and Integrity – Practical techniques for ensuring your database remains accurate and reliable.

<https://www.youtube.com/watch?v=HXV3zeQKqGY>

11.5. Modern Data Exchange Methods

Modern digital solutions often involve the exchange of data between disparate systems using standardized formats and interfaces.

11.5.1. REST, JSON, and XML

Data exchange is facilitated through:

- **REST (Representational State Transfer):** An architectural style for networked applications.
- **JSON (JavaScript Object Notation):** A lightweight, human-readable data format.
- **XML (eXtensible Markup Language):** A flexible markup language for structured data [cite: 761].



Code Comparison: Data Interchange JSON (Compact):

```
{"user": "Alice", "id": 101}
```

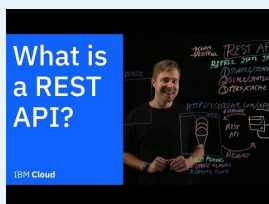
XML (Structured):

```
<data>
  <user>Alice</user>
  <id>101</id>
</data>
```

Analysis: JSON is often preferred for web APIs due to its smaller size and direct compatibility with JavaScript.

11.5.2. APIs

Application Programming Interfaces (APIs) act as a bridge, allowing different software systems to talk to each other without needing to know their internal implementation details [cite: 761].



Video: What is a REST API? – Standardizing data exchange between systems.

<https://www.youtube.com/watch?v=lsMQRaeKNDk>

11.5.3. Front-End and Back-End Collaboration

A complete digital solution consists of multiple sub-systems. Students symbolize the relationship between the **front end** (user interaction) and the **back end** (data processing and storage), showing how they collaborate to deliver a functional product [cite: 764].

11.5.4. Modeling with DFDs

Data Flow Diagrams (DFD) are used to represent the movement of data through these sub-systems, identifying clear relationships and data dependencies [cite: 768].

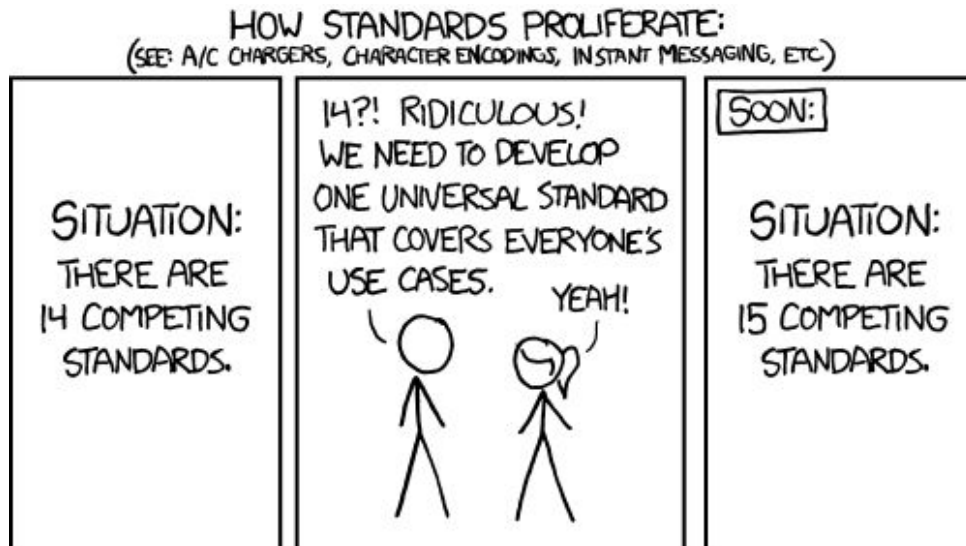


Figure 11.3 – The challenge of universal standards in data exchange!³

11.6. Revision Questions

1. Compare **Symmetric** and **Asymmetric** encryption. Provide an example algorithm for each.
2. Explain how **Two-Factor Authentication (2FA)** improves security compared to a single password.
3. List the three critical network performance metrics and define **Jitter**.
4. How do the **Australian Privacy Principles (2014)** impact the way developers handle user data?
5. Describe the role of a **REST API** in modern web development.
6. What is the **CIA Triad**, and why is it used to evaluate data security?

³Source: xkcd.com/927

Chapter 12

Complex Digital Data Exchange Problems

Table of contents

12.1. Problem Scoping and Variable Management	159
12.2. Security Risks and Data Integrity	160
12.3. Logical Modeling and Development Resources	161
12.4. Algorithmic Planning and Technical Documentation	163
12.5. Revision Questions	166

Aims and Objectives:

- Analyze scope, constraints, and risks in complex data exchange environments.
- Manage variable scope and decomposition for large-scale problems.
- Link DFD components to explain complex data processes.
- Evaluate inbuilt libraries and frameworks for solution suitability.
- Communicate technical ideas using professional language and mode-appropriate features.

12.1. Problem Scoping and Variable Management

The first step in addressing complex data exchange problems is establishing clear boundaries and managing the data's accessibility.

12.1.1. Defining the Environment

Students analyze complex problems to determine the specific **Scope**, **Constraints**, and **Limitations** of the data exchange environment. This ensures that the technical solution is tailored to the specific infrastructure and user needs [cite: 776, 777, 778].

12.1.2. Variable Scope: Local vs. Global

Managing data accessibility is critical for security and efficiency:

- **Local Variables:** Accessible only within the function or block where they are defined. This minimizes the risk of unintended data modification.
- **Global Variables:** Accessible throughout the entire program. While useful for shared state, they can lead to data integrity issues if not managed carefully [cite: 775].



Example: The Global Collision In a large exchange system, a developer uses a global variable `key = "123"` for encryption. Another developer, working on a different module, uses `key = "abc"` for a session ID. When the modules are combined, the encryption fails because the global `key` was overwritten. Using **Local Variables** would have prevented this collision.

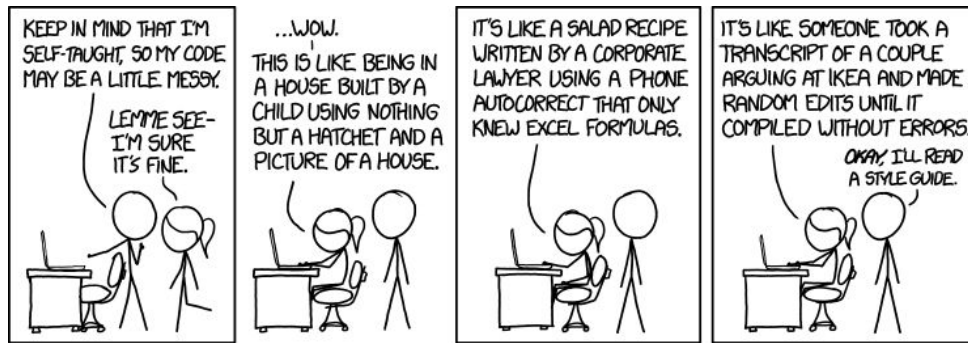


Figure 12.1 – Code quality is paramount in complex, multi-layered systems.¹

12.1.3. Decomposition and Data Sources

Complex problems are broken down into manageable aspects through **decomposition**. This involves identifying:

- **Risks & Storage Requirements:** Where data is stored and what could go wrong.
- **Data Interfaces:** How different sub-systems communicate.
- **Data Sources:** The specific sources (files, databases, APIs) required to generate the necessary data components [cite: 787, 789, 791, 792, 793].

12.2. Security Risks and Data Integrity

Protecting data throughout the exchange process requires a deep understanding of potential vulnerabilities and maintenance strategies.

12.2.1. The CIA Triad and Privacy

Students analyze factors affecting data security through the lens of the **CIA Triad**:

- **Confidentiality:** Preventing unauthorized access.
- **Integrity:** Preventing unauthorized modification.
- **Availability:** Ensuring reliable access for authorized users [cite: 759, 760, 781].

Privacy risks are identified and managed according to established principles (like the APP 2014), ensuring sensitive data remains protected during exchange [cite: 781].

12.2.2. Emerging Technologies and Integrity

The potential role of **emerging technologies**, such as machine learning for anomaly detection, is analyzed for its impact on data exchange security [cite: 784]. Throughout the process, data is refined to maintain:

- **Completeness:** Ensuring no data is lost.
- **Consistency:** Ensuring data is uniform across systems.
- **Integrity:** Ensuring data remains accurate and valid [cite: 785].

¹Source: xkcd.com/1513



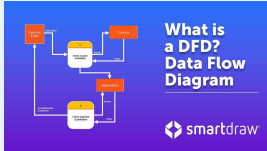
Video: Data Validation and Integrity – Practical techniques for ensuring your database remains accurate and reliable.
<https://www.youtube.com/watch?v=HXV3zeQKqGY>

12.3. Logical Modeling and Development Resources

Modeling the flow of data and evaluating available resources are essential for planning complex exchanges.

12.3.1. Data Flow Diagrams (DFDs)

Data Flow Diagrams (DFD) are developed to analyze and explain the system's data processes. By linking external entities, data sources, processes, and storage, DFDs provide a clear visual roadmap of how information moves through the environment [cite: 786].



Video: Data Flow Diagram Tutorial – Mapping information across systems.

https://www.youtube.com/watch?v=Jm9n2y4C_oQ



Example: Login DFD (Level 0)

- **External Entity:** User.
- **Data Flow:** Login Credentials.
- **Process:** Authenticate User.
- **Data Store:** User Database.
- **Result:** Access Token sent back to the User.

12.3.2. Evaluating Resources

Students must determine the best tools for their solution:

- **Inbuilt Libraries:** Analyzing existing code to determine necessary modularity and features [cite: 780, 782].
- **Frameworks & Tools:** Evaluating available code libraries and frameworks to determine their suitability for the identified problem [cite: 790].

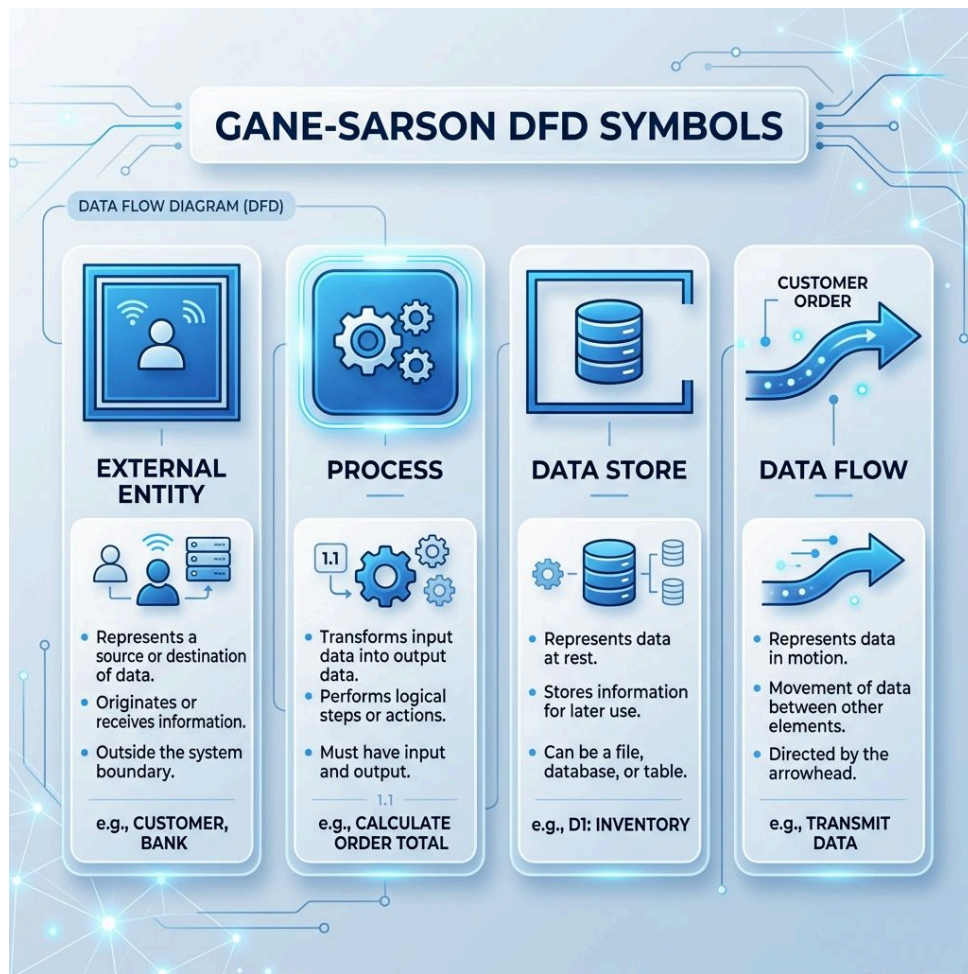


Figure 12.2 – Standard symbols for modeling data flow in complex exchange environments.

12.4. Algorithmic Planning and Technical Documentation

Before implementation, the logic of the data exchange must be meticulously planned and documented.

12.4.1. Success Criteria

Success criteria are determined to appraise the final implementation. For data exchange solutions, these focus on:

- **Protection & Security:** How well the data is shielded from threats.
- **System Interactions:** How efficiently the sub-systems communicate [cite: 783].

12.4.2. Developing the Logic

Detailed algorithmic steps are developed using **pseudocode**. This allows students to map out the exchange logic, including data validation and error handling, without being bogged down by syntax [cite: 794].

12.4.3. Annotation and Comments

Code comments and annotations are used to explain the purpose of specific code or algorithm statements. This documentation is vital for maintainability, especially in complex environments where multiple developers may collaborate [cite: 795].



Video: Pseudocode Explained – Using pseudocode to document the logic of a secure data exchange.

<https://www.youtube.com/watch?v=qfckDdsEIq8>



Joan Clarke was an English cryptanalyst and numismatist best known for her work as a code-breaker at Bletchley Park during the Second World War. Her work on the Enigma project that decrypted Nazi Germany's secret communications earned her awards and citations.²

12.4.4. Professional Communication and Presentation

Effectively conveying complex technical ideas requires mastery of specialized language and diverse communication modes.

12.4.5. Technical Language and Conventions

Information about the digital solution is conveyed using specialized technical language and appropriate language conventions. This ensures that ideas are communicated precisely to technical audiences [cite: 797, 798].



Example: Tailoring Communication

- **To an End-User:** “We use a secret code to keep your data safe.”
- **To a Technical Lead:** “We implemented a 256-bit AES encryption layer with a PBKDF2 key derivation function to ensure data-at-rest confidentiality.”

12.4.6. Ethical Scholarship and Referencing

To maintain ethical scholarship, students must use standardized referencing conventions when discussing data exchange techniques and research. This acknowledges the work of others and provides a foundation for the solution's claims [cite: 92, 799].

12.4.7. Mode-Appropriate Communication

Presenting data and ideas to a technical audience involves using a variety of modes:

- **Visual:** Sketches, diagrams, and wireframes.
- **Written:** Formal reports and technical documentation.
- **Spoken:** Presentations and technical walkthroughs [cite: 800, 801].

²Source: Hinsley, F. H., & Stripp, A. (Eds.). (1993). **Codebreakers: The Inside Story of Bletchley Park**. Oxford University Press.

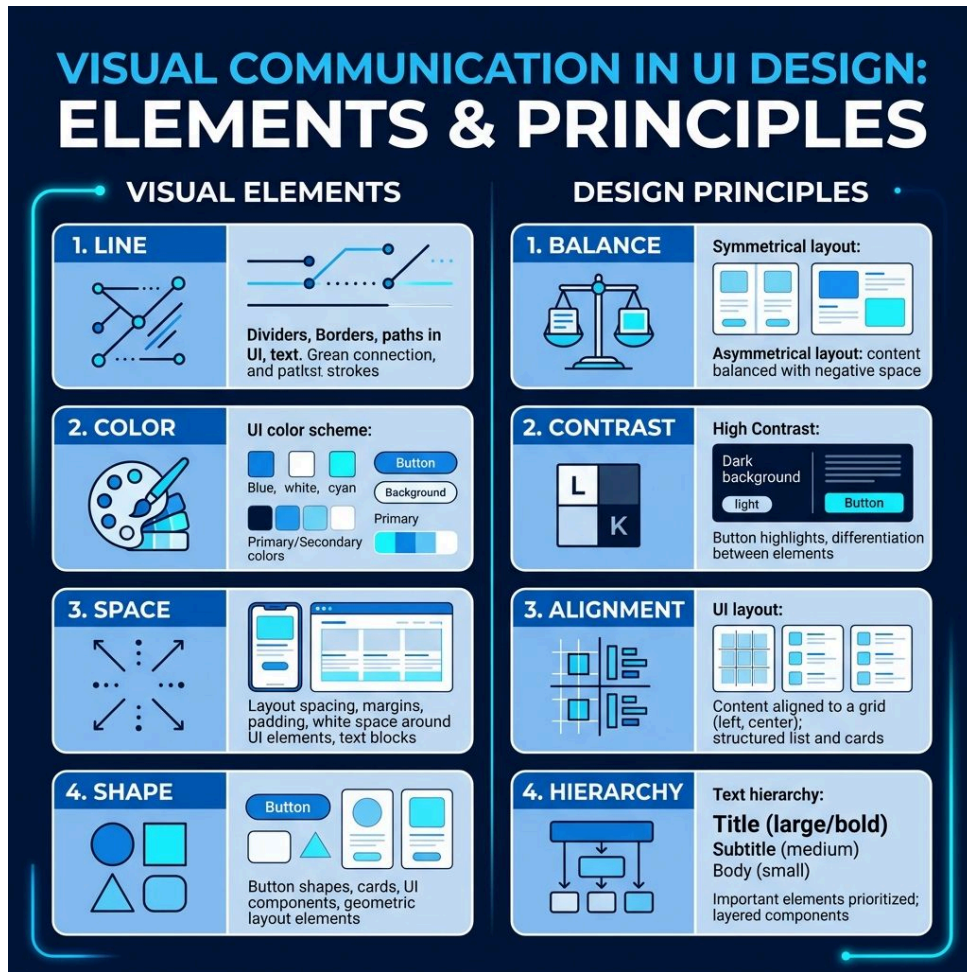


Figure 12.3 – Using diverse communication modes to present complex technical solutions.

12.5. Revision Questions

1. Contrast the scope of **local** and **global** variables. Why is local scope generally preferred?
2. How does **decomposition** help in managing the risks associated with data exchange?
3. Describe the four components of a **Data Flow Diagram (DFD)**.
4. Explain the role of **success criteria** in appraising a system's protection and security.
5. Why are **code comments** and annotations essential in a complex data exchange environment?
6. List three different communication modes used to present technical ideas to stakeholders.

Chapter 13

Prototype Digital Data Exchanges

Table of contents

13.1. Synthesising Secure Exchange Components	169
13.2. Implementing Encryption and Data Formats	170
13.3. Database Operations and Programmed Methods	172
13.4. Quality Assurance and Technical Testing	175
13.5. Evaluating Impacts and Security	176
13.6. Revision Questions	177

Aims and Objectives:

- Synthesise secure exchange components using core programming features.
- Implement encryption logic and standardized data formats (JSON/XML).
- Execute advanced SQL operations for backend data management.
- Apply coding standards and technical debugging to ensure quality.
- Evaluate the impact of AI and security recommendations against success criteria.

13.1. Synthesising Secure Exchange Components

The creation of a data exchange solution requires the seamless integration of ideas into functional prototype components [cite: 807].

13.1.1. Core Programming Features

Development is built upon a deep understanding of:

- **Variables & Control Structures:** Managing the logical flow and execution of the program [cite: 816-817].
- **Data Structures:** Organizing information (like arrays or lists) for effective processing [cite: 818].
- **Syntax & Libraries:** Adhering to the language rules and leveraging existing code classes to accelerate development [cite: 819-820].

13.1.2. Algorithmic Building Blocks

Implementation relies on basic algorithm constructs that students have mastered: **assignment**, **sequence**, **selection** (conditions), **iteration** (loops), and **modularisation** (functions) [cite: 808-814].

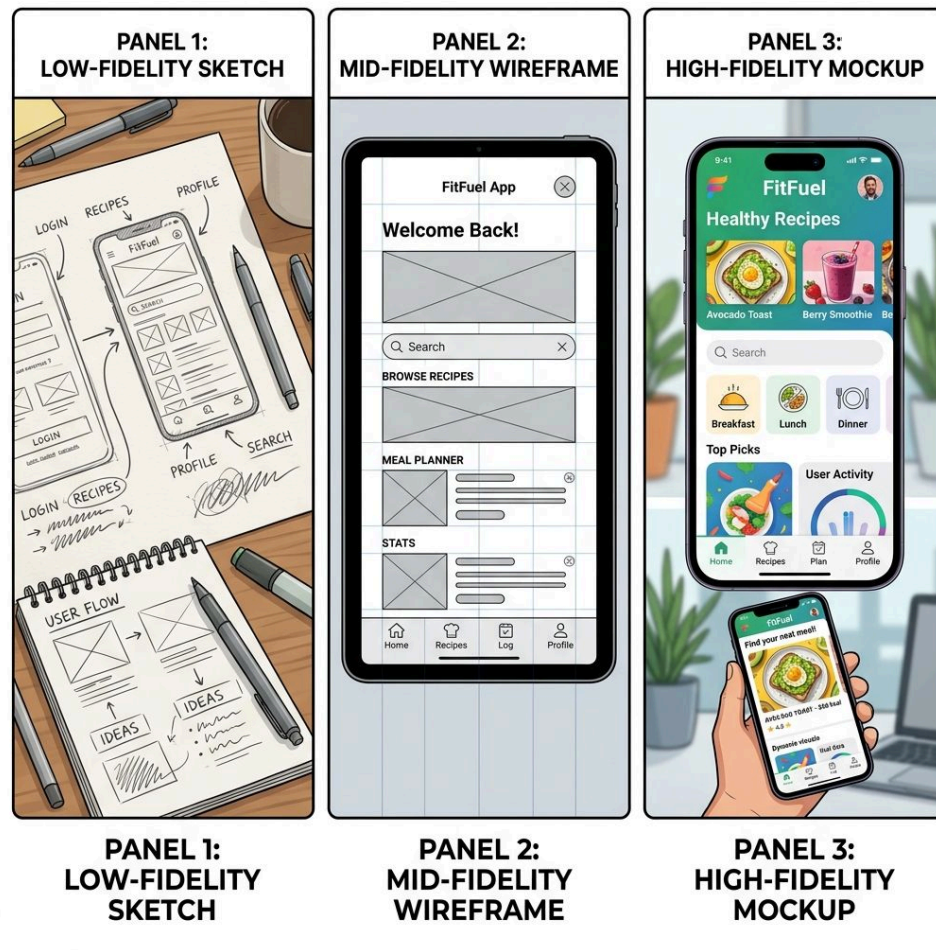


Figure 13.1 – Synthesizing diverse components into a coherent data exchange solution.

13.2. Implementing Encryption and Data Formats

Technical implementation involves the application of specific mathematical logic to protect and format data.

13.2.1. Encryption Logic

Students use **pseudocode** to develop algorithms for classical encryption methods, ensuring they understand the underlying shifts and substitutions:

- **Caesar Cipher**: Character-based shifting.
- **Polyalphabetic (Vigenere & Gronsfeld)**: Using multiple shift patterns.
- **One-time Pad**: Truly random, single-use keys [cite: 823].



Pseudocode: Caesar Cipher (Shift 3)

```

BEGIN CAESAR_ENCRYPT(plaintext, shift)
  ciphertext = ""
  FOR each character in plaintext
    IF character is a letter
      base = ASCII value of 'A'
      code = (ASCII value of character - base + shift) MOD 26
      ciphertext = ciphertext + CHAR(code + base)
    ELSE
      ciphertext = ciphertext + character
    ENDIF
  ENDFOR
  RETURN ciphertext
END

```

13.2.2. Data Interchange Formats

To ensure data can be exchanged between different systems, students generate solutions using standardized formats:

- **JSON (JavaScript Object Notation)**: For lightweight, attribute-value pairs.
- **XML (eXtensible Markup Language)**: For hierarchical, tag-based data [cite: 829].

13.2.3. External Data Handling

The programming environment must be configured to **receive** data from an external source, **process** it using the implemented logic, and **display** it in a user-friendly format [cite: 827, 830].



Video: How Encryption Works – A visual guide to symmetric and asymmetric encryption.

<https://www.youtube.com/watch?v=NuyzuNBFwxQ>

13.3. Database Operations and Programmed Methods

Managing the backend storage and logical processing of exchange data is critical for a functional prototype.

13.3.1. SQL for Data Exchange

Specific SQL statements are employed to manage the backend infrastructure:

- **Structure:** CREATE, DROP, and ALTER for managing tables.
- **Manipulation:** INSERT and UPDATE for managing stored rows [cite: 831-833].

Complex retrieval is handled through advanced SELECT clauses, including WHERE, GROUP BY, HAVING, ORDER BY, **sub-selection**, and **Inner Join** [cite: 834].



Advanced SQL: Inner Join To combine user data with their recent login activity:

```
SELECT Users.Username, Logins.LoginTime, Logins.IPAddress
FROM Users
INNER JOIN Logins ON Users.UserID = Logins.UserID
ORDER BY Logins.LoginTime DESC;
```

Analysis: This join links the Users table and Logins table using the shared UserID, allowing for a combined view of identity and activity.



Video: SQL Joins Explained – Khan Academy tutorial on complex data retrieval.

<https://www.youtube.com/watch?v=9yeOJ0ZMUYw>

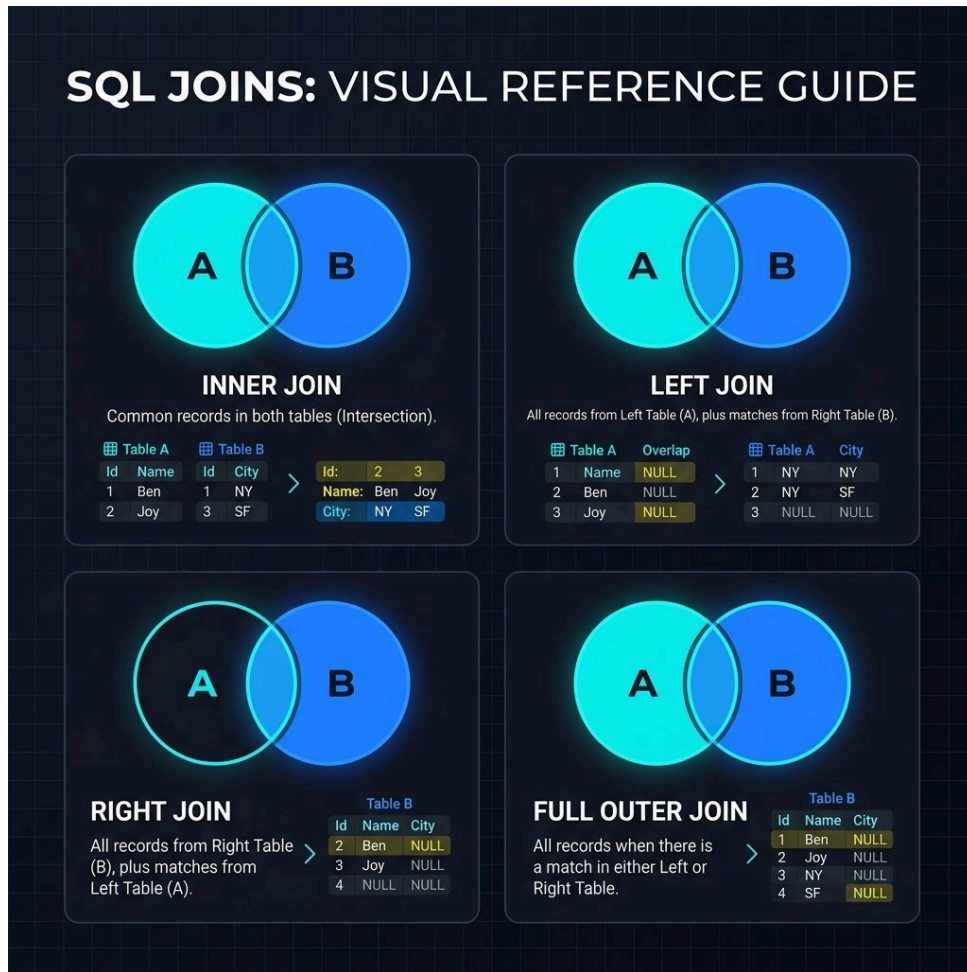


Figure 13.2 – Visualizing SQL Join operations using Venn diagrams.

13.3.2. Programmed Logic

The exchange scripts incorporate sophisticated logic:

- **Nested Conditions:** Using single and nested selection structures for complex decision-making during the exchange process [cite: 840].
- **Operators:** Applying arithmetic (including integer division and modulus), comparison, and logical operators to transform data [cite: 842].
- **Data Collections:** Using one-dimensional structures like **arrays** and **lists** to handle batches of exchange data [cite: 844].



Video: SQL Database Integrity – Using SQL to manage the robust backend of a data exchange system.

<https://www.youtube.com/watch?v=HXV3zeQKqGY>

13.4. Quality Assurance and Technical Testing

Rigorous testing and adherence to standards ensure the final prototype is robust and maintainable.

13.4.1. Coding Standards

Students must apply good programming practices throughout the build. This focus on **dependability**, **efficiency**, and consistent **naming conventions** ensures the solution is professional and easy for others to understand [cite: 821].



Example: Meaningful Naming

- **Poor:** `var a = data.get(1);`
- **Professional:** `var userEmail = userRecord.getField("email");`

Consistent naming reduces the “cognitive load” on anyone reading the code, making it significantly more **maintainable**.

13.4.2. Debugging Techniques

Before the final build, algorithmic steps are evaluated using **desk checks**. This computational thinking process allows students to predict outputs and identify logical errors early [cite: 845].

13.4.3. Refinement

The evaluation is continuous. Solutions are refined to improve their:

- **Accuracy:** Ensuring correct processing.
- **Reliability:** Ensuring stable performance.
- **Maintainability:** Ensuring code can be easily updated.
- **Usability:** Ensuring the system is user-friendly [cite: 850].

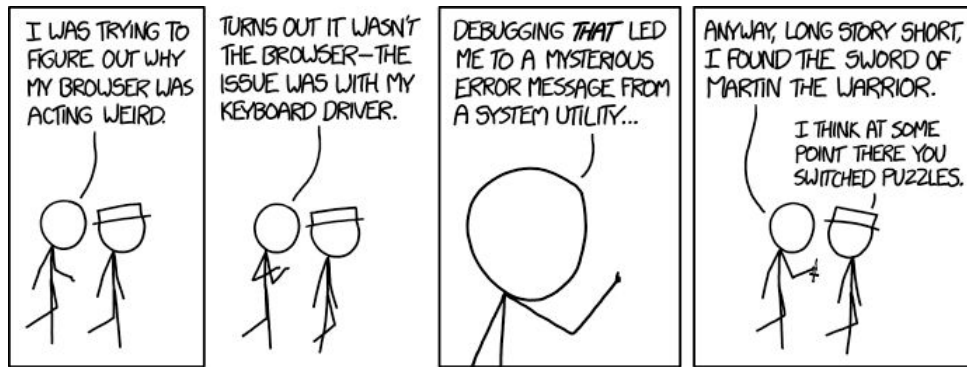


Figure 13.3 – Systematic debugging is essential for creating high-quality data exchange prototypes.¹

13.5. Evaluating Impacts and Security

The final prototype is appraised not only for its technical function but for its broader impact on society and security.

13.5.1. Data Ownership and AI

Students evaluate the personal, social, and economic impacts of how data is owned, stored, and shared. This includes analyzing the effects of **artificial intelligence (AI)** on data exchange and the potential risks to individual privacy [cite: 847, 848].



Margaret Hamilton led the team that developed the on-board flight software for NASA's Apollo program. She coined the term “software engineering” to give her team's work the same respect as other engineering disciplines. Her focus on “Error Detection and Recovery” was critical to the success of the moon landing.²

13.5.2. Security Recommendations

Based on rigorous testing, students make justified recommendations regarding security. They must consider how the solution's **interactivity** affects data sharing and identify potential vulnerabilities that require further protection [cite: 851].

13.5.3. Success Criteria Appraisal

The evaluation concludes with an appraisal against the original **success criteria**. This determines the effectiveness of the prototype and whether it has successfully addressed the identified data exchange problem [cite: 849].

¹Source: xkcd.com/1722

²Source: NASA History Office. **Margaret Hamilton: The Girl Who Taught Computers to Talk.**

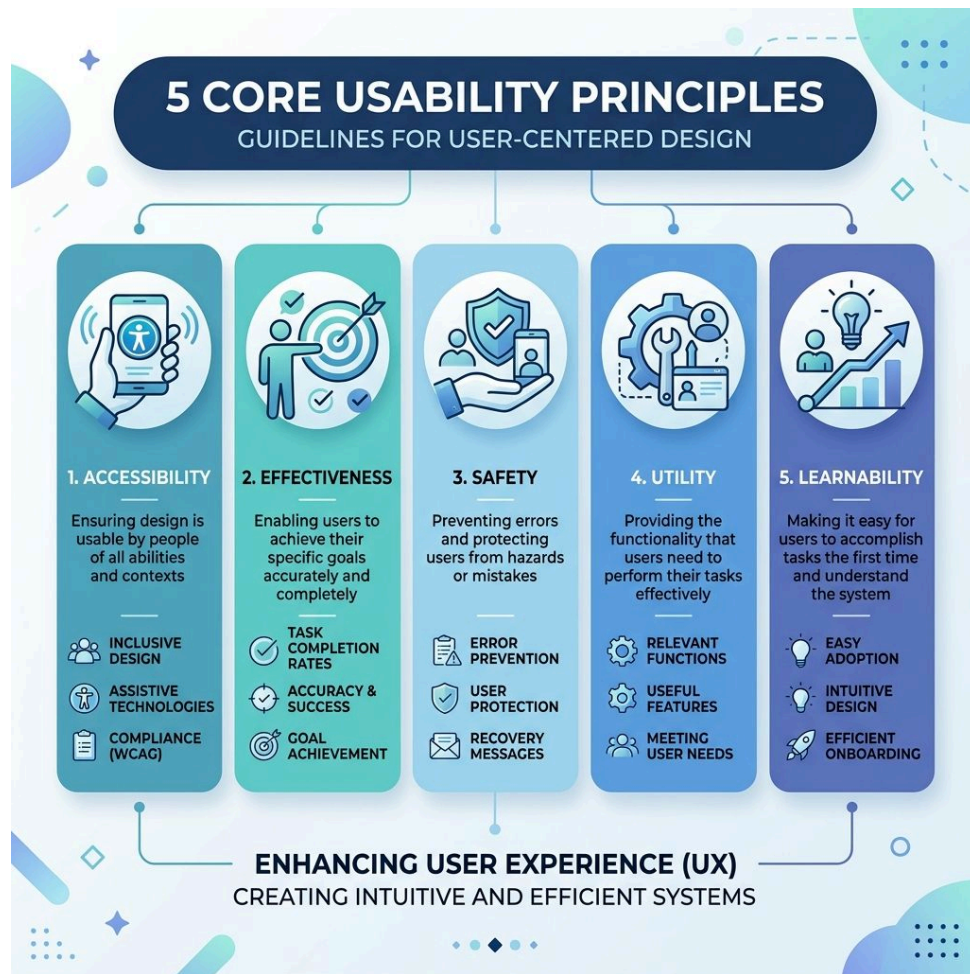


Figure 13.4 – Evaluating the final prototype against usability and security principles.

13.6. Revision Questions

1. Describe the role of **Data Collections** (like arrays and lists) in batch data exchange.
2. Provide the pseudocode logic for a simple **Caesar Cipher** shift.
3. Contrast the use of **JSON** and **XML** in modern web services.
4. Explain how **Desk Checks** help ensure the accuracy of exchange algorithms.
5. Evaluate the impact of **AI** on data privacy in a networked environment.
6. How are **Success Criteria** used to measure the effectiveness of a final prototype?

Chapter 14

Glossary

Abstraction: Filtering out unnecessary details to focus on the essential characteristics of a problem.

Accessibility: Ensuring a system can be used by people with the widest range of capabilities, regardless of disability.

Algorithm: A step-by-step set of operations to be performed to solve a specific problem.

Asymmetric Encryption: A type of encryption that uses two different but mathematically related keys: a public key and a private key.

Caesar Cipher: A basic substitution cipher where each letter in the plaintext is shifted a certain number of places down the alphabet.

Computational Thinking: A problem-solving methodology involving decomposition, abstraction, pattern recognition, and algorithm design.

Constraints: External factors that limit the design or development of a solution (e.g., technical, economic, legal).

Data Flow Diagram (DFD): A visual representation of the flow of data through an information system using Gane-Sarson notation.

Decomposition: Breaking down a complex problem or system into smaller, more manageable sub-problems.

DIKW Hierarchy: The conceptual model representing the transition from raw Data to Information, Knowledge, and finally Wisdom.

Effectiveness: A usability principle measuring how accurately and completely users can achieve their goals.

Functional Requirements: Statements describing what a system “shall” do in terms of its features and functions.

Hashing: The process of transforming data into a fixed-length unique fingerprint, often used for secure password storage.

Heuristic Review: An evaluation method where an interface is tested against established usability principles.

Innovation: The practical implementation of ideas that result in new or improved goods, services, or processes.

IPO Model: The “Input-Process-Output” framework used to describe the basic functional structure of a digital system.

JSON (JavaScript Object Notation): A lightweight, text-based data-interchange format that is easy for humans and machines to read.

Learnability: How easily users can accomplish basic tasks the first time they encounter a design.

Limitations: Inherent boundaries of a chosen technology, such as range, battery life, or processing speed.

Modularisation: The practice of breaking a program into independent, reusable units like functions and procedures.

Normalisation: The process of structuring a relational database to reduce redundancy and reach Third Normal Form (3NF).

Primary Key: A unique identifier for every record within a relational database table.

Pseudocode: A formal, language-independent method of describing algorithmic logic using structured keywords.

Scope: A clearly defined boundary of what a digital solution will and will not do.

SQL (Structured Query Language): The standard domain-specific language used for managing and querying relational databases.

Success Criteria: Measurable (SMART) standards used to evaluate whether a final prototype has solved the identified problem.

Symmetric Encryption: A type of encryption where the same secret key is used for both encryption and decryption.

User Experience (UX): The holistic view of a person's emotions, attitudes, and satisfaction when interacting with a digital product.

Wireframe: A low-fidelity visual representation of a user interface, serving as a blueprint for layout and navigation.

XML (eXtensible Markup Language): A hierarchical, tag-based markup language used for encoding and exchanging data between systems.

Chapter 15

Index

A

Abstraction	16
Accessibility	5, 40
Aggregate Functions	109
Algorithm Design	16
Anomalies	101
API	158
Assignment	59
Asymmetric Encryption	10, 153
Australian Privacy Act	93

B

Berezin, Evelyn	141
Booth, Kathleen	68
Borg, Anita	118

C

Caesar Cipher	173
Clarke, Joan	168
Constants	68
Constraints	106, 128, 162
Cooper, Alan	106
CRUD	110

D

Data Definition Language (DDL)	143
Data Flow Diagrams (DFD)	90, 164

Data Integrity	100
Data Manipulation Language (DML)	143
Data Privacy	93
Data Retrieval	109
Data Security	100
Data Sovereignty	93
Data Types	106
Data Validation	6, 65, 101
Data Verification	101
Database	87
De-identification	101
Debugging	73, 134
Deep Learning	118
Denning, Dorothy	152
Desk Checks	73
DIKW Hierarchy	83

E

Effectiveness	5, 41
Exponentiation	68

F

Feinler, Elizabeth	84
Foreign Key (FK)	87
Functional Requirements	128

G

Gane-Sarson notation	90
GDPR	93
Global Variables	69

H

Hamilton, Margaret	66, 179
Hashing	154
Heuristic Reviews	122
Hopper, Grace	33

I

Inner Join	175
Inner-Joins	109
Innovation	118
Input	20

Intelligent Systems	8, 121
Interactive Media	8, 121
Internet of Things (IoT)	21
IPO model	132
Iteration	59

J

Jitter	156
JSON	73, 174

K

Kare, Susan	47
-------------	----

L

Latency	156
Learnability	6, 43
Limitations	128, 162
Local Variables	68
Logical Operators	68
Lovelace, Ada	66

M

Machine Learning (ML)	30
Mobile Applications	8, 21, 121
Modular Programming	70
Modularisation	59
Modulus	68

N

Natural Language Processing (NLP)	118
Neural Networks	30
Non-functional Requirements	128
Normalisation	102

O

Output	20
--------	----

P

Pattern Recognition	16
Perlman, Radia	136

Primary Key (PK)	87
Process	20
Pseudocode	61

R

Redundancy	101
Relational Schema	101
RSA	10, 153

S

Safety	5, 41
Scope	162
Selection	59
Sequence	59
Spärck Jones, Karen	103
SQL	65
Sub-queries	109
Success Criteria	106, 128
Symmetric Encryption	10, 152
System Interactions	166
Systems thinking	144

T

Third Normal Form (3NF)	102
-------------------------	-----

U

User Experience (UX)	27
User Interface (UI)	110
User Personas	38
Utility	6, 43

W

WCAG	40
Web Applications	8, 120
Wireframes	140

X

XML	174
-----	-----